

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC KINH TẾ TP HỒ CHÍ MINH (UEH)
TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ

Thiếu tệp hình: resources/Logo_UEH_xanh.png

ĐỒ ÁN MÔN HỌC
ĐỀ TÀI
XÂY DỰNG CHƯƠNG TRÌNH DESKTOP
QUẢN LÝ PHÒNG KHÁM ĐA VAI TRÒ

Học Phần: DESKTOP

Danh Sách Nhóm:

Thiếu tệp hình: resources/khung_pdf.pdf

1	LÊ MINH ANH	– 31241023293
2	NGUYỄN ĐÌNH THẢO NHI	– 31241026988
3	TỪ TRƯỞNG THANH TRÚC	– 31241023563
4	ĐINH THỊ ANH THƯ	– 31241026994
5	LỮ VÕ HOÀNG PHÚC	– 31241025389

Chuyên Ngành: KỸ THUẬT PHẦN MỀM

Khóa: K50

Giảng Viên: LÊ HỮU THANH TÙNG

Tp. Hồ Chí Minh, Ngày 05 tháng 06 năm 2026

Mục lục

TÓM TẮT

LỜI CẢM ƠN

DANH MỤC THUẬT NGỮ VÀ KÝ HIỆU

1	Mở đầu	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu nghiên cứu	1
1.3	Phạm vi đề tài	2
1.4	Phương pháp thực hiện	2
2	Cơ sở lý thuyết và Công nghệ sử dụng	2
2.1	Nền tảng lập trình C# và Windows Forms	2
2.2	Các kỹ thuật WinForms được áp dụng trong đồ án	3
2.2.1	Lập trình hướng sự kiện	3
2.2.2	Partial class và Designer	4
2.2.3	Form, UserControl và mô hình màn hình con	4
2.2.4	Dock, Anchor và bố cục co giãn	5
2.2.5	DataGridView và binding dữ liệu	5
2.2.6	Form modal và hộp thoại nghiệp vụ	6
2.2.7	Custom control và card nghiệp vụ	6
2.3	Mô hình điều hướng đa vai trò trong WinForms	7
2.4	Thiết kế trải nghiệm Desktop theo từng vai trò	7
2.5	Mô hình kiến trúc nhiều lớp	8
2.6	Tầng DTO, BUS và DAL	9
2.7	ADO.NET và SQL Server	9
2.8	Entity Framework và định hướng ORM	10
2.8.1	Định hướng thiết kế DAL bằng Entity Framework	10
2.9	API, Supabase, SheetDB và PayOS	11
2.9.1	Vai trò của ứng dụng trung gian trong tích hợp API	12
2.9.2	Mô hình dữ liệu SheetDB và Supabase	12
2.9.3	Luồng đồng bộ Supabase, SheetDB và SQL Server	13
2.9.4	Đồng bộ request cận lâm sàng từ Supabase	15
2.9.5	Luồng PayOS, VietQR, MoMo QR và webhook	15
3	Phân tích và Thiết kế hệ thống	17
3.1	Tác nhân sử dụng hệ thống	17
3.2	Yêu cầu chức năng	17

3.3	Đặc tả use case chính	18
3.3.1	UC-01: Đăng nhập hệ thống	18
3.3.2	UC-02: Tạo lịch hẹn khám	19
3.3.3	UC-03: Tiếp nhận bệnh nhân và đưa vào hàng chờ	19
3.3.4	UC-04: Khám bệnh và cập nhật ERM	20
3.3.5	UC-05: Xử lý yêu cầu xét nghiệm hoặc hình ảnh	20
3.3.6	UC-06: Cấp phát thuốc và thanh toán	21
3.4	Quy trình nghiệp vụ tổng quát	21
3.5	Yêu cầu phi chức năng	22
3.6	Thiết kế cơ sở dữ liệu	22
3.7	Quan hệ dữ liệu	23
3.8	Thiết kế từ điển dữ liệu tiêu biểu	23
3.8.1	Nhóm bảng phân quyền và nhân sự	24
3.8.2	Nhóm bảng bệnh nhân, lịch hẹn và hàng chờ	24
3.8.3	Nhóm bảng khám bệnh, cận lâm sàng và hồ sơ	25
3.8.4	Nhóm bảng dược và thanh toán	26
4	Xây dựng và Cài đặt chương trình	26
4.1	Tổ chức project và kiến trúc triển khai	26
4.1.1	Tổ chức project trong Visual Studio	26
4.1.2	Phân tích cấu trúc thư mục WinForms	27
4.1.3	Quy ước thiết kế form và control	28
4.1.4	Luồng dữ liệu từ UI đến SQL Server	29
4.1.5	Cài đặt vòng đời ứng dụng WinForms	30
4.1.6	Cài đặt dashboard shell và vùng nội dung động	30
4.1.7	Cài đặt UserControl theo phân hệ	31
4.1.8	Cài đặt DataGridView cho nghiệp vụ quản lý	32
4.1.9	Cài đặt FlowLayoutPanel và card động	33
4.1.10	Cài đặt form modal, OpenFileDialog và MessageBox	33
4.1.11	Cài đặt bất đồng bộ cho API và thanh toán	34
4.1.12	Quản lý session và truyền ngữ cảnh người dùng	34
4.1.13	Cài đặt điểm khởi động và phân quyền	34
4.2	Cài đặt các tầng DTO, DAL, BUS và Core	35
4.2.1	Cài đặt tầng DTO	35
4.2.2	Cài đặt tầng DAL bằng ADO.NET	37
4.2.3	Cài đặt tầng DAL bằng Entity Framework theo hướng mở rộng	39
4.2.4	Cài đặt tầng BUS	41
4.2.5	Cài đặt tầng Core, Constants và Session	46
4.3	Thiết kế giao diện người dùng	47
4.3.1	Vai trò của Controller trong WinForms	47
4.3.2	Thiết kế DataGridView trong màn hình thanh toán	48

4.3.3	Thiết kế FlowLayoutPanel cho danh sách động	49
4.3.4	Thiết kế UserControl dùng chung và kế thừa view base	49
4.3.5	Thiết kế trạng thái nút và phản hồi người dùng	50
4.3.6	Cài đặt main form shell theo vai trò	50
4.3.7	Phân tách Designer và code-behind	50
4.3.8	Giao diện đăng nhập	51
4.3.9	Giao diện lễ tân	52
4.3.10	Giao diện bác sĩ và ERM	53
4.3.11	Giao diện điều dưỡng	54
4.3.12	Giao diện kỹ thuật viên	55
4.3.13	Giao diện dược sĩ	56
4.3.14	Cài đặt lịch làm việc dùng chung	56
4.4	Tích hợp tính năng phụ trợ	57
4.4.1	Đồng bộ API Supabase và SheetDB	57
4.4.2	Kéo request xét nghiệm và hình ảnh về màn hình kỹ thuật viên	59
4.4.3	Tích hợp thanh toán PayOS	59
4.5	Tổng hợp luồng cài đặt theo nghiệp vụ	61
4.6	Phân rã chi tiết các luồng vận hành	62
4.6.1	Luồng đăng nhập và mở dashboard	62
4.6.2	Luồng tạo lịch hẹn	64
4.6.3	Luồng check-in và đưa bệnh nhân vào hàng chờ	65
4.6.4	Luồng bác sĩ mở ERM và xử lý hồ sơ	66
4.6.5	Luồng tạo chỉ định và kỹ thuật viên xử lý request	67
4.6.6	Luồng cấp phát thuốc và cập nhật tồn kho	68
4.6.7	Luồng đồng bộ Supabase, SheetDB và SQL Server	69
4.6.8	Luồng PayOS, VietQR và cập nhật realtime	70
4.7	Đánh giá kỹ thuật cài đặt trong chương xây dựng	72
5	Kiểm thử và Giao diện kết quả	72
5.1	Chiến lược kiểm thử	72
5.2	Kiểm thử giao diện WinForms	72
5.3	Kiểm thử chức năng	73
5.4	Đánh giá ưu điểm	74
5.5	Hạn chế	74
6	Kết luận và Hướng phát triển	75
6.1	Kết quả đạt được	75
6.2	Hạn chế của phần mềm	75
6.3	Hướng phát triển trong tương lai	75
	Tài liệu tham khảo	76

Phụ lục A. Hình ảnh	77
Phụ lục B. Các thuật toán	78
Phụ lục C. Bảng, biểu	79
Bảng phân công nhiệm vụ	81
Link đồ án	81

TÓM TẮT

Đồ án xây dựng hệ thống quản lý phòng khám dưới dạng ứng dụng Desktop, triển khai bằng C# WinForms và SQL Server. Hệ thống hướng đến mô hình phòng khám ngoại trú, nơi các nghiệp vụ tiếp nhận, khám bệnh, xét nghiệm, chẩn đoán hình ảnh, cấp phát thuốc, thanh toán và theo dõi hồ sơ bệnh án cần được xử lý liên tục bởi nhiều nhóm người dùng khác nhau. Mục tiêu nghiên cứu của đề tài là thiết kế và hiện thực một mô hình phần mềm có khả năng phân quyền theo vai trò, lưu trữ dữ liệu tập trung, giảm thao tác thủ công và mô phỏng quy trình vận hành thực tế trong môi trường phòng khám.

Về phương pháp thực hiện, nhóm phân tích yêu cầu nghiệp vụ từ quy trình khám ngoại trú, thiết kế cơ sở dữ liệu quan hệ trên SQL Server, tổ chức mã nguồn theo kiến trúc nhiều lớp DTO–BUS–DAL và xây dựng giao diện WinForms theo từng vai trò người dùng. Mã nguồn hiện được chia thành các project ClinicManagementSystem.WinForms, BUS, DAL, DTO, Models, Core và ServicesExternal. Cách tổ chức này giúp tách biệt giao diện, xử lý nghiệp vụ, truy cập dữ liệu, mô hình dữ liệu, hằng số hệ thống và tích hợp dịch vụ ngoài.

Kết quả đạt được là một hệ thống có các phân hệ chính: đăng nhập và điều hướng theo vai trò; quản lý bệnh nhân; quản lý lịch khám và hàng chờ; quản lý hồ sơ bệnh án điện tử; xử lý sinh hiệu, đơn thuốc, xét nghiệm và chẩn đoán hình ảnh; quản lý thanh toán; quản lý nhà thuốc; quản lý ca làm; và đồng bộ dữ liệu với nguồn ngoài. Bên cạnh đó, hệ thống có mô phỏng tích hợp PayOS và các lớp hỗ trợ chuyển đổi dữ liệu ảnh base64 phục vụ thanh toán hoặc lưu trữ tài nguyên y tế.

Từ khóa: Clinic Management System, C# WinForms, SQL Server, 3-Tier Architecture, ADO.NET, Electronic Medical Record, Role-Based Access Control.

LỜI CẢM ƠN

Nhóm thực hiện xin gửi lời cảm ơn đến giảng viên hướng dẫn đã hỗ trợ định hướng đề tài, góp ý về phạm vi nghiệp vụ và yêu cầu kỹ thuật trong quá trình xây dựng ứng dụng Desktop quản lý phòng khám. Những góp ý này giúp nhóm tổ chức lại quy trình phân tích, thiết kế cơ sở dữ liệu, phân lớp mã nguồn và kiểm thử chức năng theo hướng có hệ thống hơn.

Nhóm cũng cảm ơn các thành viên đã phối hợp trong quá trình tìm hiểu nghiệp vụ, thiết kế giao diện, xây dựng tầng xử lý dữ liệu, kiểm tra lỗi và hoàn thiện báo cáo. Do phạm vi đồ án bao gồm nhiều vai trò người dùng và nhiều phân hệ nghiệp vụ, việc phân chia công việc và tổng hợp kết quả là yếu tố quan trọng để hoàn thành sản phẩm.

Nhóm thực hiện

DANH MỤC THUẬT NGỮ VÀ KÝ HIỆU

Phần này tổng hợp các thuật ngữ kỹ thuật và nghiệp vụ được sử dụng trong báo cáo.

Thuật ngữ chuyên ngành

Thuật ngữ	Diễn giải ngắn gọn
CMS	<i>Clinic Management System</i> : hệ thống quản lý phòng khám, bao gồm dữ liệu bệnh nhân, lịch khám, hồ sơ, thanh toán và nhân sự.
WinForms	Công nghệ xây dựng giao diện Desktop của .NET, được sử dụng để tạo các Form và UserControl trong đồ án.
DTO	<i>Data Transfer Object</i> : đối tượng truyền dữ liệu giữa giao diện, tầng nghiệp vụ và tầng truy cập dữ liệu.
BUS	<i>Business Logic Layer</i> : tầng xử lý nghiệp vụ, kiểm tra điều kiện hợp lệ và điều phối truy cập dữ liệu.
DAL	<i>Data Access Layer</i> : tầng truy cập cơ sở dữ liệu, thực hiện truy vấn SQL và ánh xạ dữ liệu sang DTO.
Repository	Lớp chịu trách nhiệm thao tác dữ liệu cho một nhóm nghiệp vụ cụ thể, ví dụ bệnh nhân, lịch hẹn, thanh toán.
RBAC	<i>Role-Based Access Control</i> : cơ chế phân quyền dựa trên vai trò người dùng như Admin, Receptionist, Doctor, Nurse, Pharmacist, Technician.
ERM/EMR	Hồ sơ bệnh án điện tử, lưu thông tin bệnh nhân, lịch sử khám, sinh hiệu, xét nghiệm, hình ảnh, đơn thuốc và hóa đơn.
Encounter	Một lượt khám hoặc lần tiếp xúc y tế giữa bệnh nhân và bác sĩ trong hệ thống.
Appointment	Lịch hẹn khám của bệnh nhân với bác sĩ, khoa và phòng khám cụ thể.
Patient Queue	Hàng chờ bệnh nhân, thể hiện trạng thái chờ khám, đang khám, hoàn tất hoặc hủy.
PayOS	Cổng thanh toán được hệ thống mô phỏng tích hợp để hỗ trợ nghiệp vụ thanh toán và QR.
Supabase/SheetDB	Nguồn dữ liệu ngoài được mô phỏng trong phân hệ đồng bộ API.

Ký hiệu và quy ước viết tắt

Ký hiệu	Ý nghĩa
<i>U</i>	Tập người dùng trong hệ thống.
<i>R</i>	Tập vai trò người dùng, gồm Admin, Receptionist, Doctor, Nurse, Pharmacist và Technician.
<i>P</i>	Tập bệnh nhân được lưu trong bảng Patients.
<i>A</i>	Tập lịch hẹn khám trong bảng Appointments.
<i>E</i>	Tập lượt khám trong bảng Encounters.
<i>Q</i>	Tập hàng chờ bệnh nhân trong bảng PatientQueues.
<i>S</i>	Tập dịch vụ lâm sàng hoặc cận lâm sàng trong bảng Services.
<i>M</i>	Tập thuốc trong bảng Medicines.
<i>I</i>	Tập hóa đơn hoặc thanh toán trong nhóm bảng Invoices, Payments.

1 Mở đầu

1.1 Lý do chọn đề tài

Trong bối cảnh tin học hóa hoạt động y tế, các phòng khám ngoại trú không chỉ cần lưu trữ hồ sơ bệnh nhân mà còn phải phối hợp nhiều nghiệp vụ liên tiếp như đặt lịch, tiếp nhận, phân phòng, khám bệnh, chỉ định xét nghiệm, trả kết quả, kê toa, cấp phát thuốc, thanh toán và theo dõi tái khám. Nếu các bước này được thực hiện bằng giấy tờ hoặc nhiều bảng tính rời rạc, dữ liệu dễ bị trùng lặp, thiếu thống nhất và khó truy vết khi cần xem lại lịch sử điều trị.

Đề tài **“Xây dựng chương trình Desktop quản lý phòng khám đa vai trò”** được lựa chọn nhằm mô phỏng một hệ thống quản lý phòng khám có cấu trúc gần với thực tế. Hệ thống hướng đến môi trường triển khai nội bộ, nơi các máy tính nghiệp vụ được sử dụng bởi lễ tân, bác sĩ, điều dưỡng, kỹ thuật viên, dược sĩ và quản trị viên. Với đặc thù này, ứng dụng Desktop bằng WinForms là lựa chọn phù hợp vì dễ thao tác, phản hồi nhanh và thuận tiện triển khai trong mạng nội bộ.

Về mặt kỹ thuật, đề án cho phép vận dụng các kiến thức về lập trình hướng đối tượng, thiết kế cơ sở dữ liệu quan hệ, kiến trúc phân lớp, phân quyền người dùng, xử lý dữ liệu y tế và tích hợp API. Đây là bài toán có nhiều thực thể liên quan, nhiều trạng thái nghiệp vụ và nhiều luồng xử lý, do đó phù hợp để đánh giá khả năng phân tích, thiết kế và hiện thực phần mềm quản lý.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề án là xây dựng một ứng dụng Desktop hỗ trợ quản lý quy trình khám chữa bệnh ngoại trú trong phòng khám, đảm bảo dữ liệu được lưu trữ tập trung, người dùng được phân quyền theo vai trò và các nghiệp vụ chính được xử lý theo quy trình thống nhất.

Các mục tiêu cụ thể gồm:

- Xây dựng ứng dụng C# WinForms sử dụng SQL Server làm cơ sở dữ liệu chính.
- Thiết kế kiến trúc nhiều lớp gồm UI, Controller, BUS, DAL, DTO, Models, Core và ServicesExternal.
- Xây dựng cơ chế đăng nhập, lưu phiên làm việc và điều hướng giao diện theo vai trò.
- Triển khai các phân hệ quản lý bệnh nhân, lịch hẹn, hàng chờ, hồ sơ bệnh án, sinh hiệu, xét nghiệm, hình ảnh, đơn thuốc, thanh toán, nhà thuốc và ca làm.
- Mô phỏng đồng bộ dữ liệu ngoài qua Supabase/SheetDB và tích hợp thanh toán PayOS.
- Đánh giá mức độ đáp ứng chức năng, ưu điểm, hạn chế và hướng phát triển của phần mềm.

1.3 Phạm vi đề tài

Phạm vi nghiệp vụ của hệ thống tập trung vào phòng khám ngoại trú. Hệ thống không triển khai đầy đủ mô hình bệnh viện nội trú, quản lý giường bệnh, bảo hiểm y tế chuyên sâu hoặc hệ thống PACS đạt chuẩn bệnh viện. Các chức năng cận lâm sàng như xét nghiệm, MRI/X-Ray và PDF kết quả được mô phỏng ở mức lưu trữ dữ liệu, upload tệp và hiển thị trong hồ sơ bệnh án điện tử.

Phạm vi kỹ thuật dựa trên mã nguồn hiện có trong thư mục `ClinicManagementSystem.WinForms`. Solution chính gồm các project `ClinicManagementSystem.WinForms`, `BUS`, `DAL`, `DTO`, `Models`, `Core`, `ServicesExternal` và script cơ sở dữ liệu `Database/CMS.sql`. Một số thư mục EF tồn tại ngoài solution chính, nhưng trong project WinForms đang khảo sát, thao tác dữ liệu chủ yếu được triển khai theo hướng ADO.NET với `Microsoft.Data.SqlClient`.

1.4 Phương pháp thực hiện

Nhóm áp dụng phương pháp phát triển theo hướng phân tích yêu cầu trước, sau đó thiết kế cơ sở dữ liệu, thiết kế kiến trúc, hiện thực từng phân hệ và kiểm thử chức năng. Các bước chính:

1. Khảo sát quy trình khám ngoại trú và xác định các vai trò người dùng.
2. Phân rã nghiệp vụ thành các module độc lập như tiếp nhận, khám bệnh, kỹ thuật viên, nhà thuốc và thanh toán.
3. Thiết kế schema SQL Server theo nhóm bảng người dùng, bệnh nhân, lịch hẹn, encounter, cận lâm sàng, dược, thanh toán và ca làm.
4. Hiện thực tầng DTO, BUS, DAL và giao diện WinForms theo từng module.
5. Kiểm thử luồng đăng nhập, tạo lịch hẹn, tiếp nhận, xử lý hàng chờ, xem ERM, thanh toán và cấp phát thuốc.

2 Cơ sở lý thuyết và Công nghệ sử dụng

2.1 Nền tảng lập trình C# và Windows Forms

C# là ngôn ngữ lập trình hướng đối tượng trên nền tảng .NET, phù hợp để xây dựng ứng dụng quản lý có nhiều thực thể nghiệp vụ và yêu cầu bảo trì dài hạn. Trong hệ thống quản lý phòng khám, các khái niệm như bệnh nhân, nhân viên, lịch hẹn, phòng khám, ca làm, đơn thuốc và hóa đơn đều có thể được mô hình hóa bằng lớp dữ liệu hoặc DTO.

Windows Forms là công nghệ xây dựng giao diện Desktop của .NET. Ứng dụng WinForms hoạt động theo mô hình sự kiện: người dùng nhấn nút, chọn dòng, nhập thông tin hoặc mở form, sau đó chương trình xử lý event tương ứng. Cách tiếp cận này phù hợp với quy trình

vận hành phòng khám vì nhân viên thường thao tác qua các form nhập liệu, bảng danh sách, nút xác nhận và hộp thoại chi tiết.

Trong project, giao diện được chia thành:

- Mainforms: form chính theo vai trò như Admin, Receptionist, Doctor, Nurse, Pharmacy và Technician.
- UserControls: màn hình chức năng được nạp vào panel nội dung của main form.
- Forms: các popup, form đăng nhập hoặc workflow độc lập.
- Shareforms: thành phần dùng chung như ERM và WorkingShifts.
- Controllers: lớp điều phối giữa UI và BUS để giảm logic trong code-behind.

2.2 Các kỹ thuật WinForms được áp dụng trong đồ án

Vì đây là đồ án môn thiết kế ứng dụng Desktop bằng WinForms, phần giao diện không chỉ dừng ở việc kéo thả control trên Designer. Project sử dụng nhiều kỹ thuật đặc trưng của WinForms để tạo một ứng dụng nhiều màn hình, có phân quyền, có điều hướng nội bộ và có khả năng xử lý dữ liệu nghiệp vụ liên tục. Các kỹ thuật quan trọng gồm lập trình hướng sự kiện, chia form bằng partial class, nạp động UserControl, sử dụng Dock/Anchor, tổ chức bố cục bằng TableLayoutPanel/FlowLayoutPanel, hiển thị dữ liệu bằng DataGridView, mở hộp thoại modal bằng ShowDialog và xử lý tác vụ bất đồng bộ với async/await.

2.2.1 Lập trình hướng sự kiện

WinForms vận hành theo mô hình event-driven. Mỗi hành động của người dùng tạo ra một sự kiện, sau đó form hoặc user control xử lý sự kiện đó trong code-behind. Trong project, mô hình này xuất hiện ở hầu hết các màn hình: nhấn nút đăng nhập, chọn khoa, chọn bác sĩ, chọn slot giờ, click dòng lịch hẹn, mở chi tiết thanh toán, tải file PDF/MRI, đăng ký ca làm hoặc kiểm tra trạng thái thanh toán.

Ưu điểm của mô hình sự kiện là phù hợp với phần mềm Desktop vì người dùng không đi theo một luồng tuyến tính cố định như chương trình console. Một lễ tân có thể tìm bệnh nhân, mở lịch hôm nay, quay lại hàng chờ rồi chuyển sang thanh toán. Một kỹ thuật viên có thể từ danh sách request đi sang upload MRI hoặc nhập kết quả xét nghiệm. Do đó, mỗi nút chức năng cần được gắn event rõ ràng, sau đó event gọi đúng controller/BUS thay vì xử lý toàn bộ trong giao diện.

```
1 private void btnPayment_Click(object sender, EventArgs e)
2 {
3     LoadContent(new Payment());
4 }
5
6 private void dgvAppointment_CellContentClick(
7     object sender,
8     DataGridViewCellEventArgs e)
9 {
10    if (e.RowIndex < 0)
```

```

11 {
12     return;
13 }
14
15 int appointmentId = Convert.ToInt32(
16     dgvAppointment.Rows[e.RowIndex].Cells["AppointmentID"].Value);
17
18 OpenAppointmentDetail(appointmentId);
19 }

```

2.2.2 Partial class và Designer

Mỗi form hoặc user control trong WinForms thường được tách thành hai phần: file .cs chứa logic xử lý và file .Designer.cs chứa mã sinh tự động bởi Visual Studio Designer. Đây là một kỹ thuật quan trọng vì nó giúp nhóm phát triển vừa sử dụng công cụ kéo thả giao diện, vừa viết logic nghiệp vụ thủ công mà không làm rối lẫn hai loại mã.

Trong project, các màn hình như ScheduleToday, WaitingList, RegisterShiftForm, PharmacyMainform và TechnicianMainform đều có cấu trúc partial class. File Designer khai báo control, thiết lập vị trí, màu sắc, font, Dock, Anchor, gán event cơ bản. File code-behind chịu trách nhiệm gọi BUS, bind dữ liệu, xử lý click, điều hướng và hiển thị thông báo.

Cách tổ chức này cần được tuân thủ nghiêm túc. Nếu chỉnh sửa trực tiếp phần Designer quá nhiều, Visual Studio có thể ghi đè khi mở lại giao diện. Vì vậy, báo cáo xem Designer là nơi mô tả cấu trúc hiển thị, còn code-behind và controller là nơi mô tả hành vi.

2.2.3 Form, UserControl và mô hình màn hình con

Một ứng dụng Desktop nhiều phân hệ nếu mở mỗi chức năng bằng một form độc lập sẽ nhanh chóng gây rối trải nghiệm người dùng: nhiều cửa sổ chồng lên nhau, khó quay lại màn hình chính và khó kiểm soát trạng thái đăng nhập. Project giải quyết bằng cách dùng form chính theo vai trò làm “shell”, sau đó nạp các UserControl vào vùng nội dung trung tâm.

Trong mô hình này:

- MainFormRole quyết định người dùng thuộc vai trò nào và mở main form tương ứng.
- Các main form như ReceptionistMainform, TechnicianMainform, PharmacyMainform giữ thanh menu, thông tin người dùng và vùng nội dung.
- Mỗi chức năng cụ thể được hiện thực thành UserControl: quản lý bệnh nhân, tạo lịch hẹn, lịch hôm nay, hàng chờ, cấp phát thuốc, upload kết quả, quản lý thuốc.
- Các thao tác cần xác nhận hoặc nhập liệu ngắn được mở bằng form modal, ví dụ form đăng ký ca làm hoặc form thanh toán PayOS.

```

1 private void LoadContent(UserControl control)
2 {
3     contentPanel.Controls.Clear();
4     control.Dock = DockStyle.Fill;
5     contentPanel.Controls.Add(control);
6 }

```

Đây là kỹ thuật quan trọng trong project vì nó biến ứng dụng WinForms từ tập hợp nhiều cửa sổ rời rạc thành một dashboard thống nhất theo vai trò. Người dùng có cảm giác đang làm việc trong một phần mềm hoàn chỉnh, không phải mở từng form riêng biệt.

2.2.4 Dock, Anchor và bố cục cơ bản

WinForms không có hệ thống layout hiện đại như web CSS, vì vậy việc dùng đúng Dock, Anchor, TableLayoutPanel và FlowLayoutPanel quyết định rất lớn đến chất lượng giao diện. Trong đồ án, các main form và nhiều user control dùng DockStyle.Fill để phần nội dung chiếm toàn bộ vùng còn lại. Các bảng như DataGridView cũng được dock fill để không bị trống hoặc lệch khi phóng to cửa sổ.

TableLayoutPanel phù hợp với các màn hình cần chia vùng rõ ràng như header, bộ lọc, bảng dữ liệu và footer. FlowLayoutPanel phù hợp với các danh sách thể động như danh sách khoa, bác sĩ, time slot hoặc card thuốc vì số lượng phần tử có thể thay đổi theo dữ liệu. Anchor được dùng cho các nút, ô tìm kiếm hoặc nhóm thao tác cần giữ khoảng cách với cạnh trái/phải của form.

Bảng 2.2.1: Kỹ thuật layout WinForms sử dụng trong đồ án

Kỹ thuật	Cách dùng	Ý nghĩa trong project
DockStyle.Fill	Cho control lấp đầy vùng cha	Dùng cho content panel, bảng lịch hẹn, danh sách request, màn hình pharmacy/technician.
Anchor	Neo control vào một hoặc nhiều cạnh	Giữ ô tìm kiếm, nút thao tác và panel lọc không bị lệch khi re-size.
TableLayoutPanel	Chia giao diện thành hàng/cột tỷ lệ	Tạo bố cục ổn định cho màn hình có header, filter, grid và action bar.
FlowLayoutPanel	Sắp xếp control theo dòng chảy	Hiển thị card khoa, card bác sĩ, danh sách thuốc hoặc danh sách lựa chọn động.
Panel	Tạo vùng chứa trung gian	Chia menu trái, header, content, footer và vùng thao tác phụ.

2.2.5 DataGridView và binding dữ liệu

DataGridView là control trung tâm của các ứng dụng quản lý Desktop. Trong hệ thống phòng khám, bảng dữ liệu xuất hiện ở lịch hẹn trong ngày, danh sách bệnh nhân, hàng chờ, danh sách request xét nghiệm, danh sách đơn thuốc, danh mục thuốc, tồn kho và thanh toán. Cách sử dụng phù hợp là tắt AutoGenerateColumns khi cần kiểm soát cột, đặt tên cột rõ ràng, bind

danh sách DTO hoặc DataTable, sau đó xử lý event chọn dòng/click nút trong bảng.

```
1 private void LoadAppointmentsToday()
2 {
3     dgvAppointment.AutoGenerateColumns = false;
4     dgvAppointment.DataSource = scheduleTodayController.GetAppointmentsToday();
5 }
6
7 private void RefreshAfterCheckIn()
8 {
9     LoadAppointmentsToday();
10    LoadWaitingListSummary();
11 }
```

Trong báo cáo này, DataGridView không được xem như một bảng hiển thị đơn giản mà là điểm giao tiếp nghiệp vụ. Người dùng chọn một dòng để mở chi tiết, chuyển trạng thái, tạo thanh toán hoặc điều hướng sang hồ sơ bệnh án. Vì vậy, các cột ID nội bộ thường cần giữ trong dữ liệu nhưng có thể ẩn khỏi giao diện; các cột trạng thái nên hiển thị bằng chữ dễ hiểu; các thao tác nguy hiểm cần xác nhận bằng MessageBox.

2.2.6 Form modal và hộp thoại nghiệp vụ

Ngoài dashboard chính, project vẫn sử dụng form modal cho các tình huống cần người dùng hoàn tất một thao tác nhỏ trước khi quay lại màn hình chính. Ví dụ: form đăng ký ca làm, form thanh toán PayOS, form xem ảnh/PDF, form xác nhận hoặc chỉnh sửa nhanh. Với modal, lời gọi ShowDialog khóa luồng thao tác ở form cha cho đến khi form con đóng lại, phù hợp với các nghiệp vụ cần kết quả rõ ràng.

```
1 using (var form = new RegisterShiftForm(currentUser.EmployeeID))
2 {
3     DialogResult result = form.ShowDialog();
4     if (result == DialogResult.OK)
5     {
6         LoadShifts();
7     }
8 }
```

2.2.7 Custom control và card nghiệp vụ

Một số màn hình không phù hợp để trình bày bằng bảng thuần túy, ví dụ danh sách thuốc trong nhà thuốc, danh sách khoa/bác sĩ khi tạo lịch hẹn hoặc các card thống kê tổng quan. Project sử dụng các user control dạng card để đóng gói giao diện và hành vi của một phần tử nghiệp vụ. Ví dụ, ucPharmacyMedicineItemCard đại diện một thuốc trong danh sách nhà thuốc; card có thể hiển thị tên thuốc, dạng bào chế, tồn kho, trạng thái và các nút thao tác.

Lợi ích của custom control là tránh lặp lại code tạo giao diện ở nhiều nơi. Thay vì mỗi màn hình tự tạo label, button, màu sắc và event cho từng thuốc, project có thể tạo một control riêng rồi truyền dữ liệu vào. Khi cần đổi cách trình bày thuốc, chỉ cần sửa một control.

2.3 Mô hình điều hướng đa vai trò trong WinForms

Điều hướng trong project được thiết kế theo hai lớp. Lớp thứ nhất là điều hướng sau đăng nhập: hệ thống xác định role và mở main form tương ứng. Lớp thứ hai là điều hướng bên trong từng role: main form nhận lệnh từ menu hoặc từ sự kiện của user control con, sau đó thay màn hình nội dung.

```
1 private Form CreateDashboardForCurrentUser()
2 {
3     string role = RoleNormalizer.Normalize(currentUser.Role);
4
5     switch (role)
6     {
7         case Role.Admin:
8             return new AdminMainform(currentUser);
9         case Role.Doctor:
10            return new DoctorMainform(currentUser);
11        case Role.Nurse:
12            return new NurseMainform(currentUser);
13        case Role.Pharmacist:
14            return new PharmacyMainform(currentUser);
15        case Role.Receptionist:
16            return new ReceptionistMainform(currentUser);
17        case Role.Technician:
18            return new TechnicianMainform(currentUser);
19        default:
20            return null;
21    }
22 }
```

Ở nhóm kỹ thuật viên và dược sĩ, project còn dùng sự kiện điều hướng riêng giữa các user control. Đây là cách thiết kế tốt hơn việc để user control tự biết toàn bộ main form. User control chỉ phát sự kiện “tôi muốn đi đến màn hình này”, còn main form quyết định nạp view nào. Nhờ đó quan hệ phụ thuộc được giữ một chiều: view con không cần phụ thuộc trực tiếp vào cấu trúc toàn bộ dashboard.

```
1 public event EventHandler<PharmacyNavigationEventArgs> NavigateRequested;
2
3 protected void NavigateTo(PharmacyViewTarget target, int id = 0)
4 {
5     NavigateRequested?.Invoke(
6         this,
7         new PharmacyNavigationEventArgs(target, id));
8 }
```

2.4 Thiết kế trải nghiệm Desktop theo từng vai trò

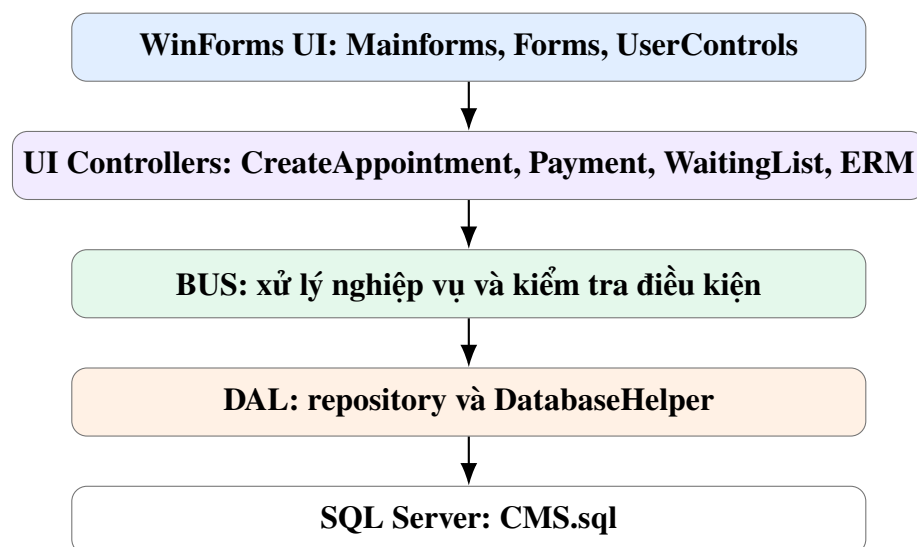
Khác với website giới thiệu, ứng dụng Desktop quản lý phòng khám là công cụ làm việc lặp lại hằng ngày. Vì vậy, giao diện cần ưu tiên tốc độ thao tác, khả năng quét thông tin và giảm nhầm lẫn. Cách chia dashboard theo vai trò giúp người dùng chỉ thấy đúng chức năng cần thiết. Báo cáo phân tích trải nghiệm theo từng vai trò như sau:

- **Lễ tân:** cần truy cập nhanh bệnh nhân, tạo lịch, lịch hôm nay, hàng chờ và thanh toán. Các màn hình nên có ô tìm kiếm, bảng danh sách rõ trạng thái và nút thao tác trực tiếp.

- **Bác sĩ:** cần xem bệnh nhân đang chờ, mở hồ sơ bệnh án, xem lịch sử khám, chỉ định xét nghiệm/hình ảnh, kê toa và hẹn tái khám. Giao diện cần ưu tiên thông tin y khoa và giảm thao tác chuyển màn hình.
- **Điều dưỡng:** cần nhập sinh hiệu, hỗ trợ phòng khám và theo dõi hàng chờ. Các form nhập liệu cần ngắn, rõ đơn vị đo, hạn chế nhập sai.
- **Kỹ thuật viên:** cần xem request, upload PDF/MRI, nhập kết quả xét nghiệm và xem hồ sơ liên quan. Giao diện cần thể hiện rõ trạng thái request để tránh xử lý trùng hoặc bỏ sót.
- **Được sĩ:** cần xem đơn chờ cấp phát, danh mục thuốc, tồn kho và ca làm. Giao diện dạng card kết hợp bảng giúp kiểm tra nhanh thuốc, số lượng và trạng thái.
- **Quản trị viên:** cần quản lý tài khoản, nhân sự, cấu hình và dữ liệu nền. Giao diện nên tách nhóm rõ để tránh thao tác nhầm trên dữ liệu hệ thống.

2.5 Mô hình kiến trúc nhiều lớp

Kiến trúc nhiều lớp giúp tách biệt trách nhiệm giữa giao diện, xử lý nghiệp vụ và truy cập dữ liệu. Trong đồ án, cách tổ chức chính là DTO–BUS–DAL, bổ sung thêm Core, Models, ServicesExternal và controller phía UI.



Hình 2.5.1: Kiến trúc phân lớp của hệ thống quản lý phòng khám

Luồng xử lý tổng quát:

```

1 private void btnSave_Click(object sender, EventArgs e)
2 {
3     var dto = MapInputToDto();
4     var result = patientBus.Save(dto);
5
6     if (result.Success)
7     {
8         LoadPatients();
9         MessageBox.Show(result.Message, "Thông báo");
10    }
11 }
  
```

```

10     }
11     else
12     {
13         MessageBox.Show(result.Message, "Loi");
14     }
15 }

```

2.6 Tầng DTO, BUS và DAL

DTO là lớp truyền dữ liệu, không chứa truy vấn SQL hoặc xử lý giao diện. BUS là tầng xử lý nghiệp vụ, nhận dữ liệu từ UI/controller, kiểm tra điều kiện hợp lệ và gọi DAL khi cần. DAL là tầng làm việc trực tiếp với SQL Server thông qua ADO.NET.

Bảng 2.6.1: Vai trò các tầng trong kiến trúc DTO–BUS–DAL

Tầng	Thư mục chính	Vai trò
DTO	DTO/Auth, DTO/Clinical, DTO/Facility, DTO/Scheduling	Đóng gói dữ liệu truyền giữa UI, BUS và DAL.
BUS	BUS/Services, BUS/Interfaces	Kiểm tra nghiệp vụ, điều phối repository và trả kết quả cho UI.
DAL	DAL/Repositories, DAL/DataContext	Tạo truy vấn SQL, truyền tham số, đọc dữ liệu và map sang DTO.
Core	Core/Identity, Core/Constants, Core/Session, Core/Utils	Lưu vai trò, trạng thái, session và tiện ích dùng chung.

2.7 ADO.NET và SQL Server

ADO.NET là mô hình truy cập dữ liệu truyền thống của .NET, cho phép lập trình viên kiểm soát trực tiếp kết nối, câu lệnh SQL, tham số và kết quả trả về. Trong hệ thống hiện tại, lớp DatabaseHelper sử dụng SqlConnection, SqlCommand, SqlDataAdapter và SqlParameter để thực thi truy vấn.

```

1 public static DataTable ExecuteQuery(string query, SqlParameter[] parameters = null)
2 {
3     using (SqlConnection conn = GetConnection())
4     using (SqlCommand cmd = new SqlCommand(query, conn))
5     {
6         if (parameters != null)
7         {
8             cmd.Parameters.AddRange(parameters);
9         }
10
11     using (SqlDataAdapter adapter = new SqlDataAdapter(cmd))

```

```

12     {
13         DataTable table = new DataTable();
14         adapter.Fill(table);
15         return table;
16     }
17 }
18 }

```

Ưu điểm của ADO.NET là rõ ràng, dễ tối ưu câu SQL và phù hợp với đồ án có nhiều truy vấn tùy biến. Nhược điểm là lập trình viên phải tự map dữ liệu từ DataRow sang DTO và phải cẩn thận với việc lặp lại câu truy vấn.

2.8 Entity Framework và định hướng ORM

Entity Framework là ORM cho phép ánh xạ bảng dữ liệu thành lớp entity và truy vấn bằng LINQ. Trong repository hiện tại có các thư mục liên quan đến EF ở ngoài solution WinForms chính, tuy nhiên mã nguồn WinForms đang khảo sát chủ yếu sử dụng ADO.NET. Vì vậy, báo cáo xem EF như một hướng so sánh và phát triển, không phải cơ chế truy cập dữ liệu chính của project hiện tại.

So với ADO.NET, Entity Framework giúp giảm số lượng câu SQL thủ công và tăng tốc độ phát triển CRUD. Tuy nhiên, EF cần cấu hình model, migration và kiểm soát hiệu năng truy vấn khi dữ liệu lớn. Với đồ án phòng khám, EF phù hợp cho các bảng CRUD ổn định như khoa, phòng, thuốc, nhân viên; còn ADO.NET vẫn thuận lợi với các truy vấn tổng hợp phức tạp như hàng chờ, ERM và báo cáo thanh toán.

2.8.1 Định hướng thiết kế DAL bằng Entity Framework

Trong phiên bản hiện tại, tầng DAL chính của project đang dùng ADO.NET. Tuy nhiên, hướng phát triển tiếp theo là bổ sung thêm một nhánh DAL dùng Entity Framework Core để giảm số lượng câu SQL thủ công ở các nghiệp vụ CRUD chuẩn. Cách triển khai hợp lý không phải thay toàn bộ DAL ngay lập tức, mà thiết kế song song hai kiểu repository:

- **ADO.NET Repository:** tiếp tục dùng cho các truy vấn tổng hợp phức tạp, báo cáo, ERM, hàng chờ và các truy vấn cần tối ưu SQL cụ thể.
- **EF Repository:** dùng cho bảng danh mục và CRUD ổn định như Departments, Rooms, Medicines, Suppliers, Shifts, Users, Roles.
- **Interface chung:** tầng BUS nên phụ thuộc vào interface thay vì phụ thuộc trực tiếp vào DAL cụ thể, nhờ đó có thể thay repository ADO.NET bằng EF theo từng module.

Mô hình EF dự kiến gồm các thành phần: entity class ánh xạ bảng, ClinicDbContext đại diện session làm việc với database, DbSet<T> đại diện tập bảng, configuration cho khóa chính/khóa ngoại, và repository dùng LINQ để truy vấn. Với WinForms, DbContext không nên giữ sống toàn bộ vòng đời ứng dụng; mỗi thao tác hoặc mỗi service call nên tạo context ngắn hạn để tránh dữ liệu stale và tránh giữ kết nối lâu.

```

1 public class ClinicDbContext : DbContext
2 {
3     public DbSet<PatientEntity> Patients { get; set; }
4     public DbSet<EmployeeEntity> Employees { get; set; }
5     public DbSet<MedicineEntity> Medicines { get; set; }
6     public DbSet<AppointmentEntity> Appointments { get; set; }
7
8     protected override void OnConfiguring(DbContextOptionsBuilder options)
9     {
10         options.UseSqlServer(
11             ConfigurationManager.ConnectionStrings["ClinicDB"].ConnectionString);
12     }
13 }

```

Khi chuyển một repository sang EF, cần chú ý không để entity EF đi thẳng lên giao diện. Giao diện vẫn nên nhận DTO để giữ ranh giới kiến trúc. Entity phản ánh cấu trúc bảng; DTO phản ánh dữ liệu cần cho màn hình. Hai loại này có mục đích khác nhau. Ví dụ màn hình nhà thuốc có thể cần MedicineDTO gồm tên thuốc, tồn kho, trạng thái hiển thị và nhãn cảnh báo; entity MedicineEntity chỉ nên phản ánh cột trong bảng Medicines.

```

1 public class MedicineEfRepository : IMedicineRepository
2 {
3     public List<MedicineDTO> GetActiveMedicines()
4     {
5         using (var context = new ClinicDbContext())
6         {
7             return context.Medicines
8                 .Where(x => x.IsActive)
9                 .OrderBy(x => x.MedicineName)
10                .Select(x => new MedicineDTO
11                {
12                    MedicineID = x.MedicineID,
13                    MedicineName = x.MedicineName,
14                    Unit = x.Unit,
15                    StockQuantity = x.StockQuantity
16                })
17                .ToList();
18        }
19    }
20 }

```

Đối với đồ án môn Desktop, việc trình bày EF không chỉ là nêu khái niệm ORM. Điểm cần thể hiện là nhóm hiểu rõ cách tích hợp EF vào kiến trúc hiện có: WinForms không gọi DbContext; BUS không phụ thuộc cụ thể vào EF nếu đã có interface; DTO vẫn là dữ liệu đi lên UI; migration cần được kiểm soát để không phá schema CMS.sql hiện có.

2.9 API, Supabase, SheetDB và PayOS

Phân hệ tích hợp ngoài được tổ chức trong BUS/Services/Integrations/ApiSync và DAL/Repositories/Integrations/ApiSync. Repository đồng bộ có các phương thức đọc dữ liệu từ Supabase/SheetDB, upsert bệnh nhân, nhân viên, yêu cầu xét nghiệm, yêu cầu hình ảnh và kết quả vào SQL Server cục bộ. Form MockApiSyncForm dùng để mô phỏng thao tác đồng bộ và hiển thị lỗi khi dữ liệu API thiếu UUID hoặc JSON sai định dạng.

Project ServicesExternal chứa các lớp PayOS như PayOsCreatePaymentRequest, PayOsPaymentData, PayOsApiResponse và ImageHelper. Những lớp này hỗ trợ mô phỏng tạo thanh toán, nhận QR code, trạng thái thanh toán và chuyển đổi ảnh sang base64.

2.9.1 Vai trò của ứng dụng trung gian trong tích hợp API

Trong đồ án, dữ liệu không chỉ nằm trong SQL Server local. Một phần dữ liệu được mô phỏng ở Supabase và Google Sheet thông qua SheetDB để thể hiện tình huống phòng khám cần lấy dữ liệu từ nguồn ngoài. Supabase đóng vai trò như backend dữ liệu online có REST API chuẩn; SheetDB đóng vai trò lớp API hóa Google Sheet để người dùng có thể nhập dữ liệu kiểm thử dạng bảng tính; SQL Server local là cơ sở dữ liệu chính của ứng dụng Desktop.

Luồng này có ba môi trường dữ liệu:

Bảng 2.9.1: Vai trò các nguồn dữ liệu trong tích hợp ngoài

Nguồn	Công nghệ	Vai trò trong đồ án
SQL Server local	CMS.sql, ADO.NET	Nguồn dữ liệu chính mà WinForms đọc/ghi khi vận hành phòng khám.
Supabase	REST API, PostgreSQL online	Nguồn dữ liệu online mô phỏng backend hoặc dữ liệu đồng bộ từ hệ thống khác.
Google Sheet qua SheetDB	SheetDB REST API	Nguồn nhập liệu trung gian, phù hợp cho demo, import dữ liệu mẫu và chỉnh dữ liệu không cần mở SQL Server.
Backend PayOS	API backend triển khai ngoài Desktop	Trung gian bảo vệ key, nhận web-hook, tạo payment link và kiểm tra trạng thái thanh toán.

Ứng dụng Desktop đọc cấu hình từ App.config, gồm ApiSyncSupabaseUrl, ApiSyncSupabaseKey, ApiSyncSheetDbPatientUrl, ApiSyncSheetDbPatientSheet, ApiSyncSheetDbEmployeeUrl, ApiSyncSheetDbEmployeeSheet, BackendBaseUrl, PayOSClientId, PayOSApiKey, PayOSChecksumKey, PayOSReturnUrl và PayOSCancelUrl. Việc gom cấu hình vào App.config giúp tránh hard-code endpoint trong form, đồng thời dễ thay đổi môi trường demo.

2.9.2 Mô hình dữ liệu SheetDB và Supabase

SheetDB biến từng sheet thành một REST endpoint. Trong project, sheet bệnh nhân được cấu hình với tên sheet Patient; sheet nhân viên/bác sĩ được cấu hình với tên sheet Doctor. Các cột trong sheet cần trùng tên JSON mà DTO đang map bằng JsonPropertyName. Ví dụ, sheet bệnh nhân cần có các cột như patientid, patientcode, fullname, gender, dob, phone, address, bloodtype, allergy. Sheet nhân viên cần có employeeid, employeecode, fullname, rolename, departmentid, departmentname, phone, status.

Bảng 2.9.2: Ví dụ cấu trúc sheet bệnh nhân

Cột sheet	DTO nhận	Ý nghĩa
patientid	PatientId	UUID đồng bộ. Nếu thiếu, hệ thống sinh ở local hoặc lấy từ Supabase rồi patch ngược về sheet.
patientcode	PatientCode	Mã bệnh nhân, dùng để đối chiếu khi chưa có UUID.
fullname	FullName	Họ tên bệnh nhân, là trường bắt buộc khi lọc dữ liệu hợp lệ.
gender	Gender	Giới tính.
dob	DOB	Ngày sinh dạng chuỗi, BUS chuẩn hóa về yyyy-MM-dd khi đẩy lên Supabase.
phone	Phone	Số điện thoại.
bloodtype	BloodType	Nhóm máu.
allergy	Allergy	Dị ứng; khi ghi về sheet có thể sinh thêm allergy_flag.

```

1 public class ApiPatientDTO
2 {
3     [JsonPropertyName("patientid")]
4     public Guid? PatientId { get; set; }
5
6     [JsonPropertyName("patientuuid")]
7     public Guid? LegacyPatientUuid { get; set; }
8
9     [JsonPropertyName("patientcode")]
10    public string PatientCode { get; set; }
11
12    [JsonPropertyName("fullname")]
13    public string FullName { get; set; }
14
15    [JsonIgnore]
16    public Guid? SyncUuid
17    {
18        get => PatientId ?? LegacyPatientUuid;
19        set => PatientId = value;
20    }
21 }

```

Điểm đáng chú ý là SyncUuid không map trực tiếp từ JSON, mà là thuộc tính tính toán để thống nhất hai kiểu dữ liệu cũ/mới: patientid và patientuuid. Đây là chi tiết quan trọng vì dữ liệu tích hợp thực tế thường không đồng nhất hoàn toàn. DTO trung gian giúp BUS không phải xử lý nhiều tên cột ở khắp nơi.

2.9.3 Luồng đồng bộ Supabase, SheetDB và SQL Server

Luồng ApiSyncBUS.SyncAsync() không chỉ đơn giản là gọi API rồi insert database. Nó là một quy trình hợp nhất dữ liệu nhiều nguồn:

1. Gọi `GetSupabasePatientsAsync`, `GetSupabaseEmployeesAsync`, `GetSheetPatientsAsync`, `GetSheetEmployeesAsync`.
2. Lọc dòng không hợp lệ: bệnh nhân phải có `PatientCode` và `FullName`; nhân viên phải có `SyncCode`, `FullName` và không bị nhầm thành mã bệnh nhân bắt đầu bằng BN.
3. Bổ sung dữ liệu nhân viên bằng `EnrichEmployeeFields`: tự điền tên khoa, role bác sĩ theo mã, trạng thái đang làm việc nếu thiếu.
4. Nếu dòng sheet thiếu UUID nhưng Supabase đã có bản ghi cùng mã, hệ thống patch UUID từ Supabase ngược về SheetDB.
5. Upsert dữ liệu từ SheetDB vào SQL Server local; nếu local sinh UUID mới thì patch UUID đó ngược lại sheet.
6. Upsert dữ liệu từ Supabase vào SQL Server local.
7. Đồng bộ SheetDB sang Supabase: nếu sheet có dòng mới chưa có trên Supabase thì insert; nếu có mã trùng nhưng thiếu UUID thì patch.
8. Đồng bộ Supabase sang SheetDB: nếu Supabase có dòng mới chưa có trên sheet thì insert xuống sheet.
9. Trả về `ApiSyncResultDTO` gồm số lượng insert, patch, sinh UUID, dòng bị bỏ qua và danh sách dữ liệu đã merge.

```

1 public async Task<ApiSyncResultDTO> SyncAsync()
2 {
3     var supabasePatients = await repository.GetSupabasePatientsAsync();
4     var sheetPatients = await repository.GetSheetPatientsAsync();
5
6     await LinkMissingSheetPatientUuidsFromSupabaseAsync(
7         sheetPatients,
8         supabasePatients,
9         result);
10
11     await SyncPatientsToLocalSqlServerAsync(
12         sheetPatients,
13         result,
14         patchSheetWhenMissingUuid: true);
15
16     await SyncPatientsSheetToSupabaseAsync(
17         sheetPatients,
18         supabasePatients,
19         result);
20
21     await SyncPatientsSupabaseToSheetAsync(
22         supabasePatients,
23         sheetPatients,
24         result);
25
26     return result;
27 }

```

Về thiết kế, UUID là khóa đồng bộ bền vững giữa ba nguồn. Mã bệnh nhân hoặc mã nhân viên chỉ dùng làm khóa phụ khi dữ liệu cũ chưa có UUID. Khi hệ thống phát hiện bản ghi theo mã, nó liên kết lại bằng UUID để các lần đồng bộ sau không bị tạo trùng. Đây là một điểm có thể trình bày sâu trong báo cáo vì nó thể hiện tư duy xử lý dữ liệu không đồng nhất.

2.9.4 Đồng bộ request cận lâm sàng từ Supabase

Ngoài bệnh nhân và nhân viên, project còn có hàm `SyncRequestsFromSupabaseAsync()` phục vụ kỹ thuật viên. Luồng này kéo các request hình ảnh và xét nghiệm từ Supabase, sau đó chuyển thành request local để kỹ thuật viên xử lý trong WinForms.

Quy trình xử lý request gồm:

- Đồng bộ lỗi bệnh nhân/nhân viên trước để đảm bảo có mã bệnh nhân và mã bác sĩ trong local.
- Kéo danh sách imaging request, imaging detail, encounters, patients, employees, services và imaging results từ Supabase.
- Tạo dictionary tra cứu nhanh theo ImagingID, EncounterID, ServiceID, DetailID.
- Với mỗi detail, tìm request cha, encounter, bệnh nhân, bác sĩ và service.
- Tìm local patient/doctor theo mã; nếu chưa có thì upsert từ dữ liệu Supabase.
- Ghép mã request dạng ImagingCode_DetailID hoặc IM_REQ_DET_DetailID.
- Map trạng thái ngoài về trạng thái local: Chờ xử lý, Đang xử lý, Hoàn thành.
- Gọi `UpsertLocalRequestAsync` để lưu request local cùng image URL, PDF URL, ghi chú hoặc kết quả lab.

Điểm này cho thấy phân hệ kỹ thuật viên không chỉ upload file cục bộ mà còn có khả năng kéo chỉ định từ nguồn online. Đây là mô phỏng gần với thực tế: bác sĩ hoặc hệ thống khác tạo chỉ định ở backend, kỹ thuật viên dùng ứng dụng Desktop để tiếp nhận và cập nhật kết quả.

2.9.5 Luồng PayOS, VietQR, MoMo QR và webhook

Trong giao diện thanh toán, project dùng `PayOsPaymentForm` để mô phỏng thanh toán chuyển khoản qua PayOS/VietQR. Tên một số control vẫn còn dấu vết từ MoMo QR như `picQRMOMO`, nhưng luồng xử lý hiện tại là PayOS. PayOS tạo payment link, trả về checkout URL, QR code hoặc QR data URL theo chuẩn thanh toán ngân hàng/VietQR. Người dùng quét QR bằng ứng dụng ngân hàng hoặc ví hỗ trợ chuyển khoản.

Luồng chuẩn được thiết kế như sau:

1. Người dùng chọn phương thức chuyển khoản trong `PaymentDetail`.
2. WinForms mở `PayOsPaymentForm`.
3. Người dùng nhập số tiền, người mua, nội dung; form sinh orderCode nếu chưa có.
4. Form ưu tiên gọi backend tại `BackendBaseUrl/api/payos/create-payment`.
5. Backend giữ `PayOSClientId`, `PayOSApiKey`, `PayOSChecksumKey`, tạo chữ ký và gọi PayOS.

6. PayOS trả checkoutUrl, qrCode, qrDataURL, status, paymentLinkId.
7. WinForms render QR vào PictureBox; đồng thời bật timer kiểm tra trạng thái 5 giây/lần.
8. Khi khách thanh toán, PayOS gọi webhook về backend. Backend xác thực chữ ký webhook và cập nhật trạng thái đơn.
9. WinForms kiểm tra trạng thái qua endpoint backend /api/payos/payments/{orderCode} hoặc nhận cơ chế realtime nếu backend triển khai SignalR/WebSocket trong tương lai.
10. Khi trạng thái là PAID, hệ thống dừng timer và người dùng xác nhận thanh toán trong PaymentDetail; PaymentController gọi PaymentBUS.UpdatePaymentStatus.

```

1 private static string CreatePayOsPaymentSignature(
2     int amount,
3     string cancelUrl,
4     string description,
5     long orderCode,
6     string returnUrl,
7     string checksumKey)
8 {
9     var data =
10         $"amount={amount}&cancelUrl={cancelUrl}" +
11         $"&description={description}&orderCode={orderCode}" +
12         $"&returnUrl={returnUrl}";
13
14     using (var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(checksumKey)))
15     {
16         var hash = hmac.ComputeHash(Encoding.UTF8.GetBytes(data));
17         return BitConverter.ToString(hash).Replace("-", "").ToLowerInvariant();
18     }
19 }

```

Trong cấu hình hiện tại, AllowDirectPayOSFallback đặt là false, nghĩa là flow đúng chuẩn yêu cầu Desktop gọi backend trước. Đây là thiết kế hợp lý vì ứng dụng Desktop không nên giữ bí mật thanh toán ở phía client nếu triển khai thật. Fallback gọi trực tiếp PayOS chỉ dùng cho demo kỹ thuật hoặc khi backend chưa sẵn sàng.

```

1 paymentStatusTimer = new System.Windows.Forms.Timer
2 {
3     Interval = 5000
4 };
5
6 paymentStatusTimer.Tick += async (s, args) =>
7     await CheckCurrentPaymentStatusAsync(false);

```

Về realtime, PayOS không tự đẩy trực tiếp vào WinForms. Realtime cần đi qua backend: webhook PayOS cập nhật trạng thái payment trong database backend hoặc SQL Server; WinForms có thể polling định kỳ như hiện tại, hoặc backend phát SignalR event cho client Desktop. Trong báo cáo, cơ chế hiện tại được xem là polling 5 giây/lần; hướng phát triển là thay polling bằng SignalR để trạng thái thanh toán cập nhật tức thời hơn.

3 Phân tích và Thiết kế hệ thống

3.1 Tác nhân sử dụng hệ thống

Hệ thống được thiết kế theo mô hình đa vai trò. Mỗi vai trò có phạm vi chức năng riêng, giúp giảm nhầm lẫn khi thao tác và hạn chế truy cập trái quyền.

Bảng 3.1.1: Tác nhân và phạm vi chức năng

Tác nhân	Chức năng chính
Quản trị viên	Quản lý tài khoản, nhân sự, cấu hình hệ thống, phòng ban và theo dõi dữ liệu quản trị.
Lễ tân	Quản lý bệnh nhân, tạo lịch hẹn, xác nhận lịch, tiếp nhận bệnh nhân, quản lý hàng chờ và thanh toán.
Bác sĩ	Xem bệnh nhân chờ khám, mở hồ sơ bệnh án, ghi nhận chẩn đoán, tạo chỉ định, kê toa và hẹn tái khám.
Điều dưỡng	Theo dõi hàng chờ, nhập dấu hiệu sinh tồn, hỗ trợ phòng khám và ca làm.
Kỹ thuật viên	Nhận yêu cầu xét nghiệm/chẩn đoán hình ảnh, cập nhật kết quả, upload PDF/MRI và xem hồ sơ liên quan.
Dược sĩ	Xem đơn thuốc, cấp phát thuốc, quản lý danh mục thuốc, tồn kho và ca làm.

3.2 Yêu cầu chức năng

Các yêu cầu chức năng chính được tổng hợp từ cấu trúc mã nguồn, schema CMS.sql và các màn hình WinForms.

Bảng 3.2.1: Danh sách yêu cầu chức năng chính

Mã	Chức năng	Mô tả
FR-01	Đăng nhập và phân quyền	Kiểm tra tài khoản, lấy vai trò, lưu session và mở dashboard theo role.
FR-02	Quản lý bệnh nhân	Thêm, sửa, tìm kiếm, xem thông tin bệnh nhân và bảo hiểm.
FR-03	Quản lý lịch hẹn	Tạo lịch, xác nhận lịch, kiểm tra bác sĩ, khoa, phòng và ngày khám.
FR-04	Tiếp nhận và hàng chờ	Check-in bệnh nhân, thống kê trạng thái chờ, đang khám, hoàn tất.

Mã	Chức năng	Mô tả
FR-05	Hồ sơ bệnh án điện tử	Tổng hợp lịch sử khám, sinh hiệu, lab, imaging, đơn thuốc, hóa đơn, tái khám.
FR-06	Xét nghiệm và hình ảnh	Tạo yêu cầu, kỹ thuật viên xử lý, upload kết quả PDF/MRI/X-Ray và cập nhật trạng thái.
FR-07	Đơn thuốc và nhà thuốc	Quản lý thuốc, tồn kho, danh sách đơn chờ cấp phát và trạng thái cấp phát.
FR-08	Thanh toán	Tạo payment pending, cập nhật số tiền, trạng thái, chi tiết thanh toán và lịch sử.
FR-09	Ca làm	Đăng ký, xem ngày/tuần/tháng, lịch sử ca và trạng thái ca làm.
FR-10	Đồng bộ API	Đồng bộ bệnh nhân, nhân viên, yêu cầu và kết quả từ Supabase/SheetDB về SQL Server.

3.3 Đặc tả use case chính

Phần này đặc tả chi tiết các use case quan trọng nhất của hệ thống. Mỗi use case được trình bày theo tác nhân, tiền điều kiện, luồng chính, ngoại lệ và kết quả sau xử lý. Đây là cơ sở để thiết kế giao diện WinForms và kiểm thử chức năng.

3.3.1 UC-01: Đăng nhập hệ thống

Bảng 3.3.1: Đặc tả use case đăng nhập

Thành phần	Nội dung
Tác nhân	Tất cả người dùng có tài khoản trong hệ thống.
Tiền điều kiện	Người dùng đã có username/password và tài khoản đang hoạt động. Cơ sở dữ liệu SQL Server có bảng Users, Roles, Employees.
Luồng chính	Người dùng mở ứng dụng; LoginForm hiển thị; người dùng nhập username/password; form gọi UserBUS.Login; BUS gọi DAL xác thực; hệ thống lưu UserSession; MainformRole mở dashboard đúng vai trò.
Luồng ngoại lệ	Nếu bỏ trống username/password, form báo lỗi và focus lại ô nhập. Nếu sai thông tin, hệ thống không mở dashboard. Nếu role không hợp lệ, hệ thống báo lỗi phân quyền.

Thành phần	Nội dung
Kết quả	Người dùng vào đúng màn hình chính theo role: Admin, Receptionist, Doctor, Nurse, Pharmacist hoặc Technician.

3.3.2 UC-02: Tạo lịch hẹn khám

Bảng 3.3.2: Đặc tả use case tạo lịch hẹn

Thành phần	Nội dung
Tác nhân	Lễ tân.
Tiền điều kiện	Lễ tân đã đăng nhập; bệnh nhân tồn tại hoặc được tạo mới; có dữ liệu khoa, bác sĩ, phòng và lịch làm.
Luồng chính	Lễ tân mở CreateAppointment; tìm hoặc chọn bệnh nhân; chọn khoa từ card trong FlowLayoutPanel; hệ thống tải danh sách bác sĩ; lễ tân chọn bác sĩ và khung giờ; hệ thống kiểm tra lịch làm; lễ tân xác nhận; BUS tạo bản ghi Appointments.
Luồng ngoại lệ	Nếu chưa chọn bệnh nhân/khoa/bác sĩ/giờ khám, form báo thiếu thông tin. Nếu bác sĩ không có lịch hoặc slot đã đầy, hệ thống không tạo lịch và yêu cầu chọn lại.
Kết quả	Lịch hẹn ở trạng thái chờ xác nhận hoặc chờ tiếp nhận, hiển thị trong màn hình lịch hôm nay khi đến ngày khám.

3.3.3 UC-03: Tiếp nhận bệnh nhân và đưa vào hàng chờ

Bảng 3.3.3: Đặc tả use case tiếp nhận bệnh nhân

Thành phần	Nội dung
Tác nhân	Lễ tân hoặc điều dưỡng tùy phân quyền triển khai.
Tiền điều kiện	Bệnh nhân có lịch trong ngày; lịch chưa bị hủy; phòng khám và bác sĩ đã được xác định.
Luồng chính	Người dùng mở ScheduleToday; chọn lịch hẹn; bấm check-in; hệ thống cập nhật trạng thái appointment; tạo hoặc cập nhật PatientQueues; màn hình hàng chờ được reload.
Luồng ngoại lệ	Nếu lịch không thuộc ngày hiện tại hoặc đã hoàn tất, hệ thống không cho check-in. Nếu thiếu phòng/bác sĩ, hệ thống báo lỗi dữ liệu.

Thành phần	Nội dung
Kết quả	Bệnh nhân xuất hiện trong hàng chờ theo khoa/bác sĩ, sẵn sàng để điều dưỡng hoặc bác sĩ xử lý bước tiếp theo.

3.3.4 UC-04: Khám bệnh và cập nhật ERM

Bảng 3.3.4: Đặc tả use case khám bệnh

Thành phần	Nội dung
Tác nhân	Bác sĩ, điều dưỡng.
Tiền điều kiện	Bệnh nhân đã check-in; có encounter hoặc lượt khám đang mở; bác sĩ có quyền xem hồ sơ.
Luồng chính	Bác sĩ mở danh sách chờ; chọn bệnh nhân; ERM tải tổng quan, lịch sử, sinh hiệu, lab, imaging, đơn thuốc và thanh toán; bác sĩ nhập triệu chứng/chẩn đoán; nếu cần thì tạo chỉ định xét nghiệm/hình ảnh; bác sĩ kê toa và lưu kết luận.
Luồng ngoại lệ	Nếu không lấy được hồ sơ theo UUID, ERM hiển thị thông báo lỗi. Nếu chưa có encounter, hệ thống tạo hoặc yêu cầu tiếp nhận lại tùy trạng thái.
Kết quả	Hồ sơ bệnh án được cập nhật; các chỉ định được chuyển sang phân hệ kỹ thuật viên; đơn thuốc chuyển sang nhà thuốc; thanh toán được tạo hoặc cập nhật.

3.3.5 UC-05: Xử lý yêu cầu xét nghiệm hoặc hình ảnh

Bảng 3.3.5: Đặc tả use case kỹ thuật viên xử lý yêu cầu

Thành phần	Nội dung
Tác nhân	Kỹ thuật viên.
Tiền điều kiện	Bác sĩ đã tạo request; kỹ thuật viên đã đăng nhập; request có trạng thái chờ xử lý hoặc đang xử lý.
Luồng chính	Kỹ thuật viên mở ucTechnicianRequests; chọn request; hệ thống phát sự kiện điều hướng; kỹ thuật viên nhập kết quả lab hoặc chọn file PDF/MRI bằng OpenFileDialog; BUS/DAL lưu kết quả và cập nhật trạng thái.

Thành phần	Nội dung
Luồng ngoại lệ	Nếu request đã hoàn tất, hệ thống không cho upload lại nếu chưa có quyền sửa. Nếu file không đúng định dạng, form báo lỗi. Nếu mất kết nối API khi đồng bộ, hệ thống ghi nhận thông báo lỗi.
Kết quả	Kết quả xét nghiệm/hình ảnh được gắn với encounter và bác sĩ có thể xem lại trong ERM.

3.3.6 UC-06: Cấp phát thuốc và thanh toán

Bảng 3.3.6: Đặc tả use case cấp phát thuốc và thanh toán

Thành phần	Nội dung
Tác nhân	Dược sĩ, lễ tân hoặc thu ngân.
Tiền điều kiện	Bác sĩ đã kê toa; thuốc có trong danh mục; payment đang ở trạng thái chờ hoặc chưa tạo.
Luồng chính	Dược sĩ mở danh sách đơn chờ; xem chi tiết đơn; kiểm tra thuốc và số lượng; xác nhận cấp phát; hệ thống cập nhật tồn kho/trạng thái đơn; lễ tân mở màn hình thanh toán; tạo hoặc xác nhận payment; nếu dùng PayOS thì mở form thanh toán và kiểm tra trạng thái.
Luồng ngoại lệ	Nếu thuốc không đủ tồn kho, hệ thống cảnh báo. Nếu payment chưa đủ dữ liệu chi tiết, hệ thống không cho xác nhận. Nếu PayOS chưa thanh toán, trạng thái vẫn giữ chờ thanh toán.
Kết quả	Đơn thuốc chuyển trạng thái đã cấp phát hoặc một phần; payment/invoice được cập nhật theo nghiệp vụ thanh toán.

3.4 Quy trình nghiệp vụ tổng quát

Quy trình khám ngoại trú trong đồ án có thể mô tả theo chuỗi bước sau:

1. Lễ tân tạo hoặc tra cứu bệnh nhân, sau đó tạo lịch hẹn theo khoa, bác sĩ và thời gian.
2. Đến ngày khám, lễ tân check-in bệnh nhân và đưa vào hàng chờ.
3. Điều dưỡng có thể nhập sinh hiệu hoặc hỗ trợ phân luồng trước khi bác sĩ khám.
4. Bác sĩ mở ERM, xem lịch sử, khám bệnh, nhập chẩn đoán và tạo chỉ định nếu cần.
5. Kỹ thuật viên xử lý chỉ định xét nghiệm/hình ảnh, upload file hoặc nhập kết quả.

6. Bác sĩ xem kết quả, hoàn tất kết luận, kê toa và hẹn tái khám nếu cần.
7. Dược sĩ cấp phát thuốc theo toa; lễ tân hoặc thu ngân xác nhận thanh toán.
8. Hệ thống lưu lại hồ sơ, payment, invoice, audit log và dữ liệu phục vụ báo cáo.

3.5 Yêu cầu phi chức năng

- **Tính dễ sử dụng:** giao diện chia theo vai trò, mỗi vai trò chỉ nhìn thấy nhóm chức năng cần thiết.
- **Tính toàn vẹn dữ liệu:** dữ liệu chính được lưu trong SQL Server với khóa chính, khóa ngoại và các trạng thái nghiệp vụ.
- **Tính bảo trì:** mã nguồn chia project theo tầng và chia thư mục theo domain.
- **Tính mở rộng:** có thể bổ sung phân hệ mới bằng cách thêm DTO, repository, BUS và UserControl tương ứng.
- **Tính an toàn:** DAL sử dụng SqlParameter để hạn chế nguy cơ SQL injection trong các truy vấn có tham số.

3.6 Thiết kế cơ sở dữ liệu

File Database/CMS.sql định nghĩa schema SQL Server cho toàn hệ thống. Các bảng được chia theo nhóm nghiệp vụ như người dùng, phòng ban, nhân viên, bệnh nhân, lịch hẹn, khám bệnh, xét nghiệm, hình ảnh, dược, thanh toán và ca làm.

Bảng 3.6.1: Nhóm bảng dữ liệu chính

Nhóm dữ liệu	Bảng tiêu biểu
Phân quyền	Roles, Users, Sessions, Notifications.
Cơ sở vật chất	Departments, Rooms.
Nhân sự	Employees, Shifts, EmployeeShifts, DoctorSchedules.
Bệnh nhân	Patients, PatientInsurance, PatientIdentifiers.
Khám bệnh	Appointments, Encounters, PatientQueues, QueueHistory.
Hồ sơ y tế	MedicalRecords, MedicalRecordFiles, VitalSigns, Files.
Cận lâm sàng	Services, EncounterServices, LabRequests, LabResults, ImagingRequests, ImagingResults.

Nhóm dữ liệu	Bảng tiêu biểu
Dược	Medicines, Prescriptions, PrescriptionDetails, Dispense.
Thanh toán	Payments, PaymentDetails, Invoices, InvoiceDetails.
Quản trị	AuditLogs, AuditLogDetails.

3.7 Quan hệ dữ liệu

```

1 Roles      -> Users
2 Users      -> Employees
3 Departments -> Rooms
4 Departments -> Employees
5 Patients    -> Appointments
6 Appointments -> Encounters
7 Encounters  -> MedicalRecords
8 Encounters  -> EncounterServices
9 Encounters  -> Prescriptions
10 Encounters -> Payments
11 Prescriptions -> PrescriptionDetails -> Medicines
12 Payments   -> PaymentDetails
13 Invoices    -> InvoiceDetails
14 Employees   -> DoctorSchedules
15 Employees   -> EmployeeShifts

```

3.8 Thiết kế từ điển dữ liệu tiêu biểu

Bảng 3.8.1: Từ điển dữ liệu rút gọn

Bảng	Khóa chính	Ý nghĩa
Users	UserID	Lưu tài khoản, mật khẩu băm, email, role và trạng thái hoạt động.
Employees	EmployeeID	Lưu thông tin nhân viên, vai trò, khoa, tài khoản liên kết và mã định danh UUID.
Patients	PatientID	Lưu thông tin bệnh nhân, mã bệnh nhân, họ tên, ngày sinh, giới tính, liên hệ.
Appointments	AppointmentID	Lưu lịch hẹn giữa bệnh nhân, bác sĩ, khoa, phòng và trạng thái lịch.
Encounters	EncounterID	Lưu lượt khám, thời gian bắt đầu/kết thúc và trạng thái xử lý.
LabRequests	LabID	Lưu yêu cầu xét nghiệm do bác sĩ tạo.
ImagingRequests	ImagingRequestID	Lưu yêu cầu chẩn đoán hình ảnh, bộ phận chụp, mức ưu tiên và trạng thái.

Bảng	Khóa chính	Ý nghĩa
Prescriptions	PrescriptionID	Lưu đơn thuốc theo encounter và bác sĩ kê toa.
Payments	PaymentID	Lưu thông tin thanh toán, số tiền, phương thức và trạng thái.

3.8.1 Nhóm bảng phân quyền và nhân sự

Bảng 3.8.2: Từ điển dữ liệu nhóm phân quyền và nhân sự

Bảng/Cột	Kiểu dữ liệu	Diễn giải
Users.UserID	int	Khóa chính của tài khoản người dùng.
Users.Username	nvarchar	Tên đăng nhập dùng tại LoginForm.
Users.PasswordHash	nvarchar	Mật khẩu đã băm, không lưu mật khẩu gốc.
Users.RoleID	int	Khóa ngoại đến bảng vai trò, dùng để phân quyền dashboard.
Users.IsActive	bit	Trạng thái cho phép hoặc khóa đăng nhập.
Employees.EmployeeID	int	Khóa chính nhân viên, được lưu vào UserSession.
Employees.FullName	nvarchar	Họ tên nhân viên hiển thị trên dashboard.
Employees.DepartmentID	int	Khoa/phòng ban của nhân viên.
Employees.UserID	int	Tài khoản liên kết với nhân viên.
EmployeeShifts.ShiftID	int	Ca làm mà nhân viên đăng ký hoặc được phân công.
DoctorSchedules.RoomID	int	Phòng khám bác sĩ sử dụng trong lịch làm.

3.8.2 Nhóm bảng bệnh nhân, lịch hẹn và hàng chờ

Bảng 3.8.3: Từ điển dữ liệu nhóm tiếp nhận

Bảng/Cột	Kiểu dữ liệu	Diễn giải
Patients.PatientID	int	Khóa chính bệnh nhân trong SQL Server.

Bảng/Cột	Kiểu dữ liệu	Diễn giải
Patients.PatientCode	nvarchar	Mã bệnh nhân dùng để tra cứu nhanh tại lễ tân.
Patients.FullName	nvarchar	Họ tên bệnh nhân.
Patients.DOB	date	Ngày sinh bệnh nhân.
Patients.Gender	nvarchar	Giới tính, hiển thị trong hồ sơ và lịch hẹn.
Patients.Phone	nvarchar	Số điện thoại dùng tìm kiếm và nhắc lịch.
Appointments.AppointmentID	int	Khóa chính lịch hẹn.
Appointments.PatientID	int	Bệnh nhân được đặt lịch.
Appointments.DoctorID	int	Bác sĩ phụ trách lịch khám.
Appointments.RoomID	int	Phòng khám dự kiến.
Appointments.Status	nvarchar	Trạng thái lịch: chờ, đã check-in, hủy, hoàn tất.
PatientQueues.QueueID	int	Khóa chính hàng chờ.
PatientQueues.Status	nvarchar	Trạng thái trong hàng chờ: waiting, in-progress, done.

3.8.3 Nhóm bảng khám bệnh, cận lâm sàng và hồ sơ

Bảng 3.8.4: Từ điển dữ liệu nhóm lâm sàng

Bảng/Cột	Kiểu dữ liệu	Diễn giải
Encounters.EncounterID	int	Khóa chính của lượt khám.
Encounters.PatientID	int	Bệnh nhân của lượt khám.
Encounters.DoctorID	int	Bác sĩ phụ trách.
MedicalRecords.RecordID	int	Hồ sơ bệnh án gắn với encounter.
VitalSigns.VitalID	int	Bản ghi sinh hiệu do điều dưỡng nhập.
VitalSigns.Temperature	decimal	Nhiệt độ bệnh nhân.
VitalSigns.BloodPressure	nvarchar	Huyết áp, thường nhập dạng tâm thu/tâm trương.
LabRequests.LabID	int	Yêu cầu xét nghiệm.
LabRequests.Status	nvarchar	Trạng thái request xét nghiệm.

Bảng/Cột	Kiểu dữ liệu	Diễn giải
LabResults.ResultID	int	Kết quả xét nghiệm trả về.
ImagingRequests.ImagingRequestID	int	Yêu cầu chẩn đoán hình ảnh.
ImagingResults.FileID	int	File ảnh/PDF kết quả được lưu trong hệ thống.

3.8.4 Nhóm bảng được và thanh toán

Bảng 3.8.5: Từ điển dữ liệu nhóm được và thanh toán

Bảng/Cột	Kiểu dữ liệu	Diễn giải
Medicines.MedicineID	int	Khóa chính thuốc.
Medicines.MedicineName	nvarchar	Tên thuốc hiển thị trong nhà thuốc.
Medicines.Unit	nvarchar	Đơn vị tính: viên, chai, hộp, ống.
Medicines.StockQuantity	int	Số lượng tồn kho.
Prescriptions.PrescriptionID	int	Khóa chính đơn thuốc.
PrescriptionDetails.Quantity	int	Số lượng thuốc bác sĩ kê.
PrescriptionDetails.Dosage	nvarchar	Liều dùng hướng dẫn cho bệnh nhân.
Payments.PaymentID	int	Khóa chính thanh toán.
Payments.Amount	decimal	Tổng số tiền cần thanh toán.
Payments.Method	nvarchar	Phương thức thanh toán: tiền mặt, chuyển khoản, PayOS.
Payments.Status	nvarchar	Trạng thái: pending, paid, failed hoặc canceled.
Invoices.InvoiceID	int	Hóa đơn được tạo sau thanh toán.

4 Xây dựng và Cài đặt chương trình

4.1 Tổ chức project và kiến trúc triển khai

4.1.1 Tổ chức project trong Visual Studio

Solution ClinicManagementSystem.WinForms.sln gồm các project:

Bảng 4.1.1: Tổ chức project trong solution

Project	Vai trò
ClinicManagementSystem.WinForms	Project giao diện WinForms, chứa Program.cs, Mainforms, Forms, UserControls, Controllers.
BUS	Xử lý nghiệp vụ cho Auth, Clinical, Facility, Scheduling, Staff, Technician và Integrations.
DAL	Truy cập SQL Server, repository, interface và DatabaseHelper.
DTO	DTO truyền dữ liệu cho các domain Auth, Clinical, Facility, Staff, Scheduling, Integrations.
Models	Lớp model đại diện thực thể hệ thống.
Core	Hằng số, vai trò, phân quyền, session, tiện ích sinh mã và xử lý mật khẩu.
ServicesExternal	Lớp tích hợp PayOS và helper xử lý ảnh/base64.

4.1.2 Phân tích cấu trúc thư mục WinForms

Project ClinicManagementSystem.WinForms là phần thể hiện rõ nhất yêu cầu của môn học. Thay vì gom tất cả giao diện vào một thư mục duy nhất, mã nguồn được chia theo vai trò và theo loại thành phần giao diện. Cách chia này giúp người phát triển nhanh chóng xác định vị trí màn hình cần sửa và tránh làm phình to một form tổng.

Bảng 4.1.2: Cấu trúc thư mục giao diện WinForms

Thư mục	Thành phần tiêu biểu	Ý nghĩa thiết kế
Mainforms	AdminMainform, ReceptionistMainform, DoctorMainform, NurseMainform, PharmacyMainform, TechnicianMainform	Đóng vai trò dashboard shell theo từng role, giữ menu, header và vùng content.
Forms	LoginForm, các form nhập liệu hoặc popup	Dùng cho màn hình độc lập hoặc thao tác cần modal.
UserControls/Receptionist	PatientManagement, CreateAppointment, ScheduleToday, WaitingList, Payment	Màn hình nghiệp vụ của lễ tân, nạp động vào ReceptionistMainform.

Thư mục	Thành phần tiêu biểu	Ý nghĩa thiết kế
UserControls/Nurse	Tổng quan, sinh hiệu, hàng chờ, phòng, ca làm	Hỗ trợ nhập dữ liệu trước khám và theo dõi hàng chờ.
UserControls/Technician	TechnicianRequests, upload MRI/PDF, lab result, records, shifts	Màn hình xử lý request cận lâm sàng và đồng bộ dữ liệu kiểm thử.
UserControls/Pharmacy	ucPharmacyOverview, ucPrescriptionDispense, ucMedicineCatalog, ucInventoryManagement	Màn hình cấp phát thuốc, danh mục thuốc và quản lý tồn kho.
Shareforms/ERM	ucOverview, ucHistory, ucLab, ucImaging, ucPrescription, frmPacsViewer	Thành phần hồ sơ bệnh án dùng chung, phục vụ bác sĩ và các màn hình cần xem hồ sơ.
Shareforms/WorkingShiftForm	Form và user control đăng ký/xem ca làm	Tái sử dụng chức năng ca làm cho nhiều vai trò.
Controllers	Controller cho appointment, payment, waiting list, ERM	Giảm logic trong code-behind, gom thao tác gọi BUS và chuẩn bị dữ liệu cho UI.

Việc tách thư mục theo vai trò phù hợp với mô hình phòng khám vì mỗi nhóm người dùng có nghiệp vụ riêng. Một lỗi ở màn hình nhà thuốc ít có khả năng ảnh hưởng trực tiếp đến màn hình kỹ thuật viên nếu code không bị trộn lẫn. Khi báo cáo đồ án, đây là một điểm quan trọng để chứng minh nhóm không chỉ kéo thả giao diện mà đã tổ chức project theo hướng có thể bảo trì.

4.1.3 Quy ước thiết kế form và control

Trong WinForms, chất lượng ứng dụng phụ thuộc nhiều vào sự nhất quán của form. Cùng một project nếu mỗi màn hình đặt màu, font, padding, bảng dữ liệu và nút bấm theo một kiểu khác nhau sẽ tạo cảm giác rời rạc. Dựa trên mã nguồn hiện có, báo cáo đề xuất và mô tả các quy ước sau để thống nhất giao diện:

Bảng 4.1.3: Quy ước thiết kế form/control trong đồ án

Nhóm quy ước	Cách áp dụng	Lợi ích
Đặt tên control	TextBox bắt đầu bằng txt, Button bằng btn, DataGridView bằng dgv, Label bằng lbl, Panel bằng pnl hoặc panel.	Dễ đọc code-behind và dễ xác định control khi xử lý event.
Điều hướng	Main form chỉ nạp user control vào content panel, không mở nhiều form con không cần thiết.	Trải nghiệm giống dashboard, giảm cửa sổ chồng lấn.
Bố cục	Dùng DockStyle.Fill cho màn hình con; dùng TableLayoutPanel cho bố cục hàng/cột; dùng FlowLayoutPanel cho danh sách card.	Giao diện co giãn tốt hơn khi resize.
Bảng dữ liệu	Tắt AutoGenerateColumns khi cần cột cố định; đặt ReadOnly; chọn cả dòng bằng FullRowSelect.	Tránh người dùng sửa trực tiếp sai cách và giúp thao tác trên bản ghi rõ ràng.
Thông báo	Dùng MessageBox.Show cho lỗi ngắn, xác nhận thao tác bằng YesNo.	Người dùng nhận phản hồi rõ ràng trước khi thay đổi dữ liệu.
Tải dữ liệu	Tách hàm LoadData, RefreshData, BindData.	Dễ reload sau khi thêm/sửa/xóa hoặc đóng modal.
Xử lý lỗi	Bắt lỗi ở tầng UI cho dữ liệu nhập sai; tầng BUS trả kết quả nghiệp vụ; tầng DAL chỉ báo lỗi truy cập dữ liệu.	Tránh để exception thô hiện trực tiếp với người dùng.

4.1.4 Luồng dữ liệu từ UI đến SQL Server

Một thao tác WinForms hoàn chỉnh trong project thường đi qua nhiều lớp. Ví dụ khi lễ tân tìm kiếm bệnh nhân, thao tác không đi thẳng từ TextBox xuống SQL Server. Form lấy keyword, controller hoặc BUS kiểm tra keyword, DAL tạo truy vấn có tham số, SQL Server trả kết quả, DAL map sang DTO, UI bind lại vào DataGridView.

```

1 private void btnSearch_Click(object sender, EventArgs e)
2 {
3     string keyword = txtSearch.Text.Trim();
4     List<PatientDTO> patients = patientBUS.SearchPatients(keyword);
5     dgvPatients.DataSource = patients;
6 }
7
8 public List<PatientDTO> SearchPatients(string keyword)
9 {
10     return patientDAL.Search(keyword);
11 }

```

Luồng này giúp báo cáo thể hiện rõ tư duy thiết kế phần mềm Desktop: giao diện là nơi nhận thao tác, nhưng không phải nơi chứa tất cả truy vấn và luật nghiệp vụ. Khi thay đổi giao diện từ DataGridView sang card hoặc thêm bộ lọc nâng cao, tầng DAL/BUS vẫn có thể được tái sử dụng.

4.1.5 Cài đặt vòng đời ứng dụng WinForms

Điểm khởi động của ứng dụng WinForms nằm trong `Program.cs`. Đây là nơi thiết lập visual style của Windows, cấu hình cách render text và mở form đầu tiên. Với phần mềm quản lý phòng khám, form đầu tiên phải là `LoginForm` vì mọi chức năng đều phụ thuộc vào tài khoản, vai trò và nhân viên đang đăng nhập.

```

1 [STAThread]
2 static void Main()
3 {
4     Application.EnableVisualStyles();
5     Application.SetCompatibleTextRenderingDefault(false);
6     Application.Run(new LoginForm());
7 }

```

Thuộc tính `[STAThread]` là yêu cầu quen thuộc của ứng dụng Desktop Windows khi làm việc với các thành phần COM, clipboard, dialog hệ thống hoặc một số control giao diện. `Application.EnableVisualStyles()` giúp control sử dụng giao diện hiện đại theo theme của Windows thay vì kiểu cũ. `Application.Run(new LoginForm())` tạo message loop chính; khi form này đóng, ứng dụng kết thúc.

Trong project, vòng đời đăng nhập được xử lý theo hướng: mở `LoginForm`, kiểm tra tài khoản qua BUS, ghi nhận session bằng `UserSession`, ẩn form đăng nhập, mở dashboard theo vai trò bằng `ShowDialog`, sau đó đóng form đăng nhập khi dashboard kết thúc. Cách làm này đảm bảo ứng dụng không còn giữ form đăng nhập phía sau sau khi người dùng đã vào hệ thống.

4.1.6 Cài đặt dashboard shell và vùng nội dung động

Các main form theo vai trò đóng vai trò như một shell: giữ layout tổng, menu trái, header, thông tin người dùng, nút đăng xuất và vùng nội dung chính. Khi người dùng chọn chức năng, shell không mở một cửa sổ mới mà thay thế nội dung bên trong panel trung tâm.

```

1 private void LoadContentView(UserControl view)
2 {
3     contentPanel.SuspendLayout();
4     contentPanel.Controls.Clear();
5
6     view.Dock = DockStyle.Fill;
7     contentPanel.Controls.Add(view);
8
9     contentPanel.ResumeLayout();
10 }

```

Việc dùng SuspendLayout và ResumeLayout giúp hạn chế vẽ lại giao diện nhiều lần khi thay control. Đây là kỹ thuật nhỏ nhưng quan trọng trong WinForms, nhất là khi vùng nội dung có nhiều control con hoặc màn hình chứa DataGridView. Nếu không dùng hợp lý, giao diện có thể bị nhấp nháy hoặc chậm khi chuyển màn hình.

Thiết kế shell còn giúp chuẩn hóa hành vi đăng xuất. Các form chính có thể phát sự kiện LogoutRequested hoặc CloseRequested; MainFormRole nhận sự kiện này để đóng dashboard và quay về luồng đăng nhập nếu cần. Nhờ vậy, mỗi dashboard không cần tự xử lý toàn bộ logic đóng/mở ứng dụng.

4.1.7 Cài đặt UserControl theo phân hệ

UserControl là đơn vị giao diện chính của các phân hệ. Mỗi user control nên đảm nhiệm một màn hình nghiệp vụ cụ thể, ví dụ quản lý bệnh nhân, tạo lịch hẹn, lịch hôm nay, danh sách chờ, upload kết quả, danh sách đơn thuốc hoặc quản lý tồn kho. Việc chia nhỏ thành user control giúp mã nguồn dễ bảo trì hơn so với việc đưa mọi chức năng vào một form lớn.

Một user control trong project thường có cấu trúc:

- Constructor nhận tham số cần thiết như user hiện tại, mã nhân viên, mã request hoặc controller.
- Hàm LoadData hoặc RefreshData gọi BUS/controller để lấy dữ liệu.
- Các event handler xử lý click, chọn dòng, tìm kiếm, lọc dữ liệu hoặc mở form chi tiết.
- Các hàm phụ để bind dữ liệu, đổi trạng thái button, hiển thị thông báo và reset input.

```

1 public partial class PatientManagement : UserControl
2 {
3     private readonly PatientBUS patientBUS = new PatientBUS();
4
5     public PatientManagement()
6     {
7         InitializeComponent();
8         LoadPatients();
9     }
10
11     private void LoadPatients()
12     {
13         dgvPatients.AutoGenerateColumns = false;
14         dgvPatients.DataSource = patientBUS.GetAllPatients();
15     }
16
17     private void btnSearch_Click(object sender, EventArgs e)
18     {
19         string keyword = txtSearch.Text.Trim();

```



```

20     dgvPatients.DataSource = patientBUS.SearchPatients(keyword);
21 }
22 }

```

Điểm cần chú ý là user control chỉ nên xử lý logic giao diện và gọi BUS/controller. Nếu viết SQL trực tiếp trong user control thì kiến trúc DTO–BUS–DAL bị phá vỡ, đồng thời việc kiểm thử và bảo trì sẽ khó hơn.

4.1.8 Cài đặt DataGridView cho nghiệp vụ quản lý

Trong WinForms, DataGridView là control mạnh nhưng dễ bị dùng sai nếu để tự sinh cột, bind trực tiếp dữ liệu thô và xử lý trạng thái không rõ ràng. Với project phòng khám, DataGridView được dùng nhiều nên cần thống nhất cách cài đặt.

```

1 private void ConfigureGrid()
2 {
3     dgvRequests.AutoGenerateColumns = false;
4     dgvRequests.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
5     dgvRequests.MultiSelect = false;
6     dgvRequests.ReadOnly = true;
7     dgvRequests.AllowUserToAddRows = false;
8     dgvRequests.AllowUserToDeleteRows = false;
9     dgvRequests.RowHeadersVisible = false;
10 }

```

Các màn hình danh sách nên dùng chế độ FullRowSelect để người dùng hiểu rằng họ đang thao tác trên một bản ghi hoàn chỉnh. Thuộc tính ReadOnly giúp tránh sửa trực tiếp trên grid nếu hệ thống chưa có cơ chế lưu từng ô. Các thao tác như check-in, thanh toán, upload kết quả, cấp phát thuốc nên được thực hiện qua nút chức năng hoặc cột button rõ ràng thay vì cho sửa trực tiếp vào bảng.

Bảng 4.1.4: Ứng dụng DataGridView trong các phân hệ

Phân hệ	Dữ liệu hiển thị	Thao tác chính
Lễ tân	Bệnh nhân, lịch hẹn, hàng chờ, thanh toán	Tìm kiếm, check-in, tạo lịch, mở thanh toán.
Bác sĩ	Bệnh nhân chờ khám, lịch sử khám, dịch vụ chỉ định	Mở ERM, ghi chẩn đoán, kê toa, chỉ định xét nghiệm.
Kỹ thuật viên	Request xét nghiệm/hình ảnh, kết quả, hồ sơ bệnh nhân	Nhận request, upload file, nhập kết quả, xem hồ sơ.
Dược sĩ	Đơn thuốc, danh mục thuốc, tồn kho	Cấp phát, xem chi tiết thuốc, cập nhật tồn kho.
Quản trị	Người dùng, nhân viên, phòng ban, cấu hình	Thêm, sửa, khóa tài khoản, phân quyền.

4.1.9 Cài đặt FlowLayoutPanel và card động

Không phải dữ liệu nào cũng nên đưa vào bảng. Khi người dùng cần chọn một đối tượng trực quan như khoa, bác sĩ, thuốc hoặc nhóm chức năng, card động trong FlowLayoutPanel cho trải nghiệm tốt hơn. Trong màn hình tạo lịch hẹn, danh sách khoa và bác sĩ có thể được tạo theo dữ liệu hiện có; mỗi card gắn event click để chọn đối tượng.

```
1 private void AddDepartmentCard(DepartmentDTO department)
2 {
3     Panel card = new Panel
4     {
5         Width = 180,
6         Height = 90,
7         Margin = new Padding(8),
8         Cursor = Cursors.Hand,
9         Tag = department
10    };
11
12    Label lblName = new Label
13    {
14        Text = department.DepartmentName,
15        Dock = DockStyle.Fill,
16        TextAlign = ContentAlignment.MiddleCenter
17    };
18
19    card.Controls.Add(lblName);
20    card.Click += Department_Click;
21    lblName.Click += Department_Click;
22    flpDepartment.Controls.Add(card);
23 }
```

Khi tạo card động, cần gán Tag để lưu DTO hoặc ID nghiệp vụ. Event click có thể đọc lại Tag để biết người dùng chọn khoa/bác sĩ/thuốc nào. Đây là kỹ thuật phổ biến trong WinForms vì control động không có biến thiết kế cố định như control được kéo thả trên Designer.

4.1.10 Cài đặt form modal, OpenFileDialog và MessageBox

Ứng dụng Desktop thường cần tương tác với hệ thống file cục bộ. Trong phân hệ kỹ thuật viên, người dùng upload PDF hoặc hình ảnh MRI/X-Ray; trong thanh toán, người dùng có thể xem QR hoặc kiểm tra trạng thái. WinForms cung cấp các dialog hệ thống như OpenFileDialog, SaveFileDialog, FolderBrowserDialog và MessageBox.

```
1 private void btnChooseFile_Click(object sender, EventArgs e)
2 {
3     using (OpenFileDialog dialog = new OpenFileDialog())
4     {
5         dialog.Filter = "PDF files (*.pdf)|*.pdf|Image files (*.png;*.jpg)|*.png;*.jpg";
6         dialog.Multiselect = false;
7
8         if (dialog.ShowDialog() == DialogResult.OK)
9         {
10             txtFilePath.Text = dialog.FileName;
11         }
12     }
13 }
```

MessageBox được dùng cho các thông báo ngắn như đăng nhập sai, tạo lịch thành công, thiếu dữ liệu bắt buộc hoặc xác nhận thao tác. Với thao tác làm thay đổi dữ liệu như xóa, hủy lịch, xác nhận thanh toán hoặc cấp phát thuốc, cần dùng nút YesNo để người dùng xác nhận trước khi thực thi.

4.1.11 Cài đặt bất đồng bộ cho API và thanh toán

WinForms chạy trên UI thread. Nếu gọi API, đọc file lớn hoặc kiểm tra thanh toán trực tuyến trực tiếp trên UI thread, giao diện sẽ bị đứng. Vì vậy, các thao tác mạng trong project cần dùng async/await. Khi bắt đầu xử lý, form nên disable nút thao tác; khi hoàn tất hoặc lỗi, bật lại nút trong khối finally.

```
1 private async void btnSync_Click(object sender, EventArgs e)
2 {
3     btnSync.Enabled = false;
4     lblStatus.Text = "Đang đồng bộ dữ liệu...";
5
6     try
7     {
8         ApiSyncResultDTO result = await apiSyncBUS.SyncAsync();
9         lblStatus.Text = result.Success ? "Đồng bộ thành công" : result.Message;
10    }
11    catch (Exception ex)
12    {
13        MessageBox.Show(ex.Message, "Lỗi đồng bộ");
14    }
15    finally
16    {
17        btnSync.Enabled = true;
18    }
19 }
```

Đây là điểm quan trọng với ứng dụng Desktop thực tế. Người dùng cần thấy phần mềm vẫn phản hồi trong lúc đồng bộ Supabase/SheetDB hoặc kiểm tra PayOS. Nếu không xử lý bất đồng bộ, Windows có thể đánh dấu form là *Not Responding*, làm trải nghiệm rất kém.

4.1.12 Quản lý session và truyền ngữ cảnh người dùng

Sau khi đăng nhập, nhiều màn hình cần biết người dùng hiện tại là ai, thuộc vai trò nào, mã nhân viên nào và khoa/phòng ban nào. Project dùng `UserSession` trong `Core/Session` để lưu ngữ cảnh đăng nhập. Các dashboard cũng nhận `UserDTO currentUser` qua constructor để hiển thị tên người dùng và quyết định chức năng.

```
1 UserSession.Login(
2     user.UserID,
3     user.EmployeeID,
4     RoleNormalizer.Normalize(user.Role),
5     user.DepartmentName);
```

Về thiết kế, session giúp tránh truyền lặp lại nhiều tham số qua tất cả constructor. Tuy nhiên, các màn hình quan trọng vẫn nên nhận rõ thông tin cần thiết, ví dụ `employeeId` khi đăng ký ca làm hoặc `requestId` khi upload kết quả, để code dễ đọc và giảm phụ thuộc toàn cục.

4.1.13 Cài đặt điểm khởi động và phân quyền

Ứng dụng bắt đầu từ `Program.cs`, bật visual style và mở `LoginForm`. Sau khi đăng nhập thành công, `LoginForm` gọi `UserSession.Login` để lưu thông tin người dùng và mở `MainformRole`. Lớp `MainformRole` chuẩn hóa vai trò, sau đó tạo dashboard tương ứng.

```

1 private void btnLogin_Click(object sender, EventArgs e)
2 {
3     string username = txtUsername.Text.Trim();
4     string password = txtPassword.Text;
5
6     UserDTO user = userBUS.Login(username, password);
7     if (user == null)
8     {
9         MessageBox.Show("Tên đăng nhập hoặc mật khẩu không đúng");
10        return;
11    }
12
13    UserSession.Login(
14        user.UserID,
15        user.EmployeeID,
16        RoleNormalizer.Normalize(user.Role),
17        user.DepartmentName);
18
19    using (var dashboard = new MainformRole(user))
20    {
21        Hide();
22        dashboard.ShowDialog();
23        Close();
24    }
25 }

```

Bảng 4.1.5: Ánh xạ vai trò sang giao diện chính

Vai trò	Form chính	Ghi chú
Admin	AdminMainform	Quản trị hệ thống.
Receptionist	ReceptionistMainform	Tiếp nhận, lịch hẹn, bệnh nhân, thanh toán.
Doctor	DoctorMainform	Khám bệnh và hồ sơ bệnh án.
Nurse	NurseMainform	Điều dưỡng, sinh hiệu, hàng chờ.
Pharmacist	PharmacyMainform	Nhà thuốc và cấp phát thuốc.
Technician	TechnicianMainform	Xét nghiệm, hình ảnh và upload kết quả.

4.2 Cài đặt các tầng DTO, DAL, BUS và Core

4.2.1 Cài đặt tầng DTO

Tầng DTO được chia theo domain, ví dụ DTO/Auth/UserDTO.cs, DTO/Clinical/Patient/PatientDTO.cs, DTO/Clinical/Payments/PaymentWaitingDTO.cs, DTO/Scheduling/AppointmentDTO.cs. DTO giúp giao diện không phụ thuộc trực tiếp vào DataTable hoặc entity database.

DTO trong project không phải chỉ là vài class minh họa. Đây là lớp hợp đồng dữ liệu giữa UI, BUS, DAL và API. Khi một màn hình WinForms cần dữ liệu, nó không nên nhận DataRow hoặc SqlDataReader; thay vào đó DAL/BUS trả về DTO có tên thuộc tính rõ ràng. Điều này giúp code giao diện dễ đọc và giảm phụ thuộc vào tên cột SQL.

Bảng 4.2.1: Phân nhóm DTO trong project

Nhóm DTO	File tiêu biểu	Vai trò
Auth	UserDTO	Truyền dữ liệu tài khoản, nhân viên, vai trò sau đăng nhập.
Patient	PatientDTO, PatientInsuranceDTO	Thông tin hành chính bệnh nhân, bảo hiểm, dị ứng, nhóm máu.
Encounter	EncounterDTO, VitalSignsDTO, FollowUpDTO	Lượt khám, sinh hiệu, tái khám.
ERM	PatientERMDto, EncounterHistoryDto, LabHistoryDto, ImagingHistoryDto, PrescriptionHistoryDto, InvoiceHistoryDto	Dữ liệu tổng hợp cho hồ sơ bệnh án điện tử.
Payment	PaymentWaitingDTO, PaymentDetailDTO, Payment_RecepDTO	Danh sách chờ thanh toán, chi tiết invoice, lịch sử thanh toán.
Prescription	PrescriptionDTO, PrescriptionItemDTO, MedicineDTO	Đơn thuốc, chi tiết thuốc, danh mục thuốc.
Queue	WaitingPatientDTO, DoctorQueueDTO, DoctorCardDTO	Hàng chờ theo khoa/bác sĩ và card bác sĩ.
Facility	DepartmentDTO, RoomDTO, InventoryDTO, SupplierDTO	Khoa, phòng, kho thuốc, nhà cung cấp.
Scheduling	AppointmentDTO, DoctorScheduleDTO, EmployeeShiftDTO, WorkShiftDTO	Lịch hẹn, lịch bác sĩ, ca làm.
Integrations	ApiPatientDTO, ApiEmployeeDTO, ApiRequestDTO, ApiSyncResultDTO	Dữ liệu trung gian khi đồng bộ Supabase/SheetDB.

Thiết kế DTO có ba nguyên tắc chính:

- **DTO theo màn hình:** những màn hình tổng hợp như ERM hoặc Payment không nên ép dùng lại DTO bảng gốc nếu dữ liệu cần hiển thị là kết quả join nhiều bảng.
- **DTO theo tích hợp:** dữ liệu API ngoài cần DTO riêng vì tên trường, kiểu dữ liệu và logic UUID khác dữ liệu SQL Server local.
- **DTO không chứa truy vấn:** DTO chỉ chứa thuộc tính, không chứa SQL, không gọi BUS/DAL, không xử lý UI.

```
1 public class PatientDTO
2 {
3     public int PatientID { get; set; }
4     public string PatientCode { get; set; }
5     public string Name { get; set; }
6     public DateTime? BirthDate { get; set; }
7     public string Gender { get; set; }
8     public string Phone { get; set; }
9     public string Address { get; set; }
10    public string BloodType { get; set; }
11    public string Allergies { get; set; }
12 }
```

```
1 public class ApiSyncResultDTO
2 {
3     public int InsertedPatientsToLocal { get; set; }
4     public int InsertedEmployeesToLocal { get; set; }
5     public int PatchedPatientUuidToSheet { get; set; }
6     public int PatchedEmployeeUuidToSheet { get; set; }
7     public int SkippedPatientRows { get; set; }
8     public int SkippedEmployeeRows { get; set; }
9
10    public List<ApiPatientDTO> MergedPatients { get; set; }
11        = new List<ApiPatientDTO>();
12
13    public List<ApiEmployeeDTO> MergedEmployees { get; set; }
14        = new List<ApiEmployeeDTO>();
15 }
```

DTO kết quả đồng bộ không chỉ trả true/false. Nó ghi lại số dòng insert local, số UUID sinh mới, số UUID patch ngược về SheetDB, số dòng bị bỏ qua và danh sách bệnh nhân/nhân viên sau merge. Đây là cách thiết kế tốt hơn cho giao diện WinForms vì form có thể hiển thị thông báo chi tiết cho người dùng, thay vì chỉ báo “đồng bộ thành công”.

4.2.2 Cài đặt tầng DAL bằng ADO.NET

DAL sử dụng DatabaseHelper để quản lý kết nối và thực thi truy vấn. Các repository tạo câu SQL, truyền SqlParameter, gọi helper và map kết quả sang DTO.

Tầng DAL hiện tại được chia thành interface và repository theo domain. Ví dụ: IUserDAL/UserDAL cho đăng nhập, IPatientRepository/Patient_DAL cho bệnh nhân, IServiceRequestDAL/ServiceRequestDAL cho chỉ định, ITechnicianRequestDAL/TechnicianRequestDAL cho kỹ thuật viên. Cách chia này làm rõ mỗi repository phụ trách một nhóm bảng và một nhóm truy vấn.

Bảng 4.2.2: Nhóm repository DAL và trách nhiệm

Nhóm DAL	Repository tiêu biểu	Trách nhiệm chính
Auth	UserDAL	Lấy tài khoản, role, nhân viên và xác thực đăng nhập.
Clinical	Patient_DAL, EncounterDAL, ERMRepository, PaymentDAL, PrescriptionDAL, VitalSignsDAL	Truy vấn dữ liệu khám bệnh, hồ sơ, thanh toán, đơn thuốc.
Facility	DepartmentDAL, RoomDAL, InventoryDAL, SupplierDAL	Khoa, phòng, tồn kho, nhà cung cấp.
Scheduling	AppointmentDAL, DoctorScheduleDAL, EmployeeShiftDAL, WorkShiftDAL	Lịch hẹn, lịch bác sĩ, ca làm.
Technician	TechnicianRequestDAL, TechnicianShiftDAL	Danh sách request, upload kết quả, ca kỹ thuật viên.
Integrations	ApiSyncRepository	Gọi Supabase/SheetDB, patch UUID, upsert local, kéo request ngoài.

DatabaseHelper đóng vai trò cổng truy cập SQL Server dùng chung. Nó cung cấp ba nhóm thao tác: ExecuteQuery trả DataTable cho truy vấn SELECT; ExecuteNonQuery cho INSERT/UPDATE/DELETE; ExecuteScalar cho truy vấn trả một giá trị. Tất cả đều nhận SqlParameter[] để tránh nối chuỗi SQL trực tiếp.

```

1 public List<PatientDTO> SearchPatients(string keyword)
2 {
3     const string query = @"
4         SELECT PatientID, PatientCode, FullName AS Name, DOB AS BirthDate,
5             Gender, Phone, Address, BloodType, Allergies
6         FROM Patients
7         WHERE FullName LIKE @Term
8             OR PatientCode LIKE @Term
9             OR Phone LIKE @Term";
10
11     SqlParameter[] parameters =
12     {
13         new SqlParameter("@Term", "%" + keyword + "%")
14     };
15
16     DataTable table = DatabaseHelper.ExecuteQuery(query, parameters);
17     return table.AsEnumerable().Select(MapPatient).ToList();
18 }

```

Điểm quan trọng là các truy vấn có dữ liệu nhập từ người dùng đều nên dùng tham số, tránh nối chuỗi SQL trực tiếp.

```
1 public static int ExecuteNonQuery(  
2     string query,  
3     SqlParameter[] parameters = null)  
4 {  
5     using (SqlConnection conn = GetConnection())  
6     using (SqlCommand cmd = new SqlCommand(query, conn))  
7     {  
8         if (parameters != null)  
9         {  
10             cmd.Parameters.AddRange(parameters);  
11         }  
12  
13         conn.Open();  
14         return cmd.ExecuteNonQuery();  
15     }  
16 }
```

Đối với repository tích hợp API, DAL không chỉ truy cập SQL Server mà còn gọi HTTP API bằng HttpClient. Vì vậy, ApiSyncRepository có hai trách nhiệm dữ liệu: gọi nguồn ngoài và ghi về local. Các hàm như GetSupabasePatientsAsync, GetSheetPatientsAsync, PatchPatientSheetUuidAsync, InsertSupabasePatientAsync, UpsertLocalPatientAsync cho thấy repository này là cầu nối giữa ba hệ dữ liệu.

4.2.3 Cài đặt tầng DAL bằng Entity Framework theo hướng mở rộng

Bên cạnh ADO.NET, đồ án có định hướng bổ sung Entity Framework cho các phân hệ CRUD ổn định. Phần này chưa thay thế DAL hiện tại, nhưng cần được thiết kế ngay trong báo cáo để thể hiện hướng phát triển có kiểm soát. Nếu triển khai EF một cách vội vàng, hệ thống sẽ có nguy cơ trộn lẫn entity EF với DTO giao diện, làm mất ranh giới kiến trúc. Vì vậy, cách cài đặt đề xuất là thêm một nhánh repository EF song song với repository ADO.NET.

Nguyên tắc tách ADO.NET và Entity Framework ADO.NET và EF không nên được dùng lẫn trong cùng một phương thức nghiệp vụ nếu không có lý do rõ ràng. ADO.NET phù hợp với các truy vấn đã tối ưu bằng SQL, nhiều join, nhiều aggregate hoặc cần kiểm soát câu lệnh. EF phù hợp với CRUD chuẩn, danh mục ít phức tạp, truy vấn theo khóa chính và cập nhật entity đơn giản.

Bảng 4.2.3: Định hướng chọn ADO.NET hoặc Entity Framework trong DAL

Nhóm nghiệp vụ	Cách truy cập phù hợp	Lý do
Đăng nhập, phân quyền	ADO.NET	Cần truy vấn chính xác tài khoản, role, employee và kiểm soát dữ liệu trả về.

Nhóm nghiệp vụ	Cách truy cập phù hợp	Lý do
ERM, lịch sử khám	ADO.NET	Nhiều bảng join, dữ liệu tổng hợp, cần query tối ưu theo patient/encounter.
Danh mục thuốc	Entity Framework	CRUD ổn định, dễ map entity sang DTO và dễ lọc/sắp xếp bằng LINQ.
Khoa, phòng, nhà cung cấp	Entity Framework	Bảng danh mục ít nghiệp vụ phức tạp, thao tác thêm/sửa/xóa đơn giản.
Thanh toán, hóa đơn	ADO.NET trước mắt	Có nhiều trạng thái nghiệp vụ và cần kiểm soát transaction khi hoàn thiện.
Ca làm	Có thể dùng EF	Dữ liệu dạng lịch, CRUD và truy vấn theo ngày/nhân viên tương đối rõ.
Đồng bộ API	ADO.NET	Cần upsert, patch UUID và kiểm tra schema linh hoạt.

Cấu trúc Entity và DbContext dự kiến Khi thêm EF, project nên có thêm thư mục như DAL/DataContext/Ef, DAL/Entities và DAL/RepositoriesEf. DbContext chỉ biết entity và connection string; repository EF chịu trách nhiệm query; BUS vẫn chỉ nhận DTO.

```

1 public class ClinicEfContext : DbContext
2 {
3     public DbSet<MedicineEntity> Medicines { get; set; }
4     public DbSet<DepartmentEntity> Departments { get; set; }
5     public DbSet<RoomEntity> Rooms { get; set; }
6     public DbSet<SupplierEntity> Suppliers { get; set; }
7
8     protected override void OnConfiguring(DbContextOptionsBuilder builder)
9     {
10         string connection =
11             ConfigurationManager.ConnectionStrings["ClinicDB"].ConnectionString;
12
13         builder.UseSqlServer(connection);
14     }
15 }

```

Entity không nên được thiết kế theo nhu cầu giao diện mà phải phản ánh bảng dữ liệu. Ví dụ MedicineEntity có thể chứa MedicineID, MedicineCode, MedicineName, Unit, StockQuantity, Price, Status. Trong khi đó, MedicineDTO có thể thêm các thuộc tính phục vụ hiển thị như nhãn tồn kho thấp, màu trạng thái hoặc tên nhóm thuốc.

```

1 public class MedicineEntity
2 {

```

```

3 public int MedicineID { get; set; }
4 public string MedicineCode { get; set; }
5 public string MedicineName { get; set; }
6 public string Unit { get; set; }
7 public int StockQuantity { get; set; }
8 public decimal Price { get; set; }
9 public string Status { get; set; }
10 }

```

Repository EF và mapping sang DTO Repository EF không trả entity trực tiếp cho WinForms. Cách làm đúng là query entity rồi project sang DTO. Nhờ đó, nếu sau này đổi EF configuration hoặc đổi tên cột, giao diện không bị phụ thuộc vào entity.

```

1 public class MedicineEfDAL
2 {
3     public List<MedicineDTO> GetMedicines(string keyword)
4     {
5         using (var context = new ClinicEfContext())
6         {
7             var query = context.Medicines.AsQueryable();
8
9             if (!string.IsNullOrEmpty(keyword))
10            {
11                query = query.Where(x =>
12                    x.MedicineName.Contains(keyword) ||
13                    x.MedicineCode.Contains(keyword));
14            }
15
16            return query
17                .OrderBy(x => x.MedicineName)
18                .Select(x => new MedicineDTO
19                {
20                    MedicineID = x.MedicineID,
21                    MedicineCode = x.MedicineCode,
22                    MedicineName = x.MedicineName,
23                    Unit = x.Unit,
24                    StockQuantity = x.StockQuantity,
25                    Price = x.Price,
26                    Status = x.Status
27                })
28                .ToList();
29        }
30    }
31 }

```

Trong WinForms, mỗi lần bấm tìm kiếm hoặc reload danh mục thuốc có thể tạo context mới, lấy dữ liệu, dispose context. Không nên giữ một DbContext sống trong form suốt phiên làm việc vì dữ liệu có thể stale và connection có thể bị giữ lâu.

4.2.4 Cài đặt tầng BUS

BUS nhận dữ liệu từ UI/controller, kiểm tra điều kiện nghiệp vụ và gọi repository. Ví dụ UserBUS kiểm tra username/password rồi trước khi gọi UserDAL.Authenticate; PatientBUS điều phối bệnh nhân và bảo hiểm; PaymentBUS điều phối danh sách chờ thanh toán, tạo payment và cập nhật trạng thái.

Tầng BUS là nơi thể hiện logic nghiệp vụ thật sự. Một lỗi thường gặp trong đồ án WinForms là để form tự quyết định mọi thứ: form kiểm tra dữ liệu, form viết SQL, form cập nhật

trạng thái. Project này đã tách phần lớn logic sang BUS/controller, giúp báo cáo có thể trình bày rõ quy trình từng chức năng.

Bảng 4.2.4: Nhóm BUS và logic nghiệp vụ

Nhóm BUS	Lớp tiêu biểu	Logic xử lý
Auth	UserBUS	Kiểm tra thông tin đăng nhập, gọi DAL, trả UserDTO.
Patient	PatientBUS	Thêm/sửa/tìm kiếm bệnh nhân, chuẩn hóa dữ liệu nhập.
Payment	PaymentBUS	Lấy danh sách chờ thanh toán, tạo pending payment, cập nhật số tiền/trạng thái.
Scheduling	AppointmentBUS, DoctorScheduleBUS	Kiểm tra lịch bác sĩ, tạo lịch hẹn, xử lý trạng thái lịch.
Technician	TechnicianRequestBUS, TechnicianShiftBUS	Lấy request, cập nhật kết quả, quản lý ca kỹ thuật viên.
Integrations	ApiSyncBUS	Hợp nhất dữ liệu Supabase/SheetDB/local, patch UUID, đồng bộ request.

```

1 public bool CreateAppointment(AppointmentDTO appointment)
2 {
3     if (appointment.PatientID <= 0 || appointment.DoctorID <= 0)
4     {
5         return false;
6     }
7
8     var schedule = scheduleDAL.GetDoctorSchedule(
9         appointment.DoctorID,
10        appointment.AppointmentDate);
11
12    if (schedule == null || schedule.RoomID <= 0)
13    {
14        return false;
15    }
16
17    appointment.RoomID = schedule.RoomID;
18    appointment.Status = AppointmentStatus.Pending;
19    return appointmentDAL.Insert(appointment);
20 }

```

Với ApiSyncBUS, logic nghiệp vụ còn phức tạp hơn CRUD thông thường. BUS phải biết dòng nào hợp lệ, dòng nào bị bỏ qua, khi nào dùng mã để liên kết dữ liệu cũ, khi nào sinh UUID mới, khi nào patch về sheet, khi nào insert Supabase. Nếu để logic này trong form đồng bộ, giao diện sẽ rất khó bảo trì. Do đó, form chỉ nên gọi `await apiSyncBUS.SyncAsync()` và hiển thị `result.ToUserMessage()`.

```

1 private async void btnSync_Click(object sender, EventArgs e)
2 {
3     btnSync.Enabled = false;
4
5     try
6     {
7         ApiSyncResultDTO result = await apiSyncBUS.SyncAsync();
8         MessageBox.Show(result.ToUserMessage(), "Dong bo");
9     }
10    finally
11    {
12        btnSync.Enabled = true;
13    }
14 }

```

Cài đặt BUS đăng nhập và phân quyền Luồng đăng nhập không chỉ kiểm tra đúng sai tài khoản mà còn quyết định toàn bộ hướng đi của ứng dụng. Khi người dùng bấm đăng nhập, LoginForm chỉ kiểm tra dữ liệu rỗng và chuyển username/password xuống UserBUS. Tầng BUS gọi DAL để lấy thông tin người dùng, kiểm tra mật khẩu, kiểm tra trạng thái tài khoản, lấy vai trò tương ứng rồi tạo UserSession. Sau khi có session, MainFormRole dùng role đã chuẩn hóa để nạp dashboard phù hợp.

Bảng 4.2.5: Luồng cài đặt đăng nhập và phân quyền

Bước	Thành phần	Xử lý cài đặt
Nhập liệu	LoginForm	Đọc txtUsername, txtPassword; kiểm tra rỗng; hiển thị MessageBox nếu thiếu dữ liệu.
Xác thực	UserBUS	Gọi DAL lấy user, kiểm tra mật khẩu, kiểm tra trạng thái tài khoản và role.
Chuẩn hóa role	RoleNormalizer	Đưa các cách ghi khác nhau như Bác sĩ, Doctor, Bac si về cùng một hằng số.
Tạo phiên	UserSession	Lưu UserID, EmployeeID, EmployeeUUID, RoleName, DepartmentID.
Điều hướng	MainformRole	Tạo dashboard theo role và nhúng vào vùng nội dung bằng DockStyle.Fill.

Điểm quan trọng trong thiết kế này là không để từng form tự kiểm tra quyền. Quyền được xác định một lần sau đăng nhập, sau đó truyền qua UserSession. Các màn hình con chỉ đọc ngữ cảnh hiện tại để lọc dữ liệu theo bác sĩ, kỹ thuật viên hoặc dược sĩ đang đăng nhập.

```

1 public LoginResult Login(string username, string password)
2 {
3     if (string.IsNullOrEmpty(username) ||
4         string.IsNullOrEmpty(password))
5     {
6         return LoginResult.Fail("Thieu ten dang nhap hoac mat khau");
7     }
8
9     UserDTO user = userDAL.GetByUsername(username.Trim());
10    if (user == null || !PasswordHasher.Verify(password, user.PasswordHash))
11    {
12        return LoginResult.Fail("Thong tin dang nhap khong hop le");
13    }
14
15    string normalizedRole = RoleNormalizer.Normalize(user.RoleName);
16    UserSession.SetCurrent(user, normalizedRole);
17    return LoginResult.Success(user);
18 }

```

Cài đặt BUS lịch hẹn, tiếp nhận và hàng chờ Nghiệp vụ lịch hẹn là một chuỗi xử lý liên tục chứ không phải một thao tác CRUD đơn lẻ. Khi lễ tân tạo lịch hẹn, hệ thống phải biết bệnh nhân nào được chọn, khoa nào được chọn, bác sĩ nào còn lịch, slot giờ nào hợp lệ và phòng khám nào tương ứng với lịch làm việc. Khi bệnh nhân đến phòng khám, lịch hẹn được chuyển trạng thái check-in và một dòng hàng chờ được tạo ra để bác sĩ hoặc điều dưỡng tiếp tục xử lý.

Bảng 4.2.6: Logic BUS cho lịch hẹn và hàng chờ

Nghệp vụ	Lớp liên quan	Logic cần cài đặt
Tạo lịch hẹn	AppointmentBUS, DoctorScheduleBUS	Kiểm tra bệnh nhân, bác sĩ, ngày khám, slot giờ; gán phòng theo lịch bác sĩ; lưu trạng thái ban đầu.
Xem lịch hôm nay	AppointmentBUS	Lọc theo ngày hiện tại, trạng thái, khoa hoặc bác sĩ; trả DTO cho ScheduleToday.
Check-in	AppointmentBUS, QueueBUS	Cập nhật trạng thái lịch, sinh số thứ tự hàng chờ, gán phòng/khoa/bác sĩ.
Điều phối hàng chờ	QueueBUS	Lấy danh sách theo trạng thái Waiting, InProgress, Completed; cập nhật khi bác sĩ nhận bệnh.
Nhắc lịch	AppointmentBUS	Lọc lịch sắp tới, lịch tái khám hoặc lịch chưa xác nhận để lễ tân gọi nhắc.

Ở phía WinForms, các thao tác này được chia ra nhiều màn hình để giảm độ phức tạp. CreateAppointment phụ trách chọn dữ liệu đầu vào; ScheduleToday phụ trách xác nhận bệnh

nhân đã đến; WaitingList phụ trách theo dõi luồng sau check-in. Cách chia này đúng với quy trình làm việc thực tế: người tạo lịch chưa chắc là người xử lý hàng chờ, và danh sách trong ngày phải được cập nhật liên tục.

Cài đặt BUS thanh toán và cấp phát Thanh toán trong hệ thống liên quan đến nhiều nguồn chi phí: phí khám, phí dịch vụ cận lâm sàng, thuốc trong toa và các khoản phát sinh. Vì vậy, PaymentBUS không nên chỉ cập nhật một cột trạng thái. Nó cần gom chi tiết hóa đơn, tính tổng tiền, xác định phương thức thanh toán và ghi nhận lịch sử thanh toán.

```
1 public bool ConfirmPayment(int encounterId, string method)
2 {
3     InvoiceDTO invoice = invoiceDAL.GetByEncounter(encounterId);
4     if (invoice == null || invoice.TotalAmount <= 0)
5     {
6         return false;
7     }
8
9     bool updated = paymentDAL.UpdatePaymentStatus(
10         encounterId,
11         PaymentStatus.Paid,
12         method);
13
14     if (!updated)
15     {
16         return false;
17     }
18
19     invoiceDAL.MarkInvoiceAsPaid(invoice.InvoiceID);
20     return true;
21 }
```

Khi thanh toán bằng chuyển khoản hoặc PayOS, WinForms không nên tự coi giao dịch đã hoàn tất ngay sau khi tạo QR. Form PayOS tạo giao dịch, hiển thị QR, sau đó polling hoặc nhận trạng thái qua backend/webhook. Chỉ khi trạng thái trả về là thành công thì PaymentBUS mới cập nhật hóa đơn thành Paid. Nếu người dùng đóng form hoặc giao dịch hết hạn, hóa đơn vẫn giữ trạng thái chờ thanh toán.

Cài đặt BUS kỹ thuật viên và hồ sơ cận lâm sàng Phân hệ kỹ thuật viên có logic đặc thù vì một request có thể là xét nghiệm, PDF hoặc hình ảnh MRI/X-Ray. TechnicianRequestBUS phải phân loại request theo ServiceRequestType, trả danh sách request đang chờ, cập nhật kết quả, cập nhật trạng thái và gắn tệp y tế vào đúng bệnh nhân/encounter.

Bảng 4.2.7: Logic BUS của phân hệ kỹ thuật viên

Luồng	Control/Form	Xử lý BUS/DAL
Danh sách request	ucTechnicianRequests	Lấy request theo trạng thái, nhóm theo bệnh nhân/encounter, trả dữ liệu cho card request.

Luồng	Control/Form	Xử lý BUS/DAL
Nhập kết quả lab	ucTechnicianLabResult	Kiểm tra nội dung kết quả, cập nhật bảng kết quả xét nghiệm, đổi trạng thái request.
Upload MRI/X-Ray	ucTechnicianUploadMri	Chọn file ảnh, kiểm tra đường dẫn, lưu metadata vào hồ sơ y tế, đổi trạng thái request.
Upload PDF	ucTechnicianUploadPdf	Chọn file PDF, gắn vào encounter/patient, phục vụ bác sĩ xem lại trong ERM.
Xem hồ sơ	ucPatientRecords	Lấy lịch sử bệnh nhân, kết quả lab, hình ảnh, file PDF và hiển thị theo timeline/card.

Điểm cần nhấn mạnh trong báo cáo là request không chỉ là dữ liệu hiển thị. Nó là cầu nối giữa bác sĩ và kỹ thuật viên. Bác sĩ tạo chỉ định, request chuyển sang trạng thái chờ xử lý, kỹ thuật viên cập nhật kết quả, sau đó bác sĩ đọc lại kết quả trong ERM. Nếu trạng thái không được chuẩn hóa bằng RequestStatus, giao diện rất dễ hiển thị sai danh sách chờ.

4.2.5 Cài đặt tầng Core, Constants và Session

Core chứa các thành phần dùng chung không phụ thuộc trực tiếp vào giao diện hoặc cơ sở dữ liệu. Đây là tầng nhỏ nhưng rất quan trọng vì nó gom các hằng số trạng thái, vai trò, session và tiện ích dùng lại ở nhiều nơi.

Bảng 4.2.8: Thành phần trong project Core

Nhóm	File tiêu biểu	Vai trò
Constants	AppointmentStatus, PaymentStatus, QueueStatus, RequestStatus, DispenseStatus, ShiftStatus	Chuẩn hóa chuỗi trạng thái dùng giữa UI, BUS, DAL và database.
Identity	Role, RoleNormalizer, Permission, AuthManager	Quản lý vai trò, chuẩn hóa tên role tiếng Việt/tiếng Anh và phân quyền.

Nhóm	File tiêu biểu	Vai trò
Session	UserSession	Lưu thông tin người dùng hiện tại sau đăng nhập: UserID, EmployeeID, EmployeeUUID, DepartmentID, RoleName.
Utils	PasswordHasher, IdGenerator, DateTimeHelper, Mapper, ServiceTypeHelper	Các hàm tiện ích dùng chung.

Việc đưa trạng thái vào constant giúp tránh lỗi gõ chuỗi. Ví dụ nếu nơi này ghi `CheckedIn`, nơi khác ghi `CheckIn`, câu truy vấn trạng thái có thể sai mà compiler không phát hiện. Khi dùng `AppointmentStatus.CheckedIn`, tất cả module dùng chung một giá trị.

```

1 public static string Normalize(string role)
2 {
3     if (string.IsNullOrEmpty(role))
4     {
5         return string.Empty;
6     }
7
8     if (role.Equals("Bác sĩ", StringComparison.OrdinalIgnoreCase) ||
9         role.Equals("Bác si", StringComparison.OrdinalIgnoreCase))
10    {
11        return Role.Doctor;
12    }
13
14    return role.Trim();
15 }
```

Trong báo cáo, `UserSession` cần được mô tả như ngữ cảnh làm việc của ứng dụng Desktop. Sau khi đăng nhập, nhiều màn hình cần biết người dùng hiện tại là bác sĩ nào, thuộc khoa nào, role nào. Nếu mỗi màn hình tự truy vấn lại thông tin đăng nhập, code sẽ lặp và chậm. `UserSession` lưu ngữ cảnh đó ở một nơi thống nhất.

4.3 Thiết kế giao diện người dùng

Phần giao diện của đồ án là trọng tâm môn WinForms Desktop, vì vậy cần trình bày như một hệ thống giao diện có kiến trúc, không chỉ là danh sách màn hình. Project áp dụng mô hình: Mainform làm khung, UserControl làm màn hình con, Controller làm lớp điều phối UI, BUS xử lý nghiệp vụ và DAL truy cập dữ liệu. Cách thiết kế này giúp tránh tình trạng tất cả code nằm trong event click của form.

4.3.1 Vai trò của Controller trong WinForms

Trong WinForms thuần, nhiều project sinh viên thường để form gọi trực tiếp DAL. Project này có thêm nhóm Controllers, ví dụ `PaymentController`. Controller nhận yêu cầu từ user

control, gọi BUS tương ứng và trả DTO về UI. Controller không phải tầng nghiệp vụ đầy đủ như BUS, mà là lớp điều phối phía giao diện.

```
1 public class PaymentController
2 {
3     private readonly PaymentBUS paymentBUS = new();
4
5     public List<PaymentWaitingDTO> GetWaitingPayments()
6     {
7         return paymentBUS.GetWaitingPayments();
8     }
9
10    public bool UpdatePaymentStatus(int encounterId, string method)
11    {
12        return paymentBUS.UpdatePaymentStatus(encounterId, method);
13    }
14
15    public List<PaymentDetailDTO> GetInvoiceDetails(int encounterId)
16    {
17        return paymentBUS.GetInvoiceDetails(encounterId);
18    }
19 }
```

Lợi ích của controller là làm code-behind của user control ngắn hơn. Ví dụ PaymentDetail chỉ cần biết gọi controller.GetInvoiceDetails(encounterId) để bind dữ liệu, hoặc controller.UpdatePaymentMethod) khi xác nhận thanh toán. Nếu sau này thay đổi logic tính tiền ở BUS/DAL, giao diện ít bị ảnh hưởng.

4.3.2 Thiết kế DataGridView trong màn hình thanh toán

PaymentDetail là ví dụ rõ về một màn hình WinForms có cấu hình DataGridView chỉnh chu. Màn hình này thiết lập màu nền, bỏ row header, không cho thêm/xóa dòng trực tiếp, đặt ReadOnly, tự co cột, chọn cả dòng, đổi màu header và font. Đây là các chi tiết quan trọng để giao diện bảng dữ liệu trông chuyên nghiệp và tránh thao tác sai.

```
1 dataGridView1.AllowUserToAddRows = false;
2 dataGridView1.AllowUserToDeleteRows = false;
3 dataGridView1.ReadOnly = true;
4 dataGridView1.AutoSizeColumnsMode =
5     DataGridViewAutoSizeColumnsMode.Fill;
6 dataGridView1.SelectionMode =
7     DataGridViewSelectionMode.FullRowSelect;
8 dataGridView1.EnableHeadersVisualStyles = false;
9 dataGridView1.ColumnHeadersDefaultCellStyle.BackColor =
10     Color.RoyalBlue;
```

Sau khi cấu hình, dữ liệu invoice được lấy từ controller và bind trực tiếp vào grid. Tổng tiền được tính từ danh sách PaymentDetailDTO, sau đó hiển thị ở label lớn. Đây là pattern thường dùng trong WinForms: lấy danh sách DTO, bind grid, tính toán tổng hợp phía UI nếu phép tính chỉ phục vụ hiển thị.

```
1 private void LoadInvoice()
2 {
3     List<PaymentDetailDTO> details =
4         controller.GetInvoiceDetails(encounterId);
5
6     dataGridView1.DataSource = details;
```

```

7
8     decimal total = details.Sum(x => x.Amount);
9     lblTotal.Text = total.ToString("N0") + " đ";
10    lblInvoiceID.Text = encounterId.ToString();
11 }

```

4.3.3 Thiết kế FlowLayoutPanel cho danh sách động

Một số màn hình dùng FlowLayoutPanel thay vì DataGridView. Ví dụ danh sách payment chờ trong PaymentContent được hiển thị dạng card trong flpPayment; danh sách khoa/bác sĩ trong tạo lịch hẹn cũng phù hợp với card. Lý do là người dùng không chỉ đọc dữ liệu dạng bảng mà cần chọn nhanh một đối tượng.

Khi dùng FlowLayoutPanel, quy trình cài đặt gồm:

1. Xóa control cũ bằng `Controls.Clear()` khi reload.
2. Lấy danh sách DTO từ controller/BUS.
3. Với mỗi DTO, tạo card hoặc user control con.
4. Gán dữ liệu vào label/button của card.
5. Gán Tag hoặc field riêng để lưu ID nghiệp vụ.
6. Gắn event click để chọn card hoặc mở chi tiết.
7. Thêm card vào FlowLayoutPanel.

So với DataGridView, card động cho trải nghiệm trực quan hơn nhưng cần quản lý event cẩn thận. Nếu reload nhiều lần mà không xóa control cũ, giao diện sẽ bị trùng card. Nếu không lưu ID vào Tag, event click không biết người dùng chọn bản ghi nào.

4.3.4 Thiết kế UserControl dùng chung và kế thừa view base

Phân hệ kỹ thuật viên và dược sĩ có các base view như `TechnicianDashboardViewBase` hoặc `PharmacyDashboardViewBase`. Ý tưởng là các màn hình con cùng phân hệ có chung cách phát sự kiện điều hướng, chung layout cơ bản hoặc chung helper tạo tiêu đề/nội dung. Thay vì mỗi user control tự gọi main form, view con phát `NavigateRequested`; main form nhận event và quyết định nạp màn hình mới.

Mô hình này có hai lợi ích. Thứ nhất, user control con độc lập hơn, dễ test và dễ tái sử dụng. Thứ hai, main form giữ quyền kiểm soát điều hướng, tránh việc view con biết quá nhiều về cấu trúc dashboard. Đây là cách tiếp cận gần với pattern event aggregator đơn giản trong ứng dụng Desktop.

4.3.5 Thiết kế trạng thái nút và phản hồi người dùng

Ứng dụng Desktop cần phản hồi ngay khi người dùng thao tác. Các nút gọi API hoặc xử lý lâu cần disable trong lúc chạy để tránh bấm nhiều lần. Các thao tác như thanh toán, đồng bộ, upload file cần hiển thị trạng thái hiện tại. Trong PayOS form, khi tạo payment, nút tạo link bị disable, con trỏ đổi sang WaitCursor, label trạng thái đổi thành “Đang tạo link thanh toán...”. Khi API trả về, trạng thái chuyển sang pending/paid/canceled.

```
1 button2.Enabled = false;
2 Cursor = Cursors.WaitCursor;
3 SetPaymentStatus("Đang tạo link thanh toán...", Color.DimGray);
4
5 try
6 {
7     var result = await CreatePaymentAsync(
8         orderCode,
9         amount,
10        description,
11        buyerName,
12        cancellationToken);
13 }
14 finally
15 {
16     Cursor = Cursors.Default;
17     button2.Enabled = true;
18 }
```

4.3.6 Cài đặt main form shell theo vai trò

Các dashboard chính như ReceptionistMainform, DoctorMainform, NurseMainform, TechnicianMainform và PharmacyMainform được xem như shell của từng vai trò. Shell gồm menu trái, vùng tiêu đề và panel nội dung. Khi người dùng bấm menu, shell không mở thêm cửa sổ rời rạc mà thay user control trong panel nội dung. Đây là kỹ thuật WinForms quan trọng vì nó tạo cảm giác ứng dụng có một workspace thống nhất, tương tự các phần mềm Desktop quản lý chuyên nghiệp.

```
1 private void LoadContent(UserControl control)
2 {
3     pnlContent.Controls.Clear();
4     control.Dock = DockStyle.Fill;
5     pnlContent.Controls.Add(control);
6     control.BringToFront();
7 }
```

Mẫu này được dùng lặp lại ở nhiều main form. Lợi ích là mỗi màn hình con được đóng gói thành một UserControl; shell chỉ chịu trách nhiệm điều hướng. Khi cần sửa giao diện thanh toán, lịch hôm nay hoặc danh mục thuốc, nhóm chỉ sửa đúng user control tương ứng mà không phải đụng vào toàn bộ dashboard.

4.3.7 Phân tách Designer và code-behind

Một điểm cần trình bày rõ trong báo cáo WinForms là mỗi form/control thường có hai file: file .Designer.cs và file .cs. File Designer chứa phần khai báo control, vị trí, kích thước,

màu sắc, font, event wiring do Visual Studio sinh ra. File code-behind chứa logic của nhóm phát triển như load dữ liệu, xử lý click, gọi BUS, bind DTO vào bảng hoặc card.

Bảng 4.3.1: Nguyên tắc phân tách Designer và code-behind

Loại file	Nội dung nên đặt	Nội dung không nên đặt
*.Designer.cs	Khởi tạo control, layout, font, màu, kích thước, anchor/dock, khai báo event handler.	Truy vấn SQL, gọi API, xử lý nghiệp vụ, validate phức tạp.
*.cs	Constructor, sự kiện, gọi controller/BUS, bind dữ liệu, format bảng, mở modal.	Sửa thủ công khối InitializeComponent do Designer sinh.
DTO/BUS/DAL	Định nghĩa dữ liệu, nghiệp vụ, truy xuất database.	Thiết lập tọa độ control hoặc thao tác giao diện trực tiếp.

Việc tuân thủ nguyên tắc này giúp tránh lỗi mất giao diện khi Visual Studio tự sinh lại Designer. Đối với đồ án có nhiều form như hệ thống quản lý phòng khám, đây là yếu tố quan trọng để các thành viên có thể làm song song mà không xung đột quá nhiều.

4.3.8 Giao diện đăng nhập

LoginForm tiếp nhận username và password, kiểm tra dữ liệu rỗng, gọi UserBUS.Login. Form cũng có cơ chế thử khởi tạo database khi phát hiện thiếu schema CMS.sql. Về lâu dài, phần khởi tạo này nên tách thành service riêng để giảm trách nhiệm cho form đăng nhập.

Về mặt thiết kế Desktop, màn hình đăng nhập là điểm kiểm soát đầu tiên của toàn bộ ứng dụng. Form cần có hai ô nhập chính, nút đăng nhập, thông báo lỗi rõ ràng và không để người dùng đi vào dashboard khi chưa xác thực. Sau khi đăng nhập thành công, form không tự quyết định hiển thị chức năng nào bằng cách bật/tắt control rời rạc, mà chuyển quyền điều hướng cho MainFormRole. Cách làm này giữ LoginForm đúng trách nhiệm: xác thực và tạo phiên làm việc.

```

1 private bool ValidateLoginInput()
2 {
3     if (string.IsNullOrWhiteSpace(txtUsername.Text))
4     {
5         MessageBox.Show("Vui lòng nhập tên đăng nhập");
6         txtUsername.Focus();
7         return false;
8     }
9
10    if (string.IsNullOrWhiteSpace(txtPassword.Text))
11    {
12        MessageBox.Show("Vui lòng nhập mật khẩu");
13        txtPassword.Focus();
14        return false;
15    }
16
17    return true;
18 }

```

4.3.9 Giao diện lễ tân

ReceptionistMainform nạp các màn hình:

- PatientManagement: quản lý bệnh nhân.
- CreateAppointment: tạo lịch hẹn.
- Payment: thanh toán.
- ScheduleToday: lịch trong ngày.
- WaitingList: danh sách chờ.
- RemindingList: nhắc lịch.

Dashboard lễ tân được thiết kế theo kiểu thao tác nhanh. Đây là vai trò có tần suất sử dụng cao nhất ở đầu quy trình khám bệnh, vì vậy menu cần đưa các chức năng quan trọng ra ngoài thay vì ẩn sâu trong nhiều cấp. Vùng nội dung chính thay đổi theo chức năng được chọn. Khi người dùng vào PatientManagement, hệ thống ưu tiên ô tìm kiếm và bảng danh sách bệnh nhân; khi vào ScheduleToday, hệ thống ưu tiên trạng thái lịch trong ngày; khi vào Payment, hệ thống ưu tiên danh sách chờ thanh toán và nút xác nhận.

Bảng 4.3.2: Thiết kế giao diện phân hệ lễ tân

Màn hình	Control chính	Mục tiêu thao tác
PatientManagement	TextBox tìm kiếm, DataGridView, Button thêm/sửa	Tra cứu nhanh bệnh nhân và cập nhật hồ sơ hành chính.
CreateAppointment	FlowLayoutPanel khoa/bác sĩ, Button slot giờ, Dialog xác nhận	Tạo lịch hẹn theo khoa, bác sĩ và thời gian khám.
ScheduleToday	DataGridView lịch hẹn, Button check-in	Theo dõi lịch trong ngày và chuyển bệnh nhân vào hàng chờ.
WaitingList	FlowLayoutPanel lọc khoa/bác sĩ, DataGridView hàng chờ	Biết bệnh nhân đang ở trạng thái nào trong quy trình khám.
Payment	DataGridView hóa đơn chờ, Button thanh toán, PayOS form	Tạo hoặc xác nhận thanh toán sau khi khám/cấp thuốc.

Kỹ thuật WinForms nổi bật ở phân hệ này là tạo card động trong FlowLayoutPanel. Khi chọn khoa, danh sách bác sĩ được tải lại theo khoa đó. Khi chọn bác sĩ, các khung giờ khả dụng được cập nhật. Đây là luồng tương tác nhiều bước, đòi hỏi form lưu trạng thái lựa chọn hiện tại và reset các bước sau khi bước trước thay đổi.

```

1 private int selectedDepartmentId;
2 private int selectedDoctorId;
3 private DateTime selectedDate;
4 private TimeSpan selectedStartTime;
5
6 private void Department_Click(object sender, EventArgs e)
7 {
8     selectedDepartmentId = (int)((Control)sender).Tag;
9     selectedDoctorId = 0;
10    flpDoctors.Controls.Clear();
11    flpTimeSlots.Controls.Clear();
12    LoadDoctorsByDepartment(selectedDepartmentId);
13 }
14
15 private void TimeSlot_Click(object sender, EventArgs e)
16 {
17     selectedStartTime = (TimeSpan)((Control)sender).Tag;
18     btnConfirmAppointment.Enabled = selectedDoctorId > 0;
19 }

```

Trong ScheduleToday, DataGridViewButtonColumn được dùng để gắn hành động check-in trực tiếp vào từng dòng. Đây là cách cài đặt phù hợp với Desktop vì lẽ tần không phải mở chi tiết nếu chỉ cần xác nhận bệnh nhân đã đến. Event CellContentClick đọc mã lịch hẹn ở dòng được bấm, gọi BUS cập nhật trạng thái và reload lại bảng.

4.3.10 Giao diện bác sĩ và ERM

Các thành phần ERM trong Shareforms/ERM gồm ucOverview, ucHistory, ucLab, ucImaging, ucPrescription, ucBilling, ucFollowup và frmPacsViewer. ERMController gọi ERMBus để lấy hồ sơ bệnh nhân theo UUID.

Giao diện bác sĩ tập trung vào hồ sơ y tế thay vì thông tin hành chính. Khi bác sĩ mở một bệnh nhân, ERM cần cung cấp cái nhìn tổng quan: thông tin bệnh nhân, lịch sử khám, sinh hiệu, chỉ định xét nghiệm, hình ảnh, toa thuốc, thanh toán và tái khám. Vì dữ liệu y tế có nhiều nhóm thông tin, cách chia thành các user control con giúp màn hình không quá dài và dễ điều hướng.

ERM cũng là ví dụ rõ nhất về việc dùng controller để giảm code-behind. ERMController chịu trách nhiệm gọi ERMBus, lấy dữ liệu theo patient UUID/encounter, sau đó trả về DTO cho các tab giao diện. Các user control như ucLab hoặc ucPrescription chỉ tập trung hiển thị dữ liệu và phát sự kiện khi người dùng cần thao tác thêm.

```

1 private void LoadMedicalRecord(string patientUuid)
2 {
3     var overview = ermController.GetOverview(patientUuid);
4     ucOverview.BindData(overview);
5
6     var history = ermController.GetHistory(patientUuid);
7     ucHistory.BindData(history);
8
9     var prescriptions = ermController.GetPrescriptions(patientUuid);
10    ucPrescription.BindData(prescriptions);
11 }

```

Trong phần ERM, cần trình bày rõ lý do chia thành nhiều user control nhỏ. ucOverview hiển thị thông tin tổng quan và sinh hiệu; ucHistory hiển thị các lần khám trước; ucLab và ucImaging hiển thị kết quả cận lâm sàng; ucPrescription hiển thị toa thuốc; ucBilling

hiển thị chi phí; ucFollowup hiển thị lịch tái khám. Cách chia này giúp bác sĩ đọc hồ sơ theo ngữ cảnh thay vì phải nhìn một form dài gồm quá nhiều bảng.

Bảng 4.3.3: Cấu trúc màn hình ERM của bác sĩ

UserControl	Dữ liệu hiển thị	Mục đích sử dụng
ucOverview	Thông tin bệnh nhân, sinh hiệu, chẩn đoán hiện tại.	Bác sĩ nắm nhanh tình trạng trước khi khám.
ucHistory	Lịch sử encounter và ghi chú các lần khám.	So sánh diễn tiến bệnh qua thời gian.
ucLab	Danh sách xét nghiệm và kết quả lab.	Đọc kết quả cận lâm sàng trước khi kết luận.
ucImaging	Hình ảnh MRI/X-Ray, mô tả và file liên quan.	Xem hình ảnh hoặc mở viewer phục vụ chẩn đoán.
ucPrescription	Toa thuốc, thuốc, liều dùng, số lượng.	Theo dõi thuốc đã kê và hỗ trợ được sĩ cấp phát.
ucFollowup	Lịch tái khám, ghi chú nhắc lịch.	Tạo kế hoạch chăm sóc sau khám.

4.3.11 Giao diện điều dưỡng

Nhóm UserControls/Nurse có các màn hình tổng quan, ca làm, dấu hiệu sinh tồn, hàng chờ và phòng. Điều dưỡng hỗ trợ thu thập thông tin trước khi bác sĩ khám, đặc biệt là sinh hiệu và điều phối phòng khám.

Giao diện điều dưỡng cần tối ưu cho nhập liệu nhanh và chính xác. Các chỉ số sinh hiệu như mạch, huyết áp, nhiệt độ, nhịp thở, chiều cao, cân nặng phải có nhãn rõ ràng và đơn vị đi kèm. Những trường có giá trị số nên kiểm tra định dạng trước khi lưu. Nếu nhập sai kiểu dữ liệu, hệ thống phải báo lỗi ngay trên form thay vì để lỗi phát sinh ở tầng DAL.

Trong WinForms, kiểm tra dữ liệu có thể thực hiện tại event Validating, tại nút lưu hoặc thông qua controller. Với đồ án, cách thực dụng là kiểm tra trong nút lưu, sau đó chỉ gọi BUS khi dữ liệu đã hợp lệ.

```

1 private bool TryReadTemperature(out decimal temperature)
2 {
3     if (!decimal.TryParse(txtTemperature.Text.Trim(), out temperature))
4     {
5         MessageBox.Show("Nhiệt độ phải là số");
6         txtTemperature.Focus();
7         return false;
8     }
9
10    return true;
11 }

```

Màn hình điều dưỡng còn sử dụng các card phòng và card bệnh nhân để theo dõi trạng thái nhanh. Với dữ liệu hàng chờ, FlowLayoutPanel phù hợp hơn bảng nếu mỗi bệnh nhân cần

hiển thị nhiều thông tin ngắn như số thứ tự, tên, phòng, trạng thái, thời gian chờ và nút chuyển trạng thái. Khi cần nhập sinh hiệu, hệ thống mở màn hình chi tiết hoặc user control nhập liệu để tránh làm bảng hàng chờ bị quá tải.

4.3.12 Giao diện kỹ thuật viên

TechnicianMainform điều hướng đến ucTechnicianRequests, ucTechnicianUploadMri, ucTechnicianUploadPdf, ucTechnicianLabResult, ucPatientRecords, ucTechnicianShifts và ucSeederTool. Các màn hình này kế thừa hoặc sử dụng TechnicianDashboardViewBase để phát sự kiện điều hướng.

Phân hệ kỹ thuật viên thể hiện rõ mô hình điều hướng nội bộ bằng event. Từ màn hình request, kỹ thuật viên có thể chuyển sang upload MRI, upload PDF, nhập kết quả lab hoặc xem hồ sơ bệnh nhân. Mỗi hướng đi cần mang theo requestId hoặc patientId. Thay vì để màn hình request tự tạo form hoặc tự thao tác trực tiếp lên panel của main form, nó phát NavigateRequested; main form nhận event và nạp đúng user control.

```
1 private void ContentView_NavigateRequested(  
2     object sender,  
3     TechnicianNavigationEventArgs e)  
4 {  
5     switch (e.Target)  
6     {  
7         case TechnicianViewTarget.Requests:  
8             ShowRequests();  
9             break;  
10        case TechnicianViewTarget.UploadMRI:  
11            ShowUploadMRI(e.RequestId);  
12            break;  
13        case TechnicianViewTarget.UploadPDF:  
14            ShowUploadPDF(e.RequestId);  
15            break;  
16        case TechnicianViewTarget.LabResult:  
17            ShowLabResult(e.RequestId);  
18            break;  
19        case TechnicianViewTarget.Records:  
20            ShowRecords(e.PatientId);  
21            break;  
22    }  
23 }
```

Các màn hình kỹ thuật viên còn có điểm cài đặt quan trọng là upload file. Khi dùng OpenFileDialog, hệ thống cần giới hạn định dạng theo nghiệp vụ: PDF cho hồ sơ scan, ảnh cho MRI/X-Ray. Sau khi chọn file, form chỉ lưu đường dẫn hoặc metadata qua BUS/DAL, còn việc hiển thị lại file cho bác sĩ được thực hiện trong ERM. Điều này giúp tách trách nhiệm giữa bên tạo kết quả và bên đọc kết quả.

```
1 using (var dialog = new OpenFileDialog())  
2 {  
3     dialog.Title = "Chọn tệp kết quả";  
4     dialog.Filter = "PDF files (*.pdf)|*.pdf|Image files (*.png;*.jpg)|*.png;*.jpg";  
5  
6     if (dialog.ShowDialog() == DialogResult.OK)  
7     {  
8         txtFilePath.Text = dialog.FileName;  
9         btnUpload.Enabled = true;  
10    }  
11 }
```


4.3.13 Giao diện dược sĩ

PharmacyMainform điều hướng đến ucPharmacyOverview, ucPrescriptionDispense, ucMedicineCatalog, ucInventoryManagement và ucPharmacyShifts. Các view kế thừa PharmacyDashboardViewBase, giúp thống nhất sự kiện điều hướng.

Giao diện dược sĩ có hai kiểu dữ liệu chính: đơn thuốc cần cấp phát và danh mục/tồn kho thuốc. Đơn thuốc phù hợp với DataGridView vì cần lọc theo trạng thái và mở chi tiết theo từng dòng. Danh mục thuốc lại phù hợp với card hoặc grid có hình thức dễ quét, vì dược sĩ cần nhận biết nhanh tên thuốc, hàm lượng, đơn vị tính và tồn kho.

ucPharmacyMedicineItemCard là một ví dụ về custom user control trong WinForms. Control này giúp đóng gói cách hiển thị một thuốc, tránh việc màn hình danh mục phải tự tạo nhiều label/button lặp lại. Khi danh sách thuốc thay đổi, màn hình chỉ cần tạo nhiều card và đưa vào panel chứa.

```
1 public void BindMedicine(MedicineDTO medicine)
2 {
3     lblMedicineName.Text = medicine.MedicineName;
4     lblDosage.Text = medicine.Dosage;
5     lblUnit.Text = medicine.Unit;
6     lblStock.Text = medicine.StockQuantity.ToString();
7
8     btnEdit.Tag = medicine.MedicineID;
9     btnEdit.Click += btnEdit_Click;
10 }
```

4.3.14 Cài đặt lịch làm việc dùng chung

Ngoài các phân hệ riêng, hệ thống có nhóm màn hình ca làm trong Shareforms/WorkingShifts. Các control như ucWeekView, ucDayView, ucMonthView, ShiftCardControl, ShiftDetailForm và RegisterShiftForm được dùng để mô phỏng lịch làm việc của nhân viên. Đây là phần nên trình bày trong chương cài đặt vì nó thể hiện kỹ thuật tái sử dụng user control giữa nhiều vai trò.

Bảng 4.3.4: Cài đặt nhóm màn hình ca làm việc

Thành phần	Kiểu WinForms	Vai trò trong hệ thống
ucWeekView	UserControl + FlowLayoutPanel	Hiển thị ca làm theo từng ngày trong tuần.
ucDayView	UserControl	Hiển thị danh sách ca trong một ngày cụ thể.
ShiftCardControl	Custom UserControl	Đóng gói thông tin một ca làm: thời gian, trạng thái, người đăng ký.
RegisterShiftForm	Modal Form	Cho nhân viên đăng ký ca hoặc gửi yêu cầu thay đổi ca.

Thành phần	Kiểu WinForms	Vai trò trong hệ thống
ShiftDetailForm	Modal Form	Xem chi tiết một ca làm và trạng thái xử lý.

Về kỹ thuật, ucWeekView dùng nhiều FlowLayoutPanel đại diện cho các ngày trong tuần. Khi dữ liệu ca làm được tải lên, mỗi ca được bind thành một ShiftCardControl rồi thêm vào panel của ngày tương ứng. Cách làm này trực quan hơn DataGridView đối với lịch làm việc vì ca làm có tính thời gian và trạng thái, không chỉ là bảng dòng-cột.

4.4 Tích hợp tính năng phụ trợ

4.4.1 Đồng bộ API Supabase và SheetDB

Phân hệ ApiSyncBUS và ApiSyncRepository hỗ trợ đồng bộ bệnh nhân, nhân viên, yêu cầu và kết quả cận lâm sàng từ nguồn ngoài. Repository có các phương thức đọc dữ liệu dạng JSON, xử lý UUID, cập nhật local database và trả về ApiSyncResultDTO.

Ở mức cài đặt, ApiSyncRepository đọc endpoint từ App.config. Với Supabase, repository gọi REST endpoint dạng /rest/v1/patients?select=* hoặc /rest/v1/employees?select=* và thêm header xác thực. Với SheetDB, repository gọi URL của SheetDB đã được ghép tên sheet bằng hàm WithSheetName. Cả hai nguồn đều deserialize JSON về DTO bằng System.Text.Json với PropertyNameCaseInsensitive = true và NumberHandling = AllowReadingFromString, giúp đọc được số khi API trả về dạng chuỗi.

```

1 supabaseUrl = ReadSetting("ApiSyncSupabaseUrl", "").TrimEnd('/');
2 supabaseKey = ReadSetting("ApiSyncSupabaseKey", "");
3 sheetDbPatientUrl = WithSheetName(
4     ReadSetting("ApiSyncSheetDbPatientUrl", ""),
5     ReadSetting("ApiSyncSheetDbPatientSheet", "Patient"));
6 sheetDbEmployeeUrl = WithSheetName(
7     ReadSetting("ApiSyncSheetDbEmployeeUrl", ""),
8     ReadSetting("ApiSyncSheetDbEmployeeSheet", "Doctor"));

```

Khi ghi về SQL Server local, repository không insert mù. Nó kiểm tra theo UUID trước, nếu không có UUID thì kiểm tra theo mã bệnh nhân hoặc mã nhân viên. Nếu bản ghi đã tồn tại thì update; nếu chưa tồn tại thì insert. Nếu dữ liệu ngoài thiếu UUID, local sinh Guid.NewGuid() rồi trả kết quả cho BUS để patch ngược về SheetDB. Đây là logic chống trùng dữ liệu quan trọng.

Bảng 4.4.1: Các bước xử lý dữ liệu API ở tầng DAL/BUS

Bước	Lớp thực hiện	Mô tả
Đọc API	ApiSyncRepository	Gọi Supabase/SheetDB, deserialize JSON về DTO.

Bước	Lớp thực hiện	Mô tả
Lọc dữ liệu	ApiSyncBUS	Bỏ dòng thiếu mã/tên, bỏ nhân viên bị nhầm với mã bệnh nhân.
Bù dữ liệu	ApiSyncBUS	Bổ sung khoa, role, trạng thái mặc định cho nhân viên.
Liên kết UUID	ApiSyncBUS + Repository	Dùng mã để tìm bản ghi cũ, patch UUID sang nguồn còn thiếu.
Upsert local	ApiSyncRepository	Insert/update SQL Server theo UUID hoặc code.
Đồng bộ hai chiều	ApiSyncBUS	SheetDB sang Supabase và Supabase sang SheetDB.
Thông báo kết quả	ApiSyncResultDTO	Tổng hợp số dòng insert/patch/bỏ qua để form hiển thị.

```

1 public async Task<ApiSyncResultDTO> SyncAsync()
2 {
3     var result = new ApiSyncResultDTO();
4
5     var supabasePatientsRaw = await repository.GetSupabasePatientsAsync();
6     var sheetPatientsRaw = await repository.GetSheetPatientsAsync();
7
8     var supabasePatients = supabasePatientsRaw
9         .Where(p => !string.IsNullOrEmpty(p.PatientCode))
10        .Where(p => !string.IsNullOrEmpty(p.FullName))
11        .ToList();
12
13     var sheetPatients = sheetPatientsRaw
14        .Where(p => !string.IsNullOrEmpty(p.PatientCode))
15        .Where(p => !string.IsNullOrEmpty(p.FullName))
16        .ToList();
17
18     await LinkMissingSheetPatientUidsFromSupabaseAsync(
19         sheetPatients,
20         supabasePatients,
21         result);
22
23     await SyncPatientsToLocalSqlServerAsync(
24         sheetPatients,
25         result,
26         patchSheetWhenMissingUuid: true);
27
28     await SyncPatientsToLocalSqlServerAsync(
29         supabasePatients,
30         result,
31         patchSheetWhenMissingUuid: false);
32
33     await SyncPatientsSheetToSupabaseAsync(
34         sheetPatients,
35         supabasePatients,
36         result);
37
38     await SyncPatientsSupabaseToSheetAsync(
39         supabasePatients,
40         sheetPatients,
41         result);
42
43     return result;
44 }

```

4.4.2 Kéo request xét nghiệm và hình ảnh về màn hình kỹ thuật viên

Phân hệ kỹ thuật viên có thêm thao tác đồng bộ request từ Supabase. Khác với đồng bộ bệnh nhân/nhân viên, request cần lâm sàng cần ghép nhiều bảng: encounter để biết bệnh nhân, employee để biết bác sĩ, services để biết loại dịch vụ, results để lấy kết quả/URL file. ApiSyncBUS.SyncRequestsFromSupabaseAsync() tạo các dictionary tra cứu để xử lý nhanh, sau đó gọi UpsertLocalRequestAsync.

```
1 private string MapRequestStatus(string status)
2 {
3     if (string.IsNullOrEmpty(status))
4     {
5         return "Chờ xử lý";
6     }
7
8     string s = status.Trim().ToLowerInvariant();
9     if (s.Contains("completed") || s.Contains("done"))
10    {
11        return "Hoàn thành";
12    }
13
14    if (s.Contains("processing") || s.Contains("inprogress"))
15    {
16        return "Đang xử lý";
17    }
18
19    return "Chờ xử lý";
20 }
```

Sau khi request được kéo về local, màn hình ucTechnicianRequests hiển thị cho kỹ thuật viên. Từ đây kỹ thuật viên có thể điều hướng sang upload MRI, upload PDF, nhập lab result hoặc xem hồ sơ bệnh nhân. Như vậy, tích hợp API không đứng riêng mà được gắn vào workflow WinForms thật.

4.4.3 Tích hợp thanh toán PayOS

Các lớp PayOsCreatePaymentRequest, PayOsPaymentData và PayOsApiResponse mô tả dữ liệu tạo thanh toán, dữ liệu phản hồi và trạng thái thanh toán. PayOsPaymentForm trong Shareforms phục vụ giao diện thanh toán.

Ở mức cài đặt, PaymentDetail cho phép người dùng chọn tiền mặt hoặc chuyển khoản. Nếu chọn chuyển khoản, form mở PayOsPaymentForm. Form này tạo payment link, render QR và kiểm tra trạng thái. Sau khi người dùng xác nhận, PaymentController.UpdatePaymentStatus gọi PaymentBUS.UpdatePaymentStatus để cập nhật trạng thái thanh toán trong SQL Server.

```
1 private void btnQR_Click(object sender, EventArgs e)
2 {
3     paymentMethod = "Chuyển khoản";
4     lblPaymentMethod.Text =
5         "Phương thức thanh toán: Chuyển khoản";
6
7     PayOsPaymentForm frm = new();
8     frm.ShowDialog();
9 }
10
11 private void button2_Click(object sender, EventArgs e)
12 {
13     bool result = controller.UpdatePaymentStatus(
```

```

14     encounterId,
15     paymentMethod);
16 }

```

PayOsPaymentForm có ba tầng xử lý. Tầng thứ nhất là UI: nhập số tiền, nội dung, người mua, hiển thị QR và trạng thái. Tầng thứ hai là API client: gọi backend hoặc PayOS bằng Rest-Sharp. Tầng thứ ba là bảo mật/chữ ký: nếu fallback trực tiếp, form tạo chữ ký HMACSHA256 từ amount, cancelUrl, description, orderCode, returnUrl và checksumKey. Khi triển khai thật, phần tạo chữ ký nên nằm ở backend để không lộ khóa trong ứng dụng Desktop.

Bảng 4.4.2: Dữ liệu phản hồi PayOS dùng trong WinForms

Trường	DTO	Cách sử dụng
orderCode	PayOsPaymentData	Mã đơn hàng dùng kiểm tra trạng thái.
checkoutUrl	PayOsPaymentData	Link thanh toán, có thể hiển thị hoặc tạo QR.
qrCode	PayOsPaymentData	Dữ liệu QR dạng chuỗi, render bằng dịch vụ tạo QR.
qrDataURL	PayOsPaymentData	Ảnh QR base64, chuyển thành Image bằng ImageHelper.
status	PayOsPaymentData	Trạng thái PENDING/PAID/CANCELED để cập nhật label và dừng timer.
amountPaid	PayOsPaymentData	Số tiền đã thanh toán, phục vụ kiểm tra nếu cần.

Với webhook, backend là thành phần nhận callback từ PayOS. Desktop không trực tiếp nhận webhook vì ứng dụng chạy trong mạng nội bộ, không có public endpoint ổn định. Do đó, flow hợp lý là PayOS gọi backend; backend xác thực chữ ký webhook và cập nhật payment; Desktop kiểm tra trạng thái qua backend. Hiện tại form dùng System.Windows.Forms.Timer polling 5 giây/lần. Nếu phát triển realtime đúng nghĩa, backend có thể phát SignalR event để WinForms nhận trạng thái ngay khi webhook đến.

Bảng 4.4.3: Luồng triển khai PayOS trong ứng dụng Desktop

Bước	Thành phần	Mô tả cài đặt
Tạo hóa đơn	PaymentDetail, PaymentController	Lấy chi tiết phí khám, dịch vụ, thuốc; tính tổng tiền cần thanh toán.

Bước	Thành phần	Mô tả cài đặt
Tạo payment link	PayOsPaymentForm	Gửi orderCode, amount, description, buyerName lên backend.
Render QR	PayOsPaymentForm	Ưu tiên qrDataURL; nếu không có thì dùng qrCode hoặc checkoutUrl để tạo ảnh QR.
Webhook	Backend trung gian	Nhận callback từ PayOS, xác thực chữ ký và cập nhật trạng thái giao dịch.
Polling trạng thái	System.Windows.Forms.Timer	Desktop gọi API kiểm tra trạng thái 5 giây/lần để cập nhật UI.
Cập nhật local	PaymentBUS, PaymentDAL	Khi trạng thái thành công, cập nhật hóa đơn và payment trong SQL Server.

4.5 Tổng hợp luồng cài đặt theo nghiệp vụ

Để chương xây dựng không bị rời rạc, bảng sau tổng hợp các luồng chính từ giao diện đến cơ sở dữ liệu. Đây là cách nhìn theo chiều dọc của hệ thống: người dùng thao tác trên WinForms, form/user control gọi controller hoặc BUS, BUS xử lý nghiệp vụ, DAL truy cập SQL Server/API, sau đó DTO trả về để giao diện cập nhật.

Bảng 4.5.1: Trace cài đặt end-to-end của các nghiệp vụ chính

Nghiệp vụ	UI WinForms	BUS/DAL chính	Kết quả dữ liệu
Đăng nhập	LoginForm, MainformRole	UserBUS, UserDAL, RoleNormalizer, UserSession	Session được tạo, dashboard đúng vai trò được mở.
Quản lý bệnh nhân	PatientManagement, CreateNewPatient	PatientBUS, PatientDAL	Bệnh nhân được thêm/sửa/tìm kiếm trong SQL Server.
Tạo lịch khám	CreateAppointment, ConfirmAppointment	AppointmentBUS, DoctorScheduleBUS, AppointmentDAL	Lịch hẹn có bác sĩ, phòng, ngày giờ và trạng thái ban đầu.
Check-in	ScheduleToday	AppointmentBUS, QueueBUS	Lịch hẹn chuyển trạng thái, hàng chờ được tạo.

Nghệp vụ	UI WinForms	BUS/DAL chính	Kết quả dữ liệu
Khám bệnh	DoctorMainform, ERM, các ucERM	ERMBus, EncounterDAL, PrescriptionDAL	Encounter, chẩn đoán, chỉ định, toa thuốc được lưu.
Nhập sinh hiệu	ucNurseVitalSigns	VitalSignBUS, VitalSignDAL	Sinh hiệu được gắn với lượt khám/bệnh nhân.
Xử lý request	ucTechnicianRequest, ucTechnicianLabResult, ucTechnicianUploadMri	TechnicianRequestBUS, TechnicianRequestDAL	Kết quả lab/file hình ảnh được cập nhật, re- quest đổi trạng thái.
Cấp phát thuốc	ucPrescriptionDispense, ucPharmacyMedicineInventory	PharmacyBUS, MedicineDAL, InventoryDAL	Đơn thuốc được cấp phát, tồn kho cập nhật.
Thanh toán	Payment, PaymentDetail, PayOsPaymentForm	PaymentController, PaymentBUS, PaymentDAL	Hóa đơn và lịch sử thanh toán được cập nhật.
Đồng bộ API	MockApiSyncForm, ucSeederTool	ApiSyncBUS, ApiSyncRepository	Dữ liệu Su- pabase/SheetDB được hợp nhất vào SQL Server local.

4.6 Phân rã chi tiết các luồng vận hành

Phần này mô tả sâu hơn cách các phân hệ chạy trong chương trình. Mục tiêu không chỉ liệt kê màn hình, mà làm rõ tại mỗi bước hệ thống nhận dữ liệu gì, kiểm tra gì, gọi lớp nào, cập nhật trạng thái nào và phản hồi lại giao diện ra sao. Đây là phần quan trọng đối với đồ án WinForms Desktop vì phần lớn lỗi thực tế không nằm ở việc tạo được form, mà nằm ở luồng form gọi nghiệp vụ không rõ, trạng thái dữ liệu không nhất quán hoặc giao diện không reload sau khi dữ liệu thay đổi.

4.6.1 Luồng đăng nhập và mở dashboard

Luồng đăng nhập được thiết kế như cổng vào duy nhất của hệ thống. Từ Program.Main, ứng dụng mở LoginForm; người dùng chưa đăng nhập không thể đi thẳng vào dashboard. Sau khi nhập username/password, form kiểm tra dữ liệu xong, gọi UserBUS, nhận kết quả xác thực, tạo UserSession rồi chuyển sang MainformRole. MainformRole không hiển thị nghiệp vụ trực tiếp mà tạo dashboard đúng vai trò và nhúng vào form cha.

Bảng 4.6.1: Diễn giải chi tiết luồng đăng nhập

Bước	Thành phần	Dữ liệu vào/ra	Ý nghĩa cài đặt
1	Program.Main	Không có dữ liệu nghiệp vụ	Cấu hình WinForms, bật visual style và chạy LoginForm.
2	LoginForm	Username, password	Kiểm tra rỗng để tránh gọi BUS/DAL khi dữ liệu chắc chắn không hợp lệ.
3	UserBUS	Username/password đã trim	Kiểm tra tài khoản, mật khẩu, trạng thái user và role.
4	UserDAL	Username	Truy vấn SQL Server để lấy user, employee, role liên quan.
5	RoleNormalizer	Chuỗi role từ database	Chuẩn hóa role để tránh lệch do tiếng Việt có dấu, không dấu hoặc tên tiếng Anh.
6	UserSession	UserDTO, role chuẩn hóa	Lưu ngữ cảnh dùng chung cho các dashboard và user control.
7	MainformRole	Role hiện tại	Tạo AdminMainform, DoctorMainform, ReceptionistMainform, NurseMainform, TechnicianMainform hoặc PharmacyMainform.

```

1 private Form CreateDashboardForCurrentUser()
2 {
3     string role = RoleNormalizer.Normalize(UserSession.RoleName);
4
5     switch (role)
6     {
7         case Role.Admin:
8             return new AdminMainform();
9         case Role.Doctor:
10            return new DoctorMainform();
11         case Role.Nurse:
12            return new NurseMainform();
13         case Role.Receptionist:
14            return new ReceptionistMainform();
15         case Role.Technician:
16            return new TechnicianMainform();
17         case Role.Pharmacy:
18            return new PharmacyMainform();
19         default:
20            throw new InvalidOperationException("Role không hợp lệ");
21     }
22 }

```

Nhận xét cài đặt: cách dùng MainformRole giúp tách đăng nhập khỏi điều hướng. Nếu sau này thêm vai trò thu ngân hoặc quản lý kho, nhóm chỉ cần bổ sung role và dashboard mới ở

một điểm trung tâm. Nếu để LoginForm tự mở từng form theo nhiều nhánh, form đăng nhập sẽ nhanh chóng phình to và khó kiểm soát.

4.6.2 Luồng tạo lịch hẹn

Tạo lịch hẹn là luồng có nhiều trạng thái tạm trên giao diện. Lễ tân phải chọn bệnh nhân, chọn khoa, chọn bác sĩ, chọn ngày, chọn slot giờ rồi xác nhận. Nếu bất kỳ bước nào thay đổi, các bước phía sau phải reset. Ví dụ đổi khoa thì danh sách bác sĩ cũ không còn đúng; đổi bác sĩ thì danh sách slot giờ cũ phải tải lại.

Bảng 4.6.2: Trạng thái giao diện trong luồng tạo lịch hẹn

Trạng thái	Control liên quan	Dữ liệu được giữ	Hành động kế tiếp
Chưa chọn bệnh nhân	Ô tìm kiếm, bảng bệnh nhân	Chưa có PatientID	Người dùng chọn hoặc tạo bệnh nhân mới.
Đã chọn bệnh nhân	Label thông tin bệnh nhân	PatientID, PatientCode, tên bệnh nhân	Cho phép chọn khoa khám.
Đã chọn khoa	Card khoa trong FlowLayoutPanel	DepartmentID	Tải danh sách bác sĩ thuộc khoa.
Đã chọn bác sĩ	Card bác sĩ	DoctorID, EmployeeUUID	Tải lịch làm việc và slot còn trống.
Đã chọn slot	Button giờ khám	Ngày, giờ bắt đầu, giờ kết thúc	Cho phép mở form xác nhận.
Đã xác nhận	ConfirmAppointment	AppointmentDTO	Gọi BUS lưu lịch hẹn xuống SQL Server.

```

1 private AppointmentDTO BuildAppointmentDto()
2 {
3     return new AppointmentDTO
4     {
5         PatientID = selectedPatientId,
6         DoctorID = selectedDoctorId,
7         DepartmentID = selectedDepartmentId,
8         AppointmentDate = selectedDate.Date,
9         StartTime = selectedStartTime,
10        Status = AppointmentStatus.Pending,
11        CreatedBy = UserSession.UserID
12    };
13 }
14
15 private void btnConfirmAppointment_Click(object sender, EventArgs e)
16 {
17     AppointmentDTO dto = BuildAppointmentDto();
18
19     if (!appointmentBUS.CreateAppointment(dto))
20     {
21         MessageBox.Show("Khong the tao lich hen");
22         return;
23     }

```

```

24
25     MessageBox.Show("Tạo lịch hẹn thành công");
26     ReloadTodayAppointments();
27 }

```

Nhận xét cài đặt: màn hình tạo lịch không nên tự viết SQL insert vào bảng Appointments. Form chỉ nên gom dữ liệu người dùng đã chọn thành AppointmentDTO. Việc kiểm tra bác sĩ có ca làm hay không, slot có bị trùng hay không, phòng khám có hợp lệ hay không phải đặt ở BUS/DAL. Cách này giúp logic tạo lịch không bị phụ thuộc vào layout của form.

4.6.3 Luồng check-in và đưa bệnh nhân vào hàng chờ

Khi bệnh nhân đến phòng khám, lễ tân mở ScheduleToday, tìm lịch hẹn trong ngày và bấm nút check-in. Đây là một thao tác nhỏ trên giao diện nhưng có tác động dữ liệu kép: lịch hẹn đổi trạng thái và hàng chờ được tạo. Nếu chỉ cập nhật lịch hẹn mà không tạo hàng chờ, bác sĩ sẽ không thấy bệnh nhân trong danh sách khám. Nếu tạo hàng chờ nhưng không đổi trạng thái lịch, lễ tân có thể check-in trùng.

Bảng 4.6.3: Điều kiện và kết quả của thao tác check-in

Nhóm	Điều kiện kiểm tra	Kết quả mong đợi
Lịch hẹn	Lịch tồn tại, thuộc ngày hiện tại, chưa hủy, chưa check-in.	Cho phép chuyển sang trạng thái đã đến.
Bệnh nhân	Có PatientID hợp lệ trong lịch hẹn.	Hàng chờ gắn đúng bệnh nhân.
Bác sĩ/phòng	Lịch có bác sĩ và phòng khám.	Hàng chờ gắn đúng phòng để điều phối.
Trạng thái hàng chờ	Chưa tồn tại queue active cho cùng appointment.	Tạo queue mới với trạng thái chờ khám.
Giao diện	Dòng vừa check-in không còn nút thao tác cũ.	Reload DataGridView và cập nhật màu/trạng thái.

```

1 public bool CheckInAppointment(int appointmentId)
2 {
3     AppointmentDTO appointment = appointmentDAL.GetById(appointmentId);
4     if (appointment == null ||
5         appointment.Status != AppointmentStatus.Confirmed)
6     {
7         return false;
8     }
9
10    bool statusUpdated = appointmentDAL.UpdateStatus(
11        appointmentId,
12        AppointmentStatus.CheckedIn);
13
14    if (!statusUpdated)
15    {
16        return false;
17    }

```

```

18
19 QueueDTO queue = new QueueDTO
20 {
21     AppointmentID = appointmentId,
22     PatientID = appointment.PatientID,
23     DoctorID = appointment.DoctorID,
24     RoomID = appointment.RoomID,
25     Status = QueueStatus.Waiting,
26     CreatedAt = DateTime.Now
27 };
28
29 return queueDAL.Insert(queue);
30 }

```

Trong phiên bản hoàn thiện hơn, thao tác này nên được đặt trong transaction ở tầng DAL hoặc stored procedure để đảm bảo hai cập nhật cùng thành công hoặc cùng rollback. Đây là nhận xét kỹ thuật quan trọng vì nó cho thấy nhóm hiểu rủi ro nhất quán dữ liệu, không chỉ viết được nút bấm trên WinForms.

4.6.4 Luồng bác sĩ mở ERM và xử lý hồ sơ

Khi bác sĩ nhận bệnh nhân từ hàng chờ, màn hình ERM được mở để xem toàn bộ dữ liệu y tế. ERM không nên tải mọi thứ bằng một câu truy vấn lớn vì dữ liệu bệnh án có nhiều nhóm: thông tin tổng quan, lịch sử khám, xét nghiệm, hình ảnh, toa thuốc, hóa đơn và tái khám. Cách cài đặt hợp lý là controller/BUS cung cấp từng nhóm DTO riêng, các user control con tự bind phần dữ liệu của mình.

Bảng 4.6.4: Luồng dữ liệu trong ERM

Nhóm dữ liệu	DTO/Control nhận dữ liệu	Cách hiển thị
Tổng quan	PatientOverviewDTO, ucOverview	Label/card thông tin bệnh nhân, sinh hiệu, chẩn đoán gần nhất.
Lịch sử khám	Danh sách encounter, ucHistory	FlowLayoutPanel chứa card từng lần khám.
Xét nghiệm	Danh sách lab result, ucLab	Card kết quả hoặc bảng ngắn theo loại xét nghiệm.
Hình ảnh	Danh sách imaging, ucImaging	Card ảnh, nút mở frmPacsViewer.
Toa thuốc	Prescription và detail, ucPrescription	Card toa thuốc, DataGridView thuốc trong toa.
Thanh toán	Invoice, ucBilling	Card hóa đơn và trạng thái thanh toán.

```

1 public void BindLabResults(IEnumerable<LabResultDTO> results)
2 {
3     flowLayoutPanel1.Controls.Clear();
4

```

```

5     foreach (LabResultDTO result in results)
6     {
7         var card = new ucLabCard();
8         card.Bind(result);
9         card.Dock = DockStyle.Top;
10        flowLayoutPanel1.Controls.Add(card);
11    }
12 }

```

Nhận xét cài đặt: ERM là nơi nên ưu tiên FlowLayoutPanel và card vì bác sĩ cần đọc theo cụm thông tin. DataGridView chỉ nên dùng trong phần có cấu trúc bảng rõ ràng, ví dụ danh sách thuốc trong toa. Nếu mọi dữ liệu bệnh án đều đưa vào DataGridView, giao diện sẽ khô, khó đọc và không phản ánh đúng workflow khám bệnh.

4.6.5 Luồng tạo chỉ định và kỹ thuật viên xử lý request

Luồng cận lâm sàng nổi vai trò bác sĩ và kỹ thuật viên. Bác sĩ tạo request trong ERM; request được lưu xuống database với trạng thái chờ xử lý; kỹ thuật viên mở dashboard để xem request; sau khi nhập kết quả hoặc upload file, request đổi trạng thái hoàn thành; bác sĩ mở lại ERM để đọc kết quả. Đây là luồng liên vai trò, vì vậy trạng thái phải nhất quán và tên trạng thái phải dùng constant.

Bảng 4.6.5: Vòng đời request cận lâm sàng

Trạng thái	Người thao tác	Màn hình	Ý nghĩa
Pending	Bác sĩ	ERM	Chỉ định vừa được tạo, kỹ thuật viên chưa xử lý.
Processing	Kỹ thuật viên	ucTechnicianRequest	Request đã được nhận xử lý hoặc đang nhập kết quả.
Completed	Kỹ thuật viên	Upload/Lab result form	Đã có kết quả hoặc file đính kèm.
Viewed	Bác sĩ	ERM	Bác sĩ đã mở và đọc kết quả.
Canceled	Bác sĩ/quản trị	ERM hoặc màn hình quản lý	Request bị hủy vì nhập sai hoặc không cần thực hiện.

```

1 public bool CompleteLabRequest(int requestId, string resultText)
2 {
3     if (requestId <= 0 || string.IsNullOrEmpty(resultText))
4     {
5         return false;
6     }
7
8     TechnicianRequestDTO request = requestDAL.GetById(requestId);
9     if (request == null ||

```

```

10     request.Status == RequestStatus.Completed)
11 {
12     return false;
13 }
14
15 labResultDAL.Insert(new LabResultDTO
16 {
17     RequestID = requestId,
18     ResultText = resultText.Trim(),
19     PerformedBy = UserSession.EmployeeID,
20     ResultDate = DateTime.Now
21 });
22
23 return requestDAL.UpdateStatus(
24     requestId,
25     RequestStatus.Completed);
26 }

```

Nhận xét cài đặt: đối với request có file, hệ thống cần lưu metadata thay vì chỉ lưu chuỗi đường dẫn tùy tiện. Metadata tối thiểu nên có PatientID, EncounterID, RequestID, FileName, FilePath, FileType, UploadedBy, UploadedAt. Khi bác sĩ xem lại hồ sơ, ERM có thể lọc file theo patient/encounter thay vì dò theo tên file.

4.6.6 Luồng cấp phát thuốc và cập nhật tồn kho

Phân hệ dược sĩ không chỉ hiển thị danh mục thuốc. Luồng nghiệp vụ thật là dược sĩ nhận toa thuốc, kiểm tra trạng thái toa, kiểm tra từng thuốc trong toa, đối chiếu tồn kho, xác nhận cấp phát và cập nhật tồn kho. Nếu tồn kho không đủ, hệ thống phải cảnh báo thay vì trừ âm.

Bảng 4.6.6: Luồng cấp phát thuốc

Bước	Xử lý	Nhận xét cài đặt
1	Lấy danh sách toa chờ cấp phát.	Lọc theo DispenseStatus.Pending để dược sĩ không xử lý nhầm toa đã cấp.
2	Mở chi tiết toa.	Hiển thị thuốc, hàm lượng, số lượng, cách dùng, ghi chú bác sĩ.
3	Kiểm tra tồn kho.	Gọi MedicineDAL/InventoryDAL; nếu thiếu thuốc thì báo lỗi rõ ràng.
4	Xác nhận cấp phát.	Cập nhật trạng thái toa, giảm tồn kho, ghi nhận người cấp phát và thời gian.

Bước	Xử lý	Nhận xét cài đặt
5	Reload dashboard.	Toa vừa cấp phát biến khỏi danh sách chờ hoặc chuyển sang nhóm hoàn thành.

```

1 public bool CanDispensePrescription(int prescriptionId)
2 {
3     List<PrescriptionDetailDTO> details =
4         prescriptionDAL.GetDetails(prescriptionId);
5
6     foreach (PrescriptionDetailDTO item in details)
7     {
8         MedicineDTO medicine = medicineDAL.GetById(item.MedicineID);
9         if (medicine == null ||
10             medicine.StockQuantity < item.Quantity)
11         {
12             return false;
13         }
14     }
15
16     return true;
17 }

```

Nhận xét cài đặt: với nghiệp vụ dược, phần UI nên tách rõ danh sách toa và danh mục thuốc. Danh sách toa là công việc hằng ngày của dược sĩ, còn danh mục thuốc/tồn kho là dữ liệu quản trị. Nếu trộn hai phần vào một màn hình lớn, người dùng dễ thao tác nhầm giữa cấp phát thuốc và sửa thông tin thuốc.

4.6.7 Luồng đồng bộ Supabase, SheetDB và SQL Server

Đồng bộ API là phần dễ viết sơ sài trong báo cáo, nhưng thực tế đây là luồng có nhiều quyết định kỹ thuật. Dữ liệu từ SheetDB thường có dạng bảng giống Google Sheet, có thể thiếu UUID, có cột rỗng hoặc dữ liệu số dưới dạng chuỗi. Supabase lại có mô hình REST rõ hơn và thường có UUID. SQL Server local là nguồn phục vụ WinForms. Vì vậy, ApiSyncBUS đóng vai trò hợp nhất dữ liệu, không chỉ gọi API rồi insert.

Bảng 4.6.7: Chiến lược xử lý xung đột khi đồng bộ

Tình huống	Cách nhận biết	Cách xử lý
Dòng thiếu mã	PatientCode hoặc EmployeeCode rỗng	Bỏ qua và ghi nhận vào kết quả sync để người dùng biết dữ liệu ngoài chưa đạt.
Dòng thiếu tên	FullName rỗng	Bỏ qua vì không đủ thông tin hiển thị và tra cứu.
Sheet thiếu UUID	Có mã bệnh nhân nhưng uuid rỗng	Tìm bản ghi tương ứng trên Supabase/local; nếu có thì patch UUID ngược về Sheet.

Tình huống	Cách nhận biết	Cách xử lý
Local đã có bản ghi	Trùng UUID hoặc trùng code	Update thông tin thay vì insert để tránh trùng bệnh nhân/nhân viên.
Supabase thiếu dòng từ Sheet	Có ở Sheet, không có ở Supabase	Insert lên Supabase nếu dữ liệu hợp lệ.
Sheet thiếu dòng từ Supabase	Có ở Supabase, không có ở Sheet	Ghi ngược về Sheet để hai nguồn ngoài đồng bộ.

```

1 private async Task SyncPatientsToLocalSqlServerAsync(
2     List<ApiPatientDTO> patients,
3     ApiSyncResultDTO result,
4     bool patchSheetWhenMissingUuid)
5 {
6     foreach (ApiPatientDTO patient in patients)
7     {
8         if (string.IsNullOrEmpty(patient.PatientCode) ||
9             string.IsNullOrEmpty(patient.FullName))
10        {
11            result.SkippedPatients++;
12            continue;
13        }
14
15        if (string.IsNullOrEmpty(patient.PatientUuid))
16        {
17            patient.PatientUuid = Guid.NewGuid().ToString();
18
19            if (patchSheetWhenMissingUuid)
20            {
21                await repository.PatchPatientSheetUuidAsync(patient);
22            }
23        }
24
25        bool changed = repository.UpsertLocalPatient(patient);
26        if (changed)
27        {
28            result.LocalPatientsChanged++;
29        }
30    }
31 }

```

Nhận xét cài đặt: không nên đặt logic này trong MockApiSyncForm. Form đồng bộ chỉ nên có nút sync, bảng preview bệnh nhân/nhân viên và vùng hiển thị kết quả. Nếu form tự xử lý UUID, lọc dữ liệu, gọi Supabase và ghi SQL Server, phần tích hợp sẽ rất khó kiểm thử và không thể tái sử dụng cho dashboard khác.

4.6.8 Luồng PayOS, VietQR và cập nhật realtime

Thanh toán QR trong ứng dụng Desktop có một đặc thù: WinForms chạy trên máy nội bộ, không phải server public. Vì vậy, Desktop không phải nơi phù hợp để nhận webhook trực tiếp từ PayOS. Luồng đúng là Desktop gọi backend để tạo payment link; PayOS gọi webhook về backend; Desktop kiểm tra trạng thái qua backend bằng polling hoặc nhận event realtime nếu backend hỗ trợ SignalR.

Bảng 4.6.8: Chi tiết luồng PayOS/VietQR

Giai đoạn	Thành phần	Logic vận hành
Chuẩn bị hóa đơn	PaymentDetail	Lấy encounterId, tổng tiền, tên người mua, nội dung thanh toán.
Tạo giao dịch	Backend PayOS	Nhận request từ Desktop, ký payload bằng checksum key, gọi PayOS.
Hiển thị QR	PayOsPaymentForm	Render qrDataURL hoặc tạo QR từ qrCode/checkoutUrl.
Người dùng thanh toán	Ứng dụng ngân hàng/MoMo QR/VietQR	Quét QR và chuyển tiền theo thông tin PayOS cung cấp.
Webhook	Backend	PayOS gửi callback; backend kiểm tra chữ ký và lưu trạng thái.
Realtime/polling	WinForms	Timer kiểm tra trạng thái; nếu thành công thì dừng timer và cập nhật UI.
Cập nhật hóa đơn	PaymentBUS	Chỉ đánh dấu Paid khi backend/PayOS xác nhận thành công.

```

1 private async void paymentStatusTimer_Tick(object sender, EventArgs e)
2 {
3     paymentStatusTimer.Stop();
4
5     try
6     {
7         PayOsApiResponse response =
8             await GetPaymentStatusAsync(currentOrderCode, CancellationToken.None);
9
10        string status = response?.data?.status;
11        lblStatus.Text = "Trạng thái: " + status;
12
13        if (status == "PAID")
14        {
15            DialogResult = DialogResult.OK;
16            Close();
17            return;
18        }
19
20        paymentStatusTimer.Start();
21    }
22    catch
23    {
24        paymentStatusTimer.Start();
25    }
26 }

```

Nhận xét cài đặt: nếu người dùng bấm nút xác nhận thanh toán trước khi PayOS trả PAID, hệ thống có thể ghi sai doanh thu. Vì vậy, trong bản hoàn chỉnh nên khóa nút xác nhận hoặc chỉ cho phép cập nhật SQL Server khi form PayOS trả DialogResult.OK. Đây là điểm cần ghi rõ trong báo cáo vì nó thể hiện nhận thức về tính đúng đắn của nghiệp vụ tài chính.

4.7 Đánh giá kỹ thuật cài đặt trong chương xây dựng

Từ quá trình cài đặt, có thể rút ra một số điểm kỹ thuật quan trọng của đồ án. Thứ nhất, WinForms được dùng theo hướng nhiều UserControl thay vì nhiều form rời rạc, giúp dashboard giữ được trạng thái và trải nghiệm thống nhất. Thứ hai, DataGridView được dùng cho dữ liệu dạng bảng có thao tác theo dòng, còn FlowLayoutPanel và card được dùng cho dữ liệu cần nhìn nhanh theo nhóm. Thứ ba, tầng DTO–BUS–DAL giúp tách giao diện khỏi SQL Server, làm cho code dễ bảo trì hơn khi số lượng phân hệ tăng. Thứ tư, các tích hợp ngoài như Supabase, SheetDB và PayOS được đặt sau BUS/repository hoặc backend trung gian, tránh để form WinForms phải xử lý toàn bộ API.

Trong giai đoạn hiện tại, DAL chính vẫn dựa trên ADO.NET vì phù hợp với project WinForms và dễ kiểm soát câu SQL. Tuy nhiên, báo cáo đã thiết kế hướng mở rộng Entity Framework để các phân hệ nhiều truy vấn như danh mục thuốc, lịch sử khám, hóa đơn và báo cáo có thể chuyển dần sang ORM. Cách chuyển dần này thực tế hơn việc thay toàn bộ DAL trong một lần, vì hệ thống đang có nhiều chức năng đã chạy ổn định bằng ADO.NET.

5 Kiểm thử và Giao diện kết quả

5.1 Chiến lược kiểm thử

Do đồ án có nhiều vai trò, kiểm thử được tổ chức theo luồng nghiệp vụ thay vì chỉ kiểm tra từng form rời rạc. Mỗi kịch bản kiểm thử bắt đầu từ đăng nhập, mở dashboard theo role, thao tác nghiệp vụ và kiểm tra dữ liệu được cập nhật trong giao diện hoặc database.

Đối với môn WinForms Desktop, kiểm thử không chỉ kiểm tra nghiệp vụ đúng hay sai mà còn phải kiểm tra phản ứng của giao diện. Một form có nghiệp vụ đúng nhưng layout vỡ, bảng không co giãn, nút bị che, event click không hoạt động hoặc UI đứng khi gọi API thì vẫn chưa đạt yêu cầu của phần mềm Desktop. Vì vậy, nhóm kiểm thử theo bốn nhóm:

- **Kiểm thử luồng nghiệp vụ:** đăng nhập, tạo lịch, check-in, khám, xét nghiệm, thanh toán, cấp phát thuốc.
- **Kiểm thử giao diện WinForms:** resize form, chuyển màn hình, mở modal, đóng modal, reload dữ liệu, focus ô nhập và thông báo lỗi.
- **Kiểm thử dữ liệu:** kiểm tra dữ liệu lưu xuống SQL Server, khóa ngoại, trạng thái appointment/queue/payment/request.
- **Kiểm thử tích hợp:** đồng bộ Supabase/SheetDB, upload file, kiểm tra trạng thái thanh toán PayOS ở mức mô phỏng.

5.2 Kiểm thử giao diện WinForms

Bảng sau tập trung vào các lỗi thường gặp trong ứng dụng Desktop WinForms và cách kiểm tra trong đồ án.

Bảng 5.2.1: Kiểm thử giao diện WinForms

Mã	Kịch bản UI	Kết quả mong đợi	Trạng thái
UI-01	Phóng to/thu nhỏ dashboard lễ tân	Menu giữ vị trí, content panel co giãn đúng	Đạt
UI-02	Chuyển nhanh giữa Patient, Schedule, WaitingList, Payment	Panel cũ được xóa, panel mới dock fill, không chồng control	Đạt
UI-03	Mở form modal đăng ký ca làm rồi đóng	Form cha nhận lại quyền thao tác, dữ liệu reload khi OK	Đạt
UI-04	Click dòng trong DataGrid-View lịch hẹn	Hệ thống lấy đúng AppointmentID của dòng được chọn	Đạt
UI-05	Nhập sai dữ liệu số ở sinh hiệu	Form báo lỗi và focus lại ô nhập sai	Đạt
UI-06	Bấm đồng bộ API nhiều lần liên tục	Nút sync bị disable trong lúc xử lý, UI không bị đứng	Đạt mô phỏng
UI-07	Chọn file PDF/MRI bằng OpenFileDialog	Đường dẫn file được hiển thị và dùng cho thao tác upload	Đạt mô phỏng

5.3 Kiểm thử chức năng

Bảng 5.3.1: Bảng kiểm thử chức năng tiêu biểu

Mã	Kịch bản	Kết quả mong đợi	Trạng thái
TC-01	Đăng nhập bằng tài khoản hợp lệ	Mở đúng main form theo vai trò	Đạt
TC-02	Đăng nhập thiếu username hoặc password	Hiển thị thông báo lỗi	Đạt
TC-03	Tìm kiếm bệnh nhân theo tên/mã/số điện thoại	Danh sách bệnh nhân được lọc đúng	Đạt
TC-04	Tạo lịch hẹn với bác sĩ có lịch làm	Lịch hẹn được lưu và hiển thị	Đạt

Mã	Kịch bản	Kết quả mong đợi	Trạng thái
TC-05	Tiếp nhận bệnh nhân trong ngày	Trạng thái lịch/hàng chờ được cập nhật	Đạt
TC-06	Kỹ thuật viên upload kết quả PDF/MRI	Kết quả được gắn vào request tương ứng	Đạt mô phỏng
TC-07	Dược sĩ mở danh sách đơn chờ cấp phát	Danh sách đơn thuốc hiển thị đúng	Đạt
TC-08	Tạo thanh toán và cập nhật trạng thái	Payment chuyển sang trạng thái đã thanh toán	Đạt mô phỏng
TC-09	Đồng bộ dữ liệu API thiếu UUID	Hệ thống báo lỗi rõ nguyên nhân	Đạt

5.4 Đánh giá ưu điểm

- Kiến trúc project rõ ràng, tách biệt UI, BUS, DAL, DTO, Models và Core.
- Cơ sở dữ liệu bao phủ nhiều nghiệp vụ phòng khám, có khóa chính và khóa ngoại tương đối đầy đủ.
- Giao diện chia theo vai trò, phù hợp với cách vận hành thực tế.
- Có các controller phía UI giúp giảm bớt logic trong code-behind.
- Có tích hợp mô phỏng API và PayOS, làm tăng tính thực tế của đồ án.
- Có nhóm Shareforms cho ERM và ca làm, tránh trùng lặp giao diện dùng chung.

5.5 Hạn chế

- Một số namespace và cách đặt tên chưa hoàn toàn thống nhất, ví dụ có lớp dùng BUS.Services, có lớp dùng BUS.
- Một số nghiệp vụ còn mô phỏng, chưa đạt mức sản phẩm triển khai thật như thanh toán realtime, PACS chuẩn y tế hoặc bảo hiểm y tế.
- Kiểm thử hiện chủ yếu ở mức thủ công, chưa có bộ unit test tự động cho BUS và DAL.
- Logic khởi tạo database còn nằm trong LoginForm; nên tách thành bootstrap/service riêng.
- Quản lý mật khẩu cần rà soát để sử dụng PasswordHasher nhất quán thay vì so khớp chuỗi trực tiếp ở mọi nơi.

6 Kết luận và Hướng phát triển

6.1 Kết quả đạt được

Đồ án đã xây dựng được một hệ thống quản lý phòng khám Desktop có cấu trúc nhiều lớp và phạm vi nghiệp vụ tương đối đầy đủ. Hệ thống hỗ trợ đăng nhập theo vai trò, quản lý bệnh nhân, lịch hẹn, hàng chờ, hồ sơ bệnh án điện tử, xét nghiệm, chẩn đoán hình ảnh, đơn thuốc, thanh toán, nhà thuốc, ca làm và đồng bộ API. Mã nguồn được chia project theo chức năng, thuận tiện cho việc mở rộng và bảo trì.

6.2 Hạn chế của phần mềm

Hệ thống vẫn còn một số hạn chế về mức độ hoàn thiện nghiệp vụ và chất lượng kỹ thuật. Một số màn hình có thể mới ở mức mô phỏng, giao diện chưa đồng bộ tuyệt đối, một số truy vấn nghiệp vụ cần tối ưu thêm, và cơ chế bảo mật chưa đạt mức hệ thống y tế thực tế. Ngoài ra, phần kiểm thử tự động chưa được triển khai đầy đủ nên việc đánh giá hồi quy còn phụ thuộc vào kiểm thử thủ công.

6.3 Hướng phát triển trong tương lai

- Chuẩn hóa namespace, đặt tên lớp và cấu trúc thư mục theo domain.
- Tách bootstrap database, cấu hình API và thanh toán thành service riêng.
- Bổ sung unit test cho BUS và integration test cho DAL.
- Hoàn thiện phân quyền theo permission chi tiết, không chỉ theo role.
- Nâng cấp bảo mật mật khẩu, logging và audit trail cho thao tác y tế nhạy cảm.
- Tối ưu ERM, thêm chức năng xuất PDF hồ sơ khám và hóa đơn.
- Hoàn thiện thanh toán realtime và cấu hình triển khai cho môi trường phòng khám thật.

Tài liệu tham khảo

- [1] Microsoft. *Windows Forms documentation*. Available: <https://learn.microsoft.com/dotnet/desktop/winforms/>
- [2] Microsoft. *ADO.NET Overview*. Available: <https://learn.microsoft.com/dotnet/framework/data/adonet/ado-net-overview>
- [3] Microsoft. *SQL Server technical documentation*. Available: <https://learn.microsoft.com/sql/sql-server/>
- [4] Microsoft. *Entity Framework Core documentation*. Available: <https://learn.microsoft.com/ef/core/>
- [5] PayOS. *PayOS payment gateway documentation*. Available: <https://payos.vn/>
- [6] Supabase. *Supabase documentation*. Available: <https://supabase.com/docs>
- [7] SheetDB. *SheetDB API documentation*. Available: <https://sheetdb.io/>
- [8] Nhóm phát triển. *Mã nguồn ClinicManagementSystem. Winforms trong đồ án Desktop quản lý phòng khám*. 2026.

PHỤ LỤC A. HÌNH ẢNH

Phụ lục này tổng hợp tự động các hình ảnh xuất hiện trong phần nội dung chính.

Hình 2.5.1	Kiến trúc phân lớp của hệ thống quản lý phòng khám	8
------------	--	---

PHỤ LỤC B. CÁC THUẬT TOÁN

Phụ lục này tổng hợp tự động các thuật toán giả xuất hiện trong phần nội dung chính.

PHỤ LỤC C. BẢNG, BIỂU

Phụ lục này tổng hợp các bảng, biểu quan trọng của báo cáo.

Bảng 2.2.1	Kỹ thuật layout WinForms sử dụng trong đồ án	5
Bảng 2.6.1	Vai trò các tầng trong kiến trúc DTO–BUS–DAL	9
Bảng 2.9.1	Vai trò các nguồn dữ liệu trong tích hợp ngoài	12
Bảng 2.9.2	Ví dụ cấu trúc sheet bệnh nhân	13
Bảng 3.1.1	Tác nhân và phạm vi chức năng	17
Bảng 3.2.1	Danh sách yêu cầu chức năng chính	17
Bảng 3.3.1	Đặc tả use case đăng nhập	18
Bảng 3.3.2	Đặc tả use case tạo lịch hẹn	19
Bảng 3.3.3	Đặc tả use case tiếp nhận bệnh nhân	19
Bảng 3.3.4	Đặc tả use case khám bệnh	20
Bảng 3.3.5	Đặc tả use case kỹ thuật viên xử lý yêu cầu	20
Bảng 3.3.6	Đặc tả use case cấp phát thuốc và thanh toán	21
Bảng 3.6.1	Nhóm bảng dữ liệu chính	22
Bảng 3.8.1	Từ điển dữ liệu rút gọn	23
Bảng 3.8.2	Từ điển dữ liệu nhóm phân quyền và nhân sự	24
Bảng 3.8.3	Từ điển dữ liệu nhóm tiếp nhận	24
Bảng 3.8.4	Từ điển dữ liệu nhóm lâm sàng	25
Bảng 3.8.5	Từ điển dữ liệu nhóm dược và thanh toán	26
Bảng 4.1.1	Tổ chức project trong solution	27
Bảng 4.1.2	Cấu trúc thư mục giao diện WinForms	27
Bảng 4.1.3	Quy ước thiết kế form/control trong đồ án	29
Bảng 4.1.4	Ứng dụng DataGridView trong các phân hệ	32
Bảng 4.1.5	Ánh xạ vai trò sang giao diện chính	35
Bảng 4.2.1	Phân nhóm DTO trong project	36
Bảng 4.2.2	Nhóm repository DAL và trách nhiệm	38
Bảng 4.2.3	Định hướng chọn ADO.NET hoặc Entity Framework trong DAL . .	39
Bảng 4.2.4	Nhóm BUS và logic nghiệp vụ	42
Bảng 4.2.5	Luồng cài đặt đăng nhập và phân quyền	43
Bảng 4.2.6	Logic BUS cho lịch hẹn và hàng chờ	44
Bảng 4.2.7	Logic BUS của phân hệ kỹ thuật viên	45
Bảng 4.2.8	Thành phần trong project Core	46
Bảng 4.3.1	Nguyên tắc phân tách Designer và code-behind	51
Bảng 4.3.2	Thiết kế giao diện phân hệ lễ tân	52
Bảng 4.3.3	Cấu trúc màn hình ERM của bác sĩ	54
Bảng 4.3.4	Cài đặt nhóm màn hình ca làm việc	56
Bảng 4.4.1	Các bước xử lý dữ liệu API ở tầng DAL/BUS	57
Bảng 4.4.2	Dữ liệu phản hồi PayOS dùng trong WinForms	60

Bảng 4.4.3	Luồng triển khai PayOS trong ứng dụng Desktop	60
Bảng 4.5.1	Trace cài đặt end-to-end của các nghiệp vụ chính	61
Bảng 4.6.1	Diễn giải chi tiết luồng đăng nhập	63
Bảng 4.6.2	Trạng thái giao diện trong luồng tạo lịch hẹn	64
Bảng 4.6.3	Điều kiện và kết quả của thao tác check-in	65
Bảng 4.6.4	Luồng dữ liệu trong ERM	66
Bảng 4.6.5	Vòng đời request cận lâm sàng	67
Bảng 4.6.6	Luồng cấp phát thuốc	68
Bảng 4.6.7	Chiến lược xử lý xung đột khi đồng bộ	69
Bảng 4.6.8	Chi tiết luồng PayOS/VietQR	71
Bảng 5.2.1	Kiểm thử giao diện WinForms	73
Bảng 5.3.1	Bảng kiểm thử chức năng tiêu biểu	73

Bảng phân công nhiệm vụ

Bảng phân công công việc của nhóm được nhóm lưu trữ trực tuyến để thuận tiện theo dõi và cập nhật:

XEM BẢNG PHÂN CÔNG CÔNG VIỆC

Link đồ án

Mã nguồn của hệ thống được lưu trữ trên GitHub để thuận tiện cho việc theo dõi và phát triển:

 **XEM SOURCE CODE HỆ THỐNG**